VIVANTI CONSULTING · FINAL ROUND · 5 OCTOBER 2026

The Vivanti Final Round Bible

Everything for the partner round with Tim Armour: open-ended SQL, Snowflake, data modeling, Spark, your resume, stories, consulting judgment and full mocks. One book, nothing else needed.

Vishal Barvaliya

Data Engineer II · Pod Lead
September 2026

Contents

Part 1: Start Here 4
Part 2: The Round and Your Interviewers 9
Part 3: How to Win the Room 12
Part 4: Your Fact Sheet and Introductions 16
Part 5: Your Career Story and Why Vivanti 22
Part 6: How SQL Actually Works 28
Part 7: SQL Performance 34
Part 8: Practice Schemas and the 15 Patterns 40
Part 9: Twenty Analytical SQL Cases 45
Part 10: Snowflake for a Databricks Engineer 59
Part 11: Data Modeling Masterclass 68
Part 12: Domain SQL Cases in Snowflake 76
Part 13: Analytical Discussions Without Code 84
Part 14: Spark, Databricks and Delta 91
Part 15: Python and Live Coding 98
Part 16: Rapid-Fire Questions 101
Part 17: Resume Defence and Risk Audit 105
Part 18: Line-by-Line Resume Questions 112
Part 19: Project Deep Dives 116
Part 20: SQL Straight From Your Resume 131
Part 21: Follow-Up Chains 136
Part 22: Your Stories 140
Part 23: Leadership and Behavioural Questions 155
Part 24: Consulting, Client and Partner Pressure Questions 162
Part 25: Technical and Architecture Conversations 167

Part 26: Difficult and Challenging Questions 176
Part 27: Compensation and Offer Discussion 180
Part 28: Questions to Ask the Partner 184
Part 29: Mock Interviews 186
Part 30: Revision Sheets, Numbers Card and Interview Day 193

Part 1: Start Here

This is the only book you need

This book replaces every earlier Vivanti guide: the Final Round Guide, the SQL Round Playbook, the Resume Defence Guide, the Complete Guide and the Speaking Guide. Everything useful from them is here, rebuilt around the two people who will actually be in the room, with five major new parts:

- **Part 2:** who Tim Armour and Subhash Peshwa are, and what their backgrounds tell you about the questions.
- **Part 10:** Snowflake for a Databricks engineer, because Snowflake is Tim's core platform.
- **Part 11:** a data modeling masterclass, because modeling is on Tim's current engagements.
- **Part 12:** domain SQL cases in Tim's world (education and healthcare devices), written in Snowflake SQL.
- **Parts 14 and 15:** Spark, Databricks and Delta questions, and Python live coding, in case "hands-on problem-solving" goes beyond SQL.

What you know about the round

Fact	Source
Final interview with **Tim Armour, Partner**, Sydney. **Subhash Peshwa, Director, Bengaluru** is an optional attendee	Calendar invite
Monday 5 October, 7:30–8:30 PM Toronto (Tuesday 10:30 AM Sydney, 5:00 AM India)	Calendar invite
"A **hands-on, technical and problem-solving** focused discussion. Our goal is to see **how you think and approach complex challenges**"	Invite text
Mostly **SQL**: open-ended analytical problems where they want **assumptions and alternatives**, plus **performance**, plus **anything on your resume**	Anisha, on the call
Earlier rounds: you can ask questions and get hints; make it a two-way conversation	Anisha, earlier email
Final rounds at Vivanti have been with a partner or CEO; culture matters; "why Vivanti" and "how do you handle stress" have been asked	Glassdoor candidates

Your working assumption: 60 minutes. About 5–10 minutes of introductions and background, 35–45 minutes of hands-on problems (open-ended SQL, data modeling, performance, probably in a shared document or screen share), 5 minutes of resume probing, 5 minutes for your questions.

How to read this book

Every part is marked with a priority:

- **MUST:** do this even if you run out of time.
- **SHOULD:** do this if you can.
- **IF TIME:** extra depth.

Part	What it covers	Priority
2	The round and your interviewers	MUST
3	How to win the room: talk track, two-version answers	MUST
4	Fact sheet and introductions	MUST
5	Career story and why Vivanti	MUST
6	How SQL works	MUST (Q1, 4, 6, 9, 12, 13, 19)
7	SQL performance	MUST
8	Practice schemas and 15 patterns	MUST (patterns)
9	Twenty analytical SQL cases	MUST (cases 1, 2, 3, 6, 7, 8, 9, 10, 12, 20)
10	Snowflake for a Databricks engineer	MUST
11	Data modeling masterclass	MUST
12	Domain SQL cases in Snowflake	MUST
13	Analytical discussions without code	MUST
14	Spark, Databricks and Delta	SHOULD
15	Python and live coding	IF TIME (SHOULD if Anisha says Python is in scope)
16	Rapid-fire questions	SHOULD
17	Resume defence and risk audit	MUST
18	Line-by-line resume questions	SHOULD
19	Project deep dives	MUST (1–4)
20	SQL straight from your resume	SHOULD
21	Follow-up chains	SHOULD
22	Your stories	MUST (22.1–22.3, 22.13, 22.15, 22.16, 22.19)
23	Behavioural questions	SHOULD
24	Consulting and client questions	SHOULD
25	Architecture conversations	SHOULD (25.1, 25.6, 25.7)
26	Difficult questions	SHOULD

Part	What it covers	Priority
27	Pay and offer	IF TIME (the partner is unlikely to discuss pay)
28	Questions to ask Tim and Subhash	MUST
29	Mock interviews	MUST (Mocks E and F)
30	Revision sheets, numbers card, worksheet, interview-day plan	MUST

The one rule that matters more than any page

Half of every day is solving out loud against a timer. Tim's invitation says the goal is to see *how you think*. That's only visible when you talk while you solve. You already know enough SQL to pass; you proved it in two rounds. What wins this round is structure and narration under pressure, and that only improves with practice, not reading.

Your 5-day plan

You're still working at TD this week, so weekdays assume about 3–4 hours in the evening and the weekend assumes 7–8 hours a day. Adjust to your real schedule, but keep the balance.

Day 1: Wednesday 30 September (3–4 hours)

Time	Task
60 min	**Part 30, the fill-in worksheet.** Settle what the Copilot search didn't: the $80K calculation (if any), the date of the SLA miss, what you designed, and which candidate metrics you can defend. This is the single most valuable hour of the week
30 min	Parts 2 and 3. Understand the room
30 min	Part 4. Say the 60-second and 2-minute introductions out loud, three times each, timed
60 min	Part 9, cases 1 and 2, out loud, 15 minutes each, then compare with the model answers
15 min	Start your **leak list**: a notes page of every mistake you make this week

Day 2: Thursday 1 October (3–4 hours)

Time	Task
75 min	**Part 10, Snowflake.** Read it, then explain the architecture and the Databricks mapping out loud without notes
45 min	Part 6 (how SQL works): answer Q1, 4, 6, 9, 12, 13 and 19 out loud before reading the answers

Time	Task
60 min	Part 9, cases 3, 6 and 7, timed and out loud

Day 3: Friday 2 October (3–4 hours)

Time	Task
90 min	**Part 11, data modeling.** Then design the university model from Exercise A on paper, from scratch, in 20 minutes
60 min	Part 12, domain cases 1 to 4, in Snowflake SQL
30 min	Part 7 (performance): recite the optimisation ladder from memory; answer P1–P4 out loud

Day 4: Saturday 3 October (7–8 hours)

Time	Task
60 min	Part 11, Exercise B (the medical device model) on paper, from scratch
90 min	Part 12, cases 5 to 10
90 min	Part 9, cases 8, 9, 10, 12 and 20
60 min	Part 13 (analytical discussions): scenarios 1, 2, 10 and 11, both versions, out loud
60 min	Parts 17 and 19: the risk audit and deep dives 1–4
60 min	**Mock E** from Part 29, timed, recorded on your phone. Listen back
30 min	Leak list review

Day 5: Sunday 4 October (7–8 hours)

Time	Task
60 min	Redo your three weakest cases from scratch
60 min	Part 14 (Spark and Delta): answer the questions out loud
60 min	Part 22: tell stories 22.1, 22.2, 22.13, 22.15, 22.16 and 22.19 out loud, 90 seconds each
45 min	Parts 26 and 24: difficult and consulting questions, selected
60 min	**Mock F** (Tim’s style: modeling plus Snowflake SQL). Ideally with a Bosscoder mentor or with me playing Tim
45 min	Part 15 (Python), only if Anisha said Python is in scope. Otherwise Part 16, rapid fire

Time	Task
30 min	Part 28: choose your questions. Part 30: write your numbers card by hand

Day 6: Monday 5 October (interview at 7:30 PM)

Time	Task
Morning or lunch	Two cases out loud (one from Part 9, one from Part 12). Nothing new
After work	Revision sheets 1, 2, 6 and 10 in Part 30
6:45 PM	Set up: camera, microphone, plain editor, water, notifications off
7:15 PM	Sheet 9 (the final 15 minutes)
7:25 PM	Join the call early. Breathe. You’ve cleared every round so far

Part 2: The Round and Your Interviewers

Tim Armour, Partner (your interviewer)

From his public LinkedIn profile:

Period	Role	What it tells you
Nov 2013 – May 2023	Servian: Associate Consultant → Consultant → Senior Consultant → Managing Consultant → Director → Associate Partner (Sydney)	He came up through every consulting level. He knows exactly what a good consultant looks like at each stage, because he's been one
Feb 2022 – May 2023	Associate Partner, Servian. Tagged: Snowflake, AWS	Senior leader in a data consultancy on Snowflake
Dec 2023 – Apr 2024	Principal, BluSnow	BluSnow is the Snowflake partner that became part of Vivanti
May 2024 – present	Partner, Vivanti (Sydney)	Your interviewer's current role
Sep 2024 – Jan 2025	Vivanti Lead at a university client. Tagged: Snowflake, Power BI	Hands-on engagement lead, analytics and reporting
Feb 2025 – present	Vivanti Lead at a medical device client. Tagged: Snowflake, data modeling	Hands-on engagement lead now, modeling focus
Education	BSc Computer Science (UNSW); MBA in Technology (AGSM)	Technical foundation plus business training
Skills shown	Informatica, Power BI	Classic enterprise data and BI background
Headline	"Deriving value from your data"	He thinks in business value

What this means for your preparation:

1. **Snowflake is his home.** Expect SQL written the Snowflake way (QUALIFY, FLATTEN on semi-structured data, DATEADD) and questions about how you'd do things on Snowflake. You've never used it in production, so say that honestly and map it to Databricks (Part 10).
2. **Data modeling is central to his current work.** Expect "design a model for this" followed by "now query it." Part 11 is built for this.
3. **BI and analytics are his world** (Power BI, university reporting). He cares about the grain, clean star schemas, and metrics that business users trust.
4. **He's still hands-on as an engagement lead.** He'll notice immediately whether your SQL and modeling are real.
5. **He thinks in value.** Every technical answer should end with what it means for the client.
6. **His clients aren't banks.** Present your regulated-banking experience as transferable discipline (correctness, lineage, reconciliation), which matters just as much in healthcare and education.

> **Don't mention his specific clients by name**, and don't say you studied his profile in detail. It's normal to know his general focus ("I know Vivanti has a strong Snowflake practice"). Naming his current client can feel intrusive.

Subhash Peshwa, Director (optional attendee)

Period	Role	What it tells you
Jun 2014 – Nov 2016	Senior Systems Engineer, Infosys (American Express account)	Started in large IT services, on a bank account, like your Cognizant path
2016 – Nov 2023	Servian: Data Engineer → Senior Data Engineer (data migrations; data-intensive applications) → Google Cloud Trainer → Principal Engineer (data platform modernisation), Bengaluru	Deep hands-on data engineering, especially migrations and modernisation
Nov 2023 – May 2025	Data Engineering Senior Specialist, Telstra, Bengaluru	Large-enterprise data engineering
May 2025 – present	Director, Vivanti, Bengaluru	Very likely leads the Bengaluru team you'd join
Education	BE (Dayananda Sagar); MS Computer Science (Georgia Tech)	Strong engineering foundation
Certifications	GCP Professional Data Engineer; Azure Data Engineer Associate	Multi-cloud
Now	Hiring a Service Delivery Lead for data and cloud platforms	Bengaluru is building delivery and managed-services capacity

What this means:

1. If he joins, he's probably your **future manager or skip-level**. He'll judge whether you'd be a strong, independent engineer on his team.
2. **Modernisation and migration** are his specialism. Your phased ADF-to-Workflows migration, legacy modernisation and reconciliation stories will resonate (Parts 19 and 22).
3. He came through **a large IT services firm onto a bank account**, then consulting. He understands your situation well. Don't be defensive about Cognizant or your move.
4. He's an engineer at heart. If he asks questions, expect **practical, hands-on** ones: pipelines, Spark, migration, cloud.

What the partner is really deciding

A partner owns client relationships and revenue. Underneath every question, Tim is asking: **"Can I put this person in front of a client next month, and will they make Vivanti look good?"**

That breaks into six judgments:

1. **Structure:** do you turn a vague problem into clear, measurable steps?
2. **Business sense:** do your definitions and assumptions fit the business?
3. **Correctness:** grain, joins, NULLs, duplicates and ties handled.
4. **Performance judgment:** do you know how SQL runs, what makes it slow, and what fixing it costs?
5. **Credibility:** does your resume hold up under follow-up questions?
6. **Communication:** can you explain it to an executive in plain words, calmly?

What they already assume

You've cleared the assessment, screening and SQL rounds. They assume your **basic SQL is solid**. They aren't re-testing basics. They're testing **depth, judgment, structure and credibility**.

About the Reddit reply you received

It predicted a "client-readiness assessment wrapped in an analytical discussion." That matches everything else, and this book covers all of it. But treat it as unverified: it was a promotional comment for interviews.chat, an AI tool that feeds candidates answers during live interviews.

> **Never use any real-time AI assistant in the interview.** It's dishonest, a partner conversation exposes it immediately (delays, eyes off-screen, polished answers that collapse at "why?"), and Vivanti's first value is Integrity. Being caught would end the process and follow you.

Vivanti in one page

- **What they do:** data, AI and cloud consultancy, "client-first, consultant-led." Founded by Tony Nicol after he sold his previous consultancy, Servian, to Cognizant. Both your interviewers are ex-Servian.
- **Where:** Sydney headquarters, plus the UK, US and Bengaluru. The Bengaluru centre is new, so early joiners help shape it.
- **Size:** about 227 people as of September 2026.
- **Practices:** Snowflake (via BluSnow), Databricks, Google Cloud, platform engineering and DevOps.
- **Values, ICARE:** Integrity (do the right thing for the client, always), Collaboration, Accountability (no-blame, but own and learn), Respect (listen even when you disagree), Excellence (never settle).
- **Philosophy:** consultants act in the client's best interest, the way a doctor or lawyer would.
- **Learning:** the Academy (internal training), paid certifications, guilds.

Part 3: How to Win the Room

The five messages to reinforce all conversation

Every answer should support at least one of these. If an answer doesn't, it's probably too long or off-track.

1. **"I own production systems where correctness matters."** 150 pipelines, regulators, a hard 6 AM deadline.
2. **"I turn technical work into business outcomes."** Cost saved, deadlines protected, errors caught, onboarding faster.
3. **"I lead through influence and teaching."** Pod Lead without authority; teams migrated without mandate; a pod that can deploy without me.
4. **"I'm honest and precise."** Clear about my role, my numbers and what I don't know.
5. **"I'm ready for consulting."** I explain trade-offs, adapt to the client, and learn new stacks fast.

Balancing depth, impact, communication and leadership

Use the **headline, then layers** rule:

1. **Headline (1 sentence):** the answer or outcome.
2. **Business impact (1–2 sentences):** why it mattered.
3. **Technical depth (2–4 sentences):** only enough to prove you did it.
4. **Your role (1 sentence):** "I" for your decisions, "we" for team outcomes.
5. **Stop.** Let the partner pull more if they want it.

A partner who wants depth will ask "how exactly?" A partner who gets a 4-minute monologue stops listening at minute two.

Confident without being rehearsed or arrogant

- **Pause before answering.** Two seconds of thought reads as considered, not slow.
- **Use your own words.** If a script sentence feels unnatural, change it.
- **Admit limits plainly.** "I haven't used Fabric in production" sounds confident. Hedging sounds nervous.
- **Credit others.** "My manager backed the phased plan" makes your story more believable.
- **Don't oversell.** Let the facts carry weight; avoid "I single-handedly," "revolutionised," "world-class."

Executive presence in a virtual interview

- **Camera at eye level**, face well lit from the front, plain background.
- **Look at the camera** when making a key point, not at your own video.
- **Slow down.** Aim for about 130 words a minute. Nerves speed you up.
- **Structure out loud:** "Three things…" then actually give three.

- **Summarise at the end** of long answers: “So in short…”
- **Hold your ground calmly** if challenged: “That’s a fair challenge. Here’s how I’d think about it.”

Green flags and red flags for a consulting partner

Green flags	Red flags
Asks one or two sharp clarifying questions	Starts solving before understanding the goal
States assumptions and trade-offs	Presents one answer as the only answer
Precise about personal role and numbers	Inflates, then retracts under follow-up
Translates tech into business language	Hides behind jargon
Owns mistakes with a systemic fix	Blames others or the system
Curious about the client and the business	Only interested in technology
Calm when interrupted or challenged	Defensive or rattled
Asks thoughtful questions about the work	Asks only about pay and leave

How a partner round differs from a normal technical round

Normal technical round	Partner round
“Write a query that returns X”	“The client thinks revenue is dropping. What would you do?”
Correct syntax is the goal	Correct thinking is the goal; syntax slips are forgiven
Interviewer gives a clean schema	You may have to ask what tables exist and what a row means
One right answer	Several defensible answers. Your assumptions decide which
Depth on one tool	Breadth, trade-offs and business impact
Rarely interrupted	Interruptions, changed requirements, “now explain that to a CFO”

The 7-step talk track (use it on every problem)

Step	What you do	What you say
1. Clarify	Restate the question and the goal	“So the goal is to find customers likely to stop buying, so marketing can target them. Right?”
2. Define	Turn vague words into measurable ones	“I’ll define churned as no purchase in the last 90 days.”
3. Assume	State data assumptions	“I’m assuming one row per order, one currency, and that cancelled orders are flagged.”
4. Plan	Name the grain and the steps before writing	“Step one, last order per customer. Step two, compare with today. Step three, filter.”
5. Write	Build it CTE by CTE, narrating	“This CTE gives one row per customer with their last order date.”

Step	What you do	What you say
6. Validate	Walk through edge cases	“A customer with one order ever still appears, which is what we want.”
7. Extend	Alternative and performance	“Alternatively, use each customer’s own buying rhythm. On a billion rows I’d make sure the date filter prunes partitions.”

The golden rule: never go silent for more than 20 seconds. If you’re thinking, narrate: “I’m deciding between a window function and a self-join. The window scans once, so I’ll use that.”

How to clarify without over-asking

Ask only questions whose answer **changes what you write**. Good clarifying questions are about:

- **The goal:** “Who will use this, and what decision will they make?”
- **The grain:** “Is one row one order, or one item?”
- **Definitions:** “Does ‘active’ mean logged in, or purchased?”
- **Edge cases that change the answer:** ties, refunds, time zones, late data.

Then commit: “I’ll assume X. If that’s wrong, the change is small, just this filter.”

If the partner says “you decide,” that’s a test of judgment. Pick the definition that fits the business goal, say why, and move on.

When you don’t know

> “I haven’t used that specific feature. Here’s how I’d expect it to work based on [the principle], and here’s how I’d verify it before relying on it.”

Partners respect this far more than bluffing. One exposed bluff makes them doubt everything else you’ve said.

Phrases that score well

- “Before I write anything, what does one row in this table represent?”
- “There are two reasonable definitions here. I’ll go with X because it matches the goal, and I can switch to Y.”
- “Let me sanity-check this with a tiny three-row example.”
- “This works as asked. If the data were ten times bigger, here’s what I’d change.”
- “Before sharing this number with a client, I’d reconcile it against a total they already trust.”
- “The trade-off is cost versus freshness. Which matters more to the business here?”

Two versions of every answer: technical and executive

The partner may say “now explain that to the CFO.” Practise switching. The executive version drops jargon, leads with the business outcome, and is two sentences at most.

Topic	Technical version	Executive version
Join fan-out	“The fact-to-dimension join was many-to-many, which multiplied rows and inflated totals. I fixed the dimension grain and validated key uniqueness.”	“The report double-counted because two tables were combined at the wrong level of detail. We fixed it and the totals now match finance.”
Partition pruning	“The filter wrapped the date column in a function, so the engine couldn’t skip partitions and scanned 5 TB.”	“The query read the whole history to answer a question about one month. Now it reads only that month, so it’s faster and cheaper.”
Cluster right-sizing	“Across 170 runs, peak CPU was under 19% but memory peaked at 133 GiB, so I moved to a smaller profile that kept memory headroom: runtime up 15 to 25 percent, compute down a projected 50 to 65 percent.”	“We were paying for computers that sat idle. We matched capacity to what each job actually needs, cutting compute by a projected half or more on over-sized jobs, without risking deadlines.”
Data Vault	“Hubs, links and satellites give insert-only history with record source and load timestamps on every row.”	“We can show a regulator exactly where every number came from and what it looked like on any past date.”
Broadcast OOM	“Raising the broadcast threshold caused a table too large for executor memory to be broadcast, triggering an OOM.”	“A tuning change I made caused a job to fail. I reverted it, fixed the process so it can’t recur, and owned it openly.”
Watermark	“A 5-minute watermark bounds state; events later than that are dropped from streaming aggregates.”	“The live view waits five minutes for stragglers. Anything later appears in the overnight numbers, which are the official ones.”
Skew	“One key held most of the rows, so one task ran far longer than the rest.”	“One very large customer made one part of the job do all the work. We spread that work out.”
Liquid Clustering	“It replaces fixed partitioning with clustering keys that can change without a rewrite.”	“We reorganised how data is stored so queries skip what they don’t need, without downtime.”
Reconciliation	“Full outer join on keys, classify missing and mismatched rows, with null-safe comparison.”	“We compared the old and new systems record by record and proved they match before switching.”
Idempotent loads	“MERGE on business keys with dedup makes reruns safe.”	“If a job fails and reruns, nothing is duplicated, so the numbers stay right.”

Part 4: Your Fact Sheet and Introductions

Every answer in this book is built from the facts on this page. If you say something in the interview that isn't here, you're improvising, and that's where credibility slips.

Your factual profile (the source of truth for every answer)

Every script in this guide is built from these facts. If you say something in the interview that isn't on this page, you're improvising, and that's where credibility slips.

Area	Fact to use
Formal title	Data Engineer II, TD Bank, Toronto
Leadership role	Pod Lead: technical lead of the pod, **no direct reports**. Code reviews, releases, technical decisions, mentoring
Experience	At TD since May 2023, plus an earlier streaming internship at Mandelbulb Technologies (Sep–Dec 2021). Say "close to four years"
Platform	EDF2 Financial Reporting Platform, retail lending. Config-driven Databricks lakehouse on Delta Lake and ADLS, orchestrated by ADF and Databricks Workflows
Scale	15+ source systems; 150+ production workloads in your delivery and support scope, including about 161 production curation ETL jobs; 48 downstream reporting outputs; 35 monitored FRP link jobs; multi-year history; nightly Data Vault dependency chain with a hard morning SLA (6 AM)
Data Vault	SHA-256 hash keys, deterministic key generation, record source (source attribution), audit lineage, point-in-time and bridge concepts, multi-source integration
Pod responsibilities	Story readiness, sprint planning, PR reviews, defect triage, release readiness, production deployments, post-production validation, cross-team coordination
Consumers	Regulatory reporting to OSFI, the Bank of Canada, CDIC and Statistics Canada
Control plane	SQL Server control tables keyed by pipeline ID (registry, dependencies, run state); JSON job configs and YAML environment configs in Git
Data quality	Rules-driven PySpark engine: key, null-rate and referential checks, quarantine routing, freshness monitoring, volume anomaly alerts
Optimisation	TRIM removed from join keys after a lowercase guarantee from the source team; broadcast tuning; skew handling; right-sizing against the critical path; shuffle about 500 GB to 200 GB; runtime up to 60% faster. **Both are candidate metrics:** use only with a baseline, an after state and a validation method
Right-sizing (verified case)	170 production runs: 99.4% success, 24.6 min average, 32 min P95; peak CPU 18.92%, peak memory 133.44 GiB on Standard_E32ds_v4 (32 vCPU, 256 GB). Recommended a smaller profile keeping memory headroom: runtime +15–25% expected, compute −50–65% projected. Lesson: CPU alone never drives downsizing
Right-sizing (counter-case)	Large retail-loan satellite: 51 runs, 94.1% success, 112 min P95, 131 GiB peak memory, 5-way UNION ALL, 65 window functions, hundreds of joins. Rejected aggressive downsizing; kept E32 family; reduced workers conservatively

Area	Fact to use
$80K (resume)	**Not verified by the Copilot search.** Only quote it if you can show how it was calculated; otherwise lead with the verified case
Delta optimiser	Candidate analysis (size, file count, average file size, benefit estimate → optimise / review / skip); 128 MB target file size; retries, batch sizing, per-partition optimisation, file comparisons; runbook, DEV testing, stories, deployment planning
Synapse	Dedicated SQL Pool concurrency caused 30+ minute queue waits; materialising to ADLS removed them
Migration	Phased ADF to Databricks Workflows migration, one source system at a time, which you argued for
Liquid Clustering	Cross-team migration off date partitioning; teams owned 50–70 pipelines each; shared tracker; six-week deadline, done by week five, no downtime
Failure story	Raised the broadcast threshold from the 10 MB default to about 80 MB on a 10-node, 1.2 TB cluster; caused an out-of-memory failure in production; owned and reverted it
Incident story	A consumption job failed with UNRESOLVED_COLUMN and caused an SLA miss; development support was needed; **root cause not confirmed** (possible: schema or source change, deployment mismatch, broken view, refactoring defect)
Other failures seen on the platform	ConcurrentAppendException, TABLE_OR_VIEW_NOT_FOUND, AlreadyExistsException, DELTA_PATH_DOES_NOT_EXIST, DELTA_REPLACE_WHERE_MISMATCH, cluster termination, partial or empty source files, bad data, trigger dependency failures, ingestion delays, PGP failures, catch-up failures, duplicate job runs, schedule configuration issues. **Not all fixed by you: say "our platform experienced"**
Prioritisation story	50 open defects before a release; fixed about 20 upstream hub and link defects first; 12–13 downstream defects closed automatically; became standard process
Process improvement	Built a Delta time-travel diff utility for the shared EDF reference table, replacing error-prone manual pre-release comparisons that had caused failures across teams
Awards	Ten TD recognition awards from six colleagues, including a Legendary Award from Varun Khanna ("Execute with Speed and Impact") and a Win Together award from Chantal Paquette for building the pod's deployment knowledge
Mentoring	Onboarded and mentored engineers, including offshore colleagues. Resume says 10+: be ready to name them. Mentoring is not direct reports
Other metrics (candidate)	25% less data staleness; ~75% less onboarding effort. **Candidate metrics:** use only with baseline, after state, comparison window and affected datasets
Portfolio	Real-time transaction reconciliation: Kafka, Spark Structured Streaming, Iceberg, Airflow; 5-minute watermark and windows; batch recomputation; MATCH/ MISMATCH via full outer join
Writing	DataWithVishal on Medium: 290+ articles, about 8,500 subscribers
Certifications	Databricks Data Engineer Professional, Databricks Spark Developer, DP-203 (retired by Microsoft in 2025), DP-700, Astronomer Airflow
Other interviews	Lyft Data Engineer: six-round loop, final-round non-offer; time pressure in the SQL coding round
Relocation	Canadian work permit ends mid-October 2026; relocating to Bengaluru

Area	Fact to use
Snowflake	**No production experience.** Say so honestly, then map the concepts (Part 10)
Parallel track	Cognizant rebadge in progress (verbal 16.5 LPA fixed; you asked for a grade review and anchored at 30)

Details that must NOT go into your answers

These came up across our sessions. Leave them out unless you've verified them:

- **"15+ PB" or any TB-per-day figure:** unverified. Use workload counts, run counts, P95 runtimes and memory evidence instead. They're more defensible.
- **"$80K a year" as a realised saving:** the Copilot search found a projected 50–65% reduction on an analysed workload, not the $80K calculation. Lead with the verified case.
- **"An upstream schema change caused the UNRESOLVED_COLUMN failure" or "3 AM":** neither is confirmed. Say the root cause wasn't confirmed.
- **25%, 75%, 60% and 500→200 GB:** candidate metrics. Only with baseline, method and comparison window.
- **"99.9% SLA":** you have a documented SLA miss. Say "consistently met the deadline" and own the miss.
- **"Architected" or "led the enterprise transition":** only if the design is actually yours. Say "I own and evolved."
- **"Trained" Copilot agents:** they were **grounded** on repositories.
- **The 30% QE improvement:** that's Radhey's. Never claim it.
- **"Senior" as a title:** your title is Data Engineer II.
- **Databricks Professional certification:** your Credly profile showed it as expired in an earlier session. Renew it or describe it accurately if asked.
- **RRMDM, SAS decomposition specifics, source-to-target mapping automation details, Rahona onboarding:** these were listed in your prompt but **no details exist anywhere in our project history**. Part 22 gives you a structure to fill with your real details. Don't speak about them until you have.

30-second introduction

> "I'm Vishal. I'm a Data Engineer II and Pod Lead at TD Bank in Toronto. I own a regulatory reporting platform on Databricks: about 150 production pipelines that have to finish by 6 AM every night, because the output goes to Canadian regulators. I'm relocating to Bengaluru, and I'm excited about Vivanti because I want to take that production depth to a range of clients."

60-second introduction

"I'm Vishal, a Data Engineer II and Pod Lead at TD Bank in Toronto, with close to four years in data engineering.

My main responsibility is the Financial Reporting Platform for retail lending. It's a config-driven lakehouse on Databricks and Delta Lake, running about 150 pipelines from fifteen-plus source systems through a Data Vault model. It runs every night with a hard 6 AM deadline, because the numbers go to OSFI, the Bank of Canada and other regulators.

My work splits into three parts: keeping that chain reliable, making it faster and cheaper (for example, right-sizing clusters from execution evidence: runtime, P95, CPU and memory, rather than guesswork), and leading the pod technically through code reviews, releases and mentoring.

I'm relocating to Bengaluru, and consulting is the next step I want: bringing that experience to different clients and stacks."

2-minute "Tell me about yourself"

"I'm Vishal. I'm a Data Engineer II and Pod Lead at TD Bank in Toronto, and I've been in data engineering for close to four years, starting with a streaming internship working on vehicle telematics with Kafka and Spark.

At TD, I own the Financial Reporting Platform for retail lending. It's a config-driven lakehouse on Databricks, Delta Lake and Azure. About 150 pipelines take data from fifteen-plus source systems, model it in Data Vault, and produce the numbers TD reports to OSFI, the Bank of Canada, CDIC and Statistics Canada. The whole chain runs overnight and has to finish by 6 AM, so reliability and correctness are the core of my job.

A few things I'm proud of. I made the platform faster and cheaper by working from evidence. I tuned Spark (join keys, broadcast joins, skew) and right-sized clusters from execution data: runtime, P95, CPU and memory, not CPU alone. On one over-provisioned workload that meant a projected 50 to 65 percent compute reduction; on a much heavier one, the same evidence told me not to downsize. That performance and cost work was recognised with TD's Legendary Award. I also led changes across teams without authority over them, like a phased migration from ADF to Databricks Workflows and a move to Liquid Clustering that other teams had to adopt.

As Pod Lead I don't have direct reports, but I own code reviews, releases and technical decisions for the pod, and I've trained the team to run deployments themselves.

Outside work, I write about data engineering on Medium for about 8,500 readers, which is honestly how I learn.

I'm relocating to Bengaluru, and I want to move into consulting: to take regulated, production-grade experience to clients who need it, and to broaden into Snowflake, GCP and new problems. That's what brought me to Vivanti."

Client-facing introduction (as if meeting a Vivanti client)

> “Hi, I’m Vishal, a data engineer with Vivanti. My background is in banking: I’ve spent the last few years running a regulatory reporting platform on Databricks for one of Canada’s largest banks, where the data has to be correct and on time every single night. So I tend to think about reliability, auditability and cost from day one. I’m looking forward to understanding what you’re trying to achieve, and then we can talk about how to get there.”

Technical introduction (for an engineer or architect)

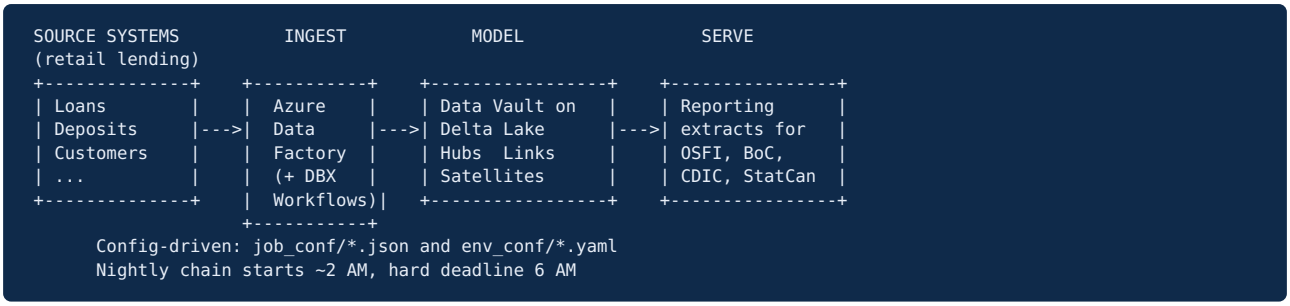
> “I work mostly in PySpark, Databricks and Delta Lake on Azure. My platform is metadata-driven: pipelines are registered in SQL Server control tables and configured in JSON and YAML, and a shared engine runs them. We model in Data Vault 2.0, with hubs, links and satellites, and build reporting on top. I’ve spent a lot of time on Spark performance (shuffle reduction, broadcast joins, skew, AQE, Liquid Clustering) and on reliability: a rules-driven data quality engine with quarantine, freshness and volume checks. On the side, I’ve built a streaming reconciliation project with Kafka, Structured Streaming and Iceberg.”

Executive-level introduction (for a CFO or business leader)

> “I make sure a bank’s regulatory numbers are correct, on time and traceable. My platform delivers the data behind TD’s retail lending reports to regulators every night by 6 AM. Beyond keeping it reliable, I’ve made it cheaper to run by sizing compute to evidence, and faster to extend, so new data sources take a fraction of the effort they used to.”

Draw your platform in 30 seconds

Executives often say “walk me through the architecture.” Draw or describe it left to right, one layer at a time.



Use your real layer names if they differ. The pipelines are registered in SQL Server control tables and configured with JSON and YAML files (see Part 19, deep dive 6). Say it in this order:

1. **Where data comes from:** regulated retail lending source systems.

2. **How it lands:** ingestion orchestrated by ADF, moving to Databricks Workflows.
3. **How it is modeled:** Data Vault on Delta Lake. Hubs for business keys, links for relationships, satellites for history.
4. **What it feeds:** regulatory extracts with a hard 6 AM deadline.
5. **What makes it different:** it is config-driven, so pipelines are defined in JSON and YAML rather than hand-coded one by one.

Part 5: Your Career Story and Why Vivanti

Walk me through your experience

> "I started with an internship at Mandelbulb Technologies in late 2021, on a vehicle telematics platform. That was my first real exposure to streaming: Kafka, Spark Structured Streaming, and what happens when topics are under-partitioned for the load.
>
> Then I moved to Canada for a postgraduate diploma in Big Data Analytics at Lambton College, and joined TD Bank in May 2023.
>
> At TD I've been on the Enterprise Data Foundation, specifically the Financial Reporting Platform for retail lending. I started by building and supporting pipelines, and over time took ownership of the platform's reliability and performance: the nightly chain, optimisation, cost, data quality. Then I became Pod Lead, where I own code reviews, releases and technical direction for the pod.
>
> So the arc is: streaming foundations, then deep production ownership in a regulated bank, and now I want to take that to consulting."

Key message: a clear, upward arc, not a list of jobs.

> Confirm the Lambton and Mandelbulb details are exactly right before saying them. If the move to Canada was for a different reason, say that instead.

Why did you choose data engineering?

> "I like problems where correctness really matters and you can see the result. Data engineering is where that happens. If a pipeline is wrong, a report to a regulator is wrong. If it's slow, a deadline is missed. I also enjoy the mix: part software engineering, part understanding the business, part performance puzzle. The moment that hooked me was seeing a Kafka pipeline fall behind in my internship, and understanding that the fix was about partitions and parallelism, not just more hardware."

What have you worked on at TD Bank?

> "One platform, in depth: the Financial Reporting Platform for retail lending. Concretely, I've worked on four things. First, the pipelines themselves: Data Vault loads on Databricks and Delta, driven by configuration. Second, performance and cost: Spark optimisation and cluster right-sizing. Third, reliability: the data quality engine, freshness and volume monitoring, and incident handling. Fourth, platform change: migrating orchestration from ADF to Databricks Workflows in phases, and moving teams to Liquid Clustering."

What are your current responsibilities?

> "Day to day: I make sure the nightly chain completes correctly by 6 AM, and I'm the first technical escalation when it doesn't. I review code and own releases for the pod through our CI/CD. I plan sprint work in Jira with the team. I work with source-system teams and downstream reporting teams on changes. And I mentor the engineers on the pod, including onboarding offshore colleagues."

Are you actually a pod lead?

This question is testing whether you'll overclaim. Answer precisely.

> "Yes, and let me be precise about what that means at TD. My formal title is Data Engineer II. Pod Lead is a technical leadership role: I don't have direct reports and I don't do performance reviews. What I own is the technical side of the pod: code reviews, release decisions, sprint planning with the team, and being the person the pod comes to for technical direction. I've also trained the team to run deployments themselves, which was recognised with a Win Together award."

Avoid: "I manage a team," "my reports," "I lead a team of 10."

What is the scale of the platform you support?

> "About 150 production pipelines, from fifteen-plus source systems in retail lending: loans, deposits, customers and so on. They run as a dependency chain every night, starting around 2 AM, and must finish by 6 AM. The output feeds regulatory reports to OSFI, the Bank of Canada, CDIC and Statistics Canada. So the scale that matters most isn't just volume; it's that every one of those pipelines has to be right, every night, on a fixed deadline."

Avoid: "15 petabytes." If pressed on volume: "I'd want to give you the exact nightly volume rather than an estimate. What I can tell you precisely is the pipeline count and the deadline."

What business outcomes does your work enable?

> "Three main ones. First, compliance: the bank meets its regulatory reporting obligations on time, with numbers it can trace back to source. Second, cost: right-sizing compute from execution evidence. On one over-provisioned workload, a projected 50 to 65 percent compute reduction. Third, speed of change: because the platform is config-driven, onboarding a new source takes roughly 75% less effort than building pipelines by hand, so the business gets new data faster."

What is your strongest technical area?

> “Spark and Databricks performance on production workloads. I’m good at finding where time and money actually go (the Spark UI, query plans, run logs) and fixing the cause rather than adding hardware. The best example: join keys were wrapped in TRIM, which Spark had to compute on every row. I got a guarantee from the source team that the values were already clean, removed the TRIM, and together with broadcast and skew work, shuffle dropped from about 500 GB to about 200 GB.”

What differentiates you from another candidate with similar experience?

> “Three things. First, I’ve owned production in a regulated environment, with a hard deadline and regulators as the consumers. That builds habits (reconciliation, lineage, idempotent reruns) that many engineers only learn later. Second, I’ve worked on cost and performance, not just features, with measurable results. Third, I explain things: I’ve written 290-plus articles, and at work I’ve taught the pod and partner teams how the platform works. For consulting, that last one matters a lot.”

Why does your experience have more depth than the years suggest?

> “Mostly because of ownership. On a platform like mine, you can’t specialise narrowly. When the chain fails in the middle of the night, you have to understand ingestion, modeling, Spark, orchestration and the business impact all at once. I’ve also been the person making cross-team changes, like the Liquid Clustering migration and the phased orchestration migration, which usually sit with more senior people. And I learn in public. Writing about a topic forces me to understand it properly. I’d rather let my answers today show the depth than argue about years.”

Avoid: claiming more years than your dates show.

Why Vivanti and why consulting

Why Vivanti?

> "Three reasons. First, the work: Vivanti is focused on data and AI, and partners with Databricks, Snowflake and Google Cloud. I'm deep in Databricks and I want to broaden into Snowflake and GCP, and a consultancy built around those platforms is the right place to do that.
>
> Second, the way you work with clients. The idea that consultants advocate for what's genuinely right for the client, like a doctor or lawyer would, matches how I already work. At TD I've pushed back when a plan was risky, with evidence, and that's the kind of consultant I want to be.
>
> Third, timing. The Bengaluru team is new, so joining now means I can help shape it, and the Academy and certification support match how I like to learn."

30-second version: "The platforms you work with are exactly where I want to grow, the client-first way you work matches how I already work, and joining the Bengaluru team early means I can help build it."

Why consulting?

> "I've spent close to four years going very deep on one platform, and I've learned a lot from it. What I want next is breadth: different clients, industries and stacks, and problems I haven't seen. Consulting is the fastest way to get that. I also enjoy the part of my job where I explain a platform to another team or talk someone through a trade-off, and in consulting that's central rather than occasional."

Why move from TD Bank to Vivanti?

> "TD has given me a strong foundation, and I'm grateful for it. My move is driven by two things: I'm relocating to India, and I want to shift from one internal platform to consulting across many clients. Vivanti gives me both, and it's in exactly the technologies I work with."

Why leave Canada?

> "My Canadian work permit ends in October, and I've decided to build the next part of my career in India rather than try to extend my time there. Bengaluru is where a lot of the most interesting data work is happening now, so it's a move I'm choosing positively, not just a necessity."

Key: truthful about the permit, positive about India. Don't dwell on it.

Why return to India?

> “It’s partly timing, with my permit ending, and partly choice. Bengaluru has become one of the strongest places in the world for data engineering, with global clients served from here. For someone who wants to work across international clients, it’s a great base.”

Why not remain with a large bank?

> “Banking has taught me discipline around correctness and control, and I’ll keep that. But inside one bank, you see one way of doing things. I want to see many: different architectures, different constraints. And I think my banking experience is more valuable spread across several clients, including banks, than inside one.”

Why not join Cognizant?

Never criticise Cognizant. Answer about fit, not flaws.

> “I do have a path with Cognizant. It’s a rebadge that would keep me on my current account, doing very similar work. It’s a good option, but it’s more of the same. Vivanti offers what I’m actually looking for: consulting across different clients, and a data-and-AI focus with Databricks, Snowflake and GCP. So if Vivanti works out, it’s my clear preference.”

Why should Vivanti hire you?

> “Because I can be useful to a client quickly, and I’ll represent Vivanti well. I bring hands-on depth in Databricks, Spark and Delta, with measurable results in performance and cost. I bring regulated-banking experience (lineage, reconciliation, deadlines), which matters to financial-services clients. And I communicate clearly: I’ve led changes across teams without authority, and I explain technical trade-offs in business terms. I’m also honest about what I don’t know, which I think clients value more than anything.”

What value can you bring to Vivanti’s clients?

> “Practical things a client pays for. Making their Databricks or Spark workloads faster and cheaper, based on evidence rather than guesswork. Making their pipelines reliable, with data quality checks, reconciliation and idempotent reruns. Modernising orchestration or storage safely, in phases. And for regulated clients, building auditability in from the start.”

How quickly can you become billable?

> “On a Databricks or Spark engagement, very quickly. That’s my daily work, so I’d expect to contribute from the first week, once I understand the client’s context. On Snowflake or GCP I’d need a short ramp-up on the platform specifics, but the underlying problems (modeling, performance, reliability, cost) are the same, and I learn quickly. I’d also use the Academy to get certified early.”

Why are you interested in working with Australian clients?

> “Australia has a mature financial-services sector with strong regulators, much like Canada, so my experience translates directly. APRA’s expectations on data and risk sound very similar to what I’ve worked with under OSFI. And practically, working with Sydney from Bengaluru is a good time-zone overlap compared with North America.”

Accurate note: APRA is Australia’s prudential regulator, the counterpart to Canada’s OSFI. Don’t claim specific knowledge of APRA standards beyond that.

What do you expect from a consulting organisation?

> “Three things. Variety of meaningful work, so I’m growing and not just filling a seat. People I can learn from, especially on the client side of consulting, which is newer to me. And honesty: clear expectations and feedback. In return I’ll bring ownership, quality and a good reputation with clients.”

How will you adapt from an internal role to consulting?

> “The main shifts are context switching, communication and commercial awareness. At TD I know the platform’s history, so I’ll need to get faster at learning a new environment. I’d do that by asking good questions early, reading before assuming, and writing down what I learn. On communication, I’ll be more deliberate about updating the client and managing expectations. And I’ll keep in mind that the client’s time and budget are real constraints. I’ve already had a taste of this: explaining our platform to a partner team so they could use it without depending on us is very close to client work.”

Part 6: How SQL Actually Works

A partner who wants to test depth will ask “what happens when you run this?” These answers are written to be spoken. Aim for 30 to 60 seconds each.

Q1. What is the logical order in which SQL runs a query?

> “SQL is written in one order but evaluated logically in another: 1. FROM and JOINs build the working set of rows. 2. WHERE filters individual rows. 3. GROUP BY forms groups. 4. HAVING filters groups. 5. SELECT computes expressions, including window functions. 6. DISTINCT removes duplicates. 7. ORDER BY sorts. 8. LIMIT or TOP cuts the result.”

Why it matters (say this): “It explains three common errors. You can’t use a SELECT alias in WHERE, because WHERE runs first. You can’t filter on a window function in WHERE, so you wrap it in a CTE or use QUALIFY. And aggregates can’t go in WHERE, only in HAVING.”

Follow-up: “Is that the physical order too?”

> “No. It’s the logical meaning. The optimizer is free to reorder physically, for example pushing filters below joins, as long as the result is the same.”

Q2. What happens between pressing Run and getting results?

> “Four stages. Parse: check syntax and build a tree. Bind or analyse: resolve table and column names and check types and permissions. Optimise: the optimizer considers many equivalent plans (join orders, join algorithms, where to filter) and picks the cheapest using table statistics. Execute: the engine runs the chosen physical plan, reading data, joining, aggregating, and returning rows.”

Q3. What is a cost-based optimizer and why do statistics matter?

> “The optimizer estimates the cost of each candidate plan, mainly how many rows flow through each step and how much I/O and memory that takes. Its estimates come from statistics: row counts, distinct values, min and max, and histograms. If statistics are stale, it may think a table has 1,000 rows when it has 100 million, and pick a terrible plan, like a nested loop join on two huge tables. So a sudden slowdown after a big data load often means stale statistics.”

Q4. What join algorithms exist and when is each used?

Algorithm	How it works	Best when	Weakness
Nested loop	For each row on one side, look up matches on the other	One side is tiny, or the other side has an index on the join key	Very slow for two large inputs
Hash join	Build a hash table on the smaller input, then probe it with the larger	Large equality joins, no useful index	Needs memory. Spills to disk if the build side is too big
Sort-merge join	Sort both inputs on the key, then merge them in one pass	Both inputs large, or already sorted	The sort is expensive
Broadcast join (distributed)	Copy the small table to every worker	Small dimension joined to a huge fact	Out of memory if the "small" table isn't small. You've seen this in production
Shuffle join (distributed)	Redistribute both sides across the network by join key	Two large tables	Network-heavy, and sensitive to skew

> "In distributed engines like Snowflake, BigQuery or Spark, the big decision is whether to broadcast or shuffle. Moving data across the network is usually the most expensive thing a query does."

Q5. What is an index? When does it help and when doesn't it?

> "An index is a separate sorted structure, usually a B-tree, that maps key values to row locations, like the index at the back of a book. It turns a full scan into a quick lookup.
>
> It helps for selective filters on the indexed column, joins on it, and ORDER BY on it. It doesn't help when a filter matches a large share of the table, because a scan is cheaper then. It also doesn't help when you wrap the column in a function, like `WHERE YEAR(order_date) = 2026`.
>
> The cost: every insert and update must also update the index, so write-heavy tables pay for each extra index."

Follow-ups:

- **Clustered vs non-clustered:** "A clustered index stores the table rows themselves in key order, so there can be only one per table. A non-clustered index is a separate structure pointing back to the rows."
- **Composite index:** "An index on (a, b) helps filters on a, or on a and b together, but not on b alone. That's the leftmost prefix rule."
- **Covering index:** "An index that contains every column the query needs, so the engine never touches the table."

Q6. Snowflake, BigQuery and Databricks don't use indexes like that. How do they go fast?

> "They're columnar and use metadata-based pruning instead of indexes.
>
> Columnar storage means a query reading 3 columns out of 100 reads only those 3, and similar values compress well. Each storage chunk also keeps min and max values per column: micro-partitions in Snowflake, files and row groups in Delta or Parquet. If a query filters on `order_date = '2026-09-01'` and a chunk's range doesn't include that date, the engine skips it entirely. That's pruning.
>
> So in these engines, 'indexing' means laying data out so pruning works: partitioning, clustering keys in Snowflake, clustering in BigQuery, liquid clustering or Z-order in Delta."

Q7. Row store vs column store: when do you use each?

> "Row stores, like Postgres or SQL Server tables for applications, keep each row together, which is ideal for OLTP: fetch or update one customer's full record quickly. Column stores keep each column together, which is ideal for OLAP: scan millions of rows but only a few columns, and aggregate. Operational systems use row stores, and analytics warehouses use column stores."

Q8. OLTP vs OLAP?

> "OLTP handles many small, fast transactions: insert an order, update a balance. It's normalised to avoid update anomalies and needs strong consistency. OLAP handles fewer, large analytical queries over history. It's denormalised or dimensional for fast reads, and loaded in batches or streams. My platform at TD is OLAP. It reads from operational source systems and builds history for reporting."

Q9. What is sargability?

> "A predicate is 'sargable' if the engine can use an index or pruning to satisfy it. `WHERE order_date >= '2026-01-01'` is sargable. `WHERE YEAR(order_date) = 2026` is not, because the function must run on every row first. Rewrite functions on columns as ranges on the raw column. In my own work, removing TRIM from join keys was the same idea."

Q10. CTE vs subquery vs temp table vs materialized view?

Option	What it is	When to use
CTE	A named subquery inside one statement	Readability. Most engines inline it, so there's no performance gain by itself
Subquery	An inline query	Simple one-off filters, like `WHERE x IN (SELECT ...)`

Option	What it is	When to use
Temp table	Results physically stored for the session	Reusing an expensive intermediate result several times, or breaking a huge query into debuggable steps
View	A stored query, run fresh each time	Reusable logic and access control
Materialized view	A stored query **with stored results**, refreshed	Expensive aggregations queried often, where slightly stale data is acceptable

> "A common misconception is that CTEs are always materialised. In most modern engines they're inlined, so referencing a CTE three times may compute it three times. If it's expensive and reused, a temp table can be faster."

Q11. How do window functions work, and what do they cost?

> "A window function computes a value for each row using a set of related rows, the window, without collapsing rows the way GROUP BY does. PARTITION BY defines the groups, ORDER BY defines the order inside each group, and the frame defines which rows around the current row count.
>
> Cost: the engine must partition and sort the data, so a window over a huge table with a high-cardinality PARTITION BY is a big sort and shuffle. Several windows with the same PARTITION BY and ORDER BY can share one sort, so keep them consistent where possible."

Q12. What is the default window frame, and why does it trip people up?

> "When a window has ORDER BY but no explicit frame, most engines default to `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`. RANGE treats rows with equal ORDER BY values as peers, so on a running total, two transactions on the same date both get the full total for that date. If I want a strict row-by-row running total, I write `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` explicitly. And LAST_VALUE with the default frame just returns the current row, which surprises people."

Q13. How does SQL handle NULL?

> "NULL means unknown, so SQL uses three-valued logic: TRUE, FALSE and UNKNOWN. Any comparison with NULL, including `NULL = NULL`, is UNKNOWN, and WHERE keeps only TRUE rows. That's why you write `IS NULL`.
>
> The practical traps: `COUNT(col)` skips NULLs but `COUNT(*)` doesn't. `SUM` and `AVG` ignore NULLs, so an average of 10, NULL and 20 is 15, not 10. `NOT IN` against a list containing a NULL returns no rows. A NULL join key never matches. GROUP BY puts all NULLs in one group. For null-safe comparison, use `IS NOT DISTINCT FROM`."

Q14. EXISTS vs IN vs JOIN?

> “For ‘keep rows that have a match’, EXISTS and IN usually produce the same plan in modern optimizers, a semi-join. A plain JOIN can duplicate rows if the other side has multiple matches, so for existence checks EXISTS is safer. For ‘no match’, prefer NOT EXISTS or a LEFT JOIN with `IS NULL`. Avoid NOT IN, because one NULL in the subquery makes it return nothing.”

Q15. UNION vs UNION ALL?

> “UNION removes duplicates, which requires a sort or hash across the whole result. UNION ALL just appends. Use UNION ALL unless you actually need deduplication. It’s faster, and it doesn’t silently drop legitimate duplicate rows.”

Q16. DISTINCT vs GROUP BY?

> “For simple deduplication they’re usually executed the same way. But DISTINCT is often a code smell: if you need it to fix duplicates from a join, the real problem is usually a grain mistake in the join. I’d rather fix the join than hide it.”

Q17. What are ACID and isolation levels?

> “ACID: Atomicity, all or nothing. Consistency, constraints hold. Isolation, concurrent transactions don’t see each other’s partial work. Durability, committed data survives a crash.
>
> Isolation levels trade safety for concurrency. Read Uncommitted allows dirty reads. Read Committed only sees committed data but can see different values if you read twice. Repeatable Read prevents that but may allow phantom rows. Serializable behaves as if transactions ran one at a time.
>
> Delta Lake gives ACID on the lakehouse using a transaction log and optimistic concurrency. Writers check for conflicts at commit time, and readers see a consistent snapshot.”

Q18. What is MVCC?

> “Multi-version concurrency control. Instead of locking rows for readers, the database keeps multiple versions of rows, and each transaction reads a consistent snapshot as of when it started. Readers don’t block writers and writers don’t block readers. Delta’s time travel is a visible example of the same idea: old versions are kept, so you can query the table as of an earlier version.”

Q19. How do you read an execution plan?

> “Read it from the leaves, the table scans, up to the root. At each step I check four things. Rows: are estimated and actual rows close? A big gap means bad statistics. The biggest step: which operator takes most of the time or bytes. Join type: a nested loop on large inputs is a red flag. Scan versus pruning: how many partitions or files were scanned out of the total.
>
> In Snowflake I use the Query Profile and look at ‘partitions scanned vs total’, spilling to local or remote storage, and exploding joins. In Databricks I use the Spark UI, looking at stage times, shuffle size, spill and task skew. In BigQuery, the execution details show bytes processed and slot time per stage.”

Q20. What is data skew in SQL engines?

> “Skew is when a few key values hold most of the rows, so in a distributed join or GROUP BY, one worker gets far more data than the others and everyone waits for it. You see it as one task running much longer than the median. Fixes: broadcast the smaller side if possible, filter or handle the hot key separately, use the engine’s skew handling (like adaptive execution in Spark), or salt the key.”

Part 7: SQL Performance

The optimisation ladder (your answer to “this query is slow”)

Always go in this order. Cheap, high-impact fixes come first, and hardware comes last.

1. **Is it correct?** A query that is slow because a join explodes rows is a correctness bug, not a tuning problem.
2. **Read less data.** Select only needed columns, filter early, and make filters prunable (no functions on filter columns).
3. **Move less data.** Reduce rows before joins, broadcast small tables, and avoid unnecessary DISTINCT, ORDER BY or UNION.
4. **Compute less.** Pre-aggregate, go incremental instead of full recompute, and reuse expensive intermediate results.
5. **Lay data out better.** Partitioning or clustering on common filter and join columns, compaction of small files, fresh statistics.
6. **Precompute.** Materialized views or summary tables for repeated heavy queries.
7. **Scale compute.** Bigger warehouse or cluster, only after the above, and only if the job gets faster in proportion to the extra spend.

> **Say this as your opening line:** “Before tuning anything, I’d look at the query plan to see where the time actually goes: scan, join, shuffle or sort. Then I’d work from cheapest fix to most expensive. Adding compute is the last resort, not the first.”

The 20 anti-patterns and their fixes

#	Anti-pattern	Why it hurts	Fix
1	`SELECT *`	Columnar engines read every column; BigQuery bills for them	Select only needed columns
2	Function on a filter column: `WHERE YEAR(dt) = 2026`	Blocks pruning and index use	`WHERE dt >= '2026-01-01' AND dt < '2027-01-01'`
3	Function on join keys: `ON TRIM(a.k) = TRIM(b.k)`	Computed per row, blocks optimisations	Clean keys once upstream (you did this at TD)
4	Implicit type cast in join: string key joined to integer key	Per-row cast, can block pruning	Align types in the model
5	Joining before filtering	Moves and joins rows you then throw away	Filter in a CTE first. The optimizer often does this, but not always across complex logic
6	Aggregating after a fan-out join	Double counting and big intermediate results	Aggregate each side to the same grain first, then join

#	Anti-pattern	Why it hurts	Fix
7	DISTINCT to “fix” duplicates	Hides a grain bug and adds a big sort	Fix the join
8	UNION instead of UNION ALL	Unneeded dedup sort	UNION ALL
9	`NOT IN` with a nullable subquery	Wrong results (nothing returned)	NOT EXISTS
10	Correlated subquery per row	Can run once per outer row	Rewrite as a join or window function
11	OR across different columns in joins	Often forces a nested loop	Split into UNION ALL of two joins
12	Leading wildcard: `LIKE '%abc'`	Cannot use indexes or pruning	Full-text search, or store a reversed or normalised column
13	ORDER BY in subqueries or views	Wasted sort that gets discarded	Sort only in the final output
14	Many windows with different PARTITION BY	Multiple big sorts	Align partitions, or split the work
15	Self-join for running totals or previous rows	Quadratic row explosion	Window functions: SUM OVER, LAG
16	Recomputing full history every run	Cost grows forever	Incremental processing with watermarks and MERGE
17	Too many small files	Per-file overhead dominates	Compaction (OPTIMIZE), fewer and larger writes
18	Over-partitioning (e.g. by customer_id)	Millions of tiny partitions	Partition by low-cardinality date. Cluster on high-cardinality keys
19	Exploding join from non-unique dimension keys	Row multiplication	Enforce uniqueness, or dedupe the dimension first
20	Huge IN lists	Slow parsing and planning	Put the values in a table and join

Worked example 1: the slow monthly report

Before:

```
SELECT DISTINCT c.customer_id, c.country,
       SUM(o.amount) OVER (PARTITION BY c.customer_id) AS total
FROM customers c
JOIN orders o ON UPPER(TRIM(o.customer_id)) = UPPER(TRIM(c.customer_id))
WHERE YEAR(o.order_ts) = 2026;
```

Problems to say out loud:

1. `YEAR(o.order_ts)` blocks partition pruning, so the whole orders history is scanned.
2. `UPPER(TRIM(...))` on both keys runs on every row and prevents efficient joins.
3. A window SUM plus DISTINCT does an aggregation the expensive way.

After:

```
WITH orders_2026 AS (
    SELECT customer_id, SUM(amount) AS total
    FROM orders
    WHERE order_ts >= '2026-01-01' AND order_ts < '2027-01-01'
    GROUP BY customer_id
)
SELECT c.customer_id, c.country, o.total
FROM orders_2026 o
JOIN customers c ON c.customer_id = o.customer_id;
```

> "Filter with a range so pruning works, aggregate before the join so we join one row per customer instead of every order, and fix key formatting upstream so the join is on raw columns."

Worked example 2: the exploding join

A dashboard shows revenue three times higher than finance reports.

```
-- Wrong
SELECT SUM(o.amount)
FROM orders o
JOIN order_items i ON i.order_id = o.order_id;
```

> "orders is one row per order, but after joining order_items there's one row per item. An order with three items has its total counted three times. Either sum at the item level with `SUM(i.quantity * i.unit_price)`, or don't join items at all if the order total is what I need. This is a grain problem that looks like a performance problem, because the intermediate result is also three times larger."

Worked example 3: the query that got slow overnight

> "If nothing in the code changed, something in the data or environment did. I'd check four things. Volume: did a big load land? Statistics: are they stale after that load? Skew: did a new hot key appear, like a default value such as 'UNKNOWN' used for millions of rows? Concurrency: is the warehouse queueing behind other jobs?
>
> That last one is exactly what I saw with Synapse at TD. The query itself was fine. It was waiting in a concurrency queue for more than 30 minutes, and the fix was taking Synapse off the critical path."

Platform-specific performance talking points

Snowflake

- **Pruning:** check "partitions scanned vs partitions total" in the Query Profile. Good pruning is the biggest win.
- **Clustering keys:** for very large tables filtered often on a column that isn't naturally ordered. Clustering has a maintenance cost, so use it only where pruning is poor.

- **Warehouse sizing:** each size up roughly doubles compute and credits. Scale up for one big complex query. Scale out (multi-cluster) for many concurrent users.
- **Spilling:** "bytes spilled to local or remote storage" means not enough memory. Reduce data first, then size up.
- **Caching:** the result cache returns identical repeated queries instantly. The warehouse cache keeps recently read data local.
- **Search optimization service:** for highly selective point lookups on big tables.

BigQuery

- **Cost model:** on-demand pricing bills bytes scanned, so `SELECT *` and unpartitioned scans cost money directly.
- **Partitioning:** by date or ingestion time. Require a partition filter on big tables.
- **Clustering:** on common filter or join columns, within partitions.
- **Slots:** the compute units. Watch slot time and skewed stages in the execution details.
- **Approximate functions:** APPROX_COUNT_DISTINCT when an exact count isn't required.

Databricks and Delta

- **Data skipping:** from file-level min and max statistics.
- **Liquid clustering or Z-order:** on filter and join columns (you've migrated teams to liquid clustering).
- **Photon:** the vectorised engine, usually faster for SQL-heavy work.
- **Adaptive Query Execution:** re-plans at runtime, merging small partitions, switching to broadcast and splitting skewed partitions.
- **OPTIMIZE:** compacts small files.

Performance interview questions

P1. How would you make a slow query faster? Walk me through it.

Use the ladder above, opening with "look at the plan first."

P2. How do you choose partition and cluster columns?

> "Partition by what almost every query filters on and what has low-to-medium cardinality, usually date. Cluster by the next most common filter or join columns, which can be higher-cardinality, like customer_id. Avoid partitioning on high-cardinality columns, because it creates millions of tiny partitions. And validate with real query patterns: check which columns actually appear in WHERE and JOIN clauses."

P3. A query scans 5 TB but returns 100 rows. What's wrong?

> "Pruning isn't working. Either the filter column isn't how the table is laid out, the filter wraps the column in a function or cast, or the filter is applied after a join instead of on the base table. I'd check the plan for partitions scanned versus total, then fix the predicate or the layout."

P4. How do you speed up COUNT(DISTINCT) on billions of rows?

> "First ask whether exact is needed. For dashboards, APPROX_COUNT_DISTINCT, which uses HyperLogLog, is typically within about 2% and far cheaper. If exact is required, pre-aggregate to the distinct grain first, for example one row per day per user, and count from that smaller table. For rolling-window distinct counts, keep a daily user-activity table so you're not rescanning raw events."

P5. When would you denormalise?

> "For read-heavy analytics, where the same joins run thousands of times a day. A wide table or star schema avoids repeated joins. The costs are storage, and needing to rebuild when dimension attributes change. For OLTP I'd stay normalised."

P6. How do you make an incremental load efficient?

> "Process only new or changed rows using a watermark, such as the maximum updated_at already loaded, with a small lookback window for late data. Deduplicate the batch, then MERGE into the target on the business key. Make sure the MERGE's target side prunes, for example by including the partition date in the match condition, so it doesn't scan the whole target table."

P7. How do you find which queries to optimise first?

> "By total cost, not by slowest single run. A 2-minute query that runs 500 times a day costs more than a 30-minute monthly job. I'd rank by total execution time or credits over a period using the query history (Snowflake's QUERY_HISTORY, BigQuery's INFORMATION_SCHEMA.JOBS, Databricks system tables) and fix the top ten."

P8. Would you just scale up the warehouse?

> “Only as a last step, and only if it pays off. Scaling up helps if the query is compute-bound and parallelises well. It won’t fix bad pruning, exploding joins or skew. You’d just pay more for the same waste. I’d fix the query first, then right-size. At TD, right-sizing against the critical path mattered more than making everything bigger.”

Part 8: Practice Schemas and the 15 Patterns

The cases in this guide use a few small, realistic schemas. Partners often describe tables verbally, so get used to asking "what is one row?" for each.

Retail and e-commerce

```
customers (customer_id PK, name, country, signup_date, segment)
orders    (order_id PK, customer_id FK, order_ts, amount, status, channel)
            -- amount: order total
            -- status: 'COMPLETED', 'CANCELLED', 'REFUNDED'
order_items (order_id FK, product_id FK, quantity, unit_price)
products  (product_id PK, product_name, category, cost_price)
events    (event_id PK, user_id, event_ts, event_type, page, session_hint)
            -- event_type: 'view', 'add_to_cart', 'checkout', 'purchase'
marketing_touches (user_id, touch_ts, channel)
```

Banking (your domain, and Vivanti has banking clients)

```
accounts     (account_id PK, customer_id, open_date, close_date, product_type)
transactions (txn_id PK, account_id FK, txn_ts, amount, direction, merchant, status)
               -- direction: 'CREDIT' or 'DEBIT'
loan_schedule (loan_id, installment_no, due_date, amount_due, amount_paid, paid_date)
customer_dim_scd2 (customer_sk PK, customer_id, risk_segment,
                   valid_from, valid_to, is_current)
dim_date     (date_day PK, year, month, week, is_business_day)
```

HR (classic warm-up questions)

```
employees (emp_id PK, name, dept_id, manager_id, salary, hire_date)
departments (dept_id PK, dept_name)
```

The grain habit

Before writing any query, say the grain of every table you touch:

- `orders`: one row per order.
- `order_items`: one row per product within an order. **Joining orders to order_items multiplies order rows.** Summing an order-level column after that join double counts.
- `transactions`: one row per transaction.
- `customer_dim_scd2`: one row per customer **per version**. Joining without a date condition multiplies rows.

> **The most common failure in analytical SQL rounds is a grain error:** joining two tables at different grains and then aggregating. Always ask: "after this join, what does one row represent?"

Dialect notes

Vivanti works on Snowflake, BigQuery and Databricks. The partner will not fail you for dialect details, but say it when you know a difference:

Need	Standard / Postgres	Snowflake	BigQuery	Databricks SQL
Filter on a window result	subquery or CTE	`QUALIFY`	`QUALIFY`	`QUALIFY`
Date difference in days	`d2 - d1`	`DATEDIFF('day', d1, d2)`	`DATE_DIFF(d2, d1, DAY)`	`DATEDIFF(d2, d1)`
Truncate to month	`DATE_TRUNC('month', ts)`	`DATE_TRUNC('month', ts)`	`DATE_TRUNC(d, MONTH)`	`DATE_TRUNC('month', ts)`
Approximate distinct	not standard	`APPROX_COUNT_DISTINCT`	`APPROX_COUNT_DISTINCT`	`APPROX_COUNT_DISTINCT`
Null-safe equality	`IS NOT DISTINCT FROM`	`IS NOT DISTINCT FROM` or `EQUAL_NULL`	`IS NOT DISTINCT FROM`	`<=>` or `IS NOT DISTINCT FROM`
Median	`PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY x)`	same, or `MEDIAN(x)`	`APPROX_QUANTILES(x, 2)[OFFSET(1)]`	`PERCENTILE(x, 0.5)` or `MEDIAN(x)`

The answers below use readable standard-style SQL. Say “in Snowflake I’d use QUALIFY here” to show platform awareness.

The 15 patterns

Most open-ended analytical questions are built from a small set of patterns. Recognise the pattern, name it out loud (“this is a gaps-and-islands problem”), then apply the template.

1. Top N per group

```
SELECT *
FROM (
  SELECT p.*, DENSE_RANK() OVER (PARTITION BY category ORDER BY revenue DESC) AS rnk
  FROM product_revenue p
) t
WHERE rnk <= 3;
-- Snowflake/BigQuery/Databricks: use QUALIFY rnk <= 3
```

Say: “Should ties share a place? ROW_NUMBER gives exactly N rows, RANK skips numbers after ties, DENSE_RANK doesn’t skip.”

2. Latest record per key (deduplication)

```
SELECT *
FROM (
  SELECT t.*, ROW_NUMBER() OVER (PARTITION BY customer_id
                                 ORDER BY updated_at DESC, load_ts DESC) AS rn
  FROM staging_customers t
) x
WHERE rn = 1;
```

Say: “I add a second ORDER BY column as a tiebreaker so the result is deterministic.”

3. Running total and moving average

```
SELECT order_date, daily_revenue,
  SUM(daily_revenue) OVER (ORDER BY order_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_total,
  AVG(daily_revenue) OVER (ORDER BY order_date
        ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS ma_7d
FROM daily_sales;
```

Say: “This 7-day average assumes one row per day with no gaps. If days can be missing, I’d join to a date spine first.”

4. Period-over-period change

```
SELECT month, revenue,
  LAG(revenue) OVER (ORDER BY month) AS prev_month,
  100.0 * (revenue - LAG(revenue) OVER (ORDER BY month))
        / NULLIF(LAG(revenue) OVER (ORDER BY month), 0) AS pct_change
FROM monthly_revenue;
```

Say: “NULLIF avoids division by zero. Multiplying by 100.0 avoids integer division.”

5. Gaps and islands (consecutive streaks)

```
WITH d AS (
  SELECT DISTINCT user_id, activity_date FROM activity
), g AS (
  SELECT user_id, activity_date,
    activity_date - CAST(ROW_NUMBER() OVER (PARTITION BY user_id
                         ORDER BY activity_date) AS INT) AS grp
  FROM d
)
SELECT user_id, MIN(activity_date) AS start_date, MAX(activity_date) AS end_date,
       COUNT(*) AS streak_days
FROM g
GROUP BY user_id, grp;
```

Say: “Consecutive dates minus a row number give a constant. That constant identifies the island.” In Snowflake use `DATEADD('day', -rn, activity_date)` instead of subtraction.

6. Sessionization

```
WITH flagged AS (
  SELECT user_id, event_ts,
    CASE WHEN LAG(event_ts) OVER (PARTITION BY user_id ORDER BY event_ts) IS NULL
           OR event_ts - LAG(event_ts) OVER (PARTITION BY user_id ORDER BY event_ts)
              > INTERVAL '30 minutes'
         THEN 1 ELSE 0 END AS new_session
  FROM events
)
SELECT user_id, event_ts,
  SUM(new_session) OVER (PARTITION BY user_id ORDER BY event_ts
                         ROWS UNBOUNDED PRECEDING) AS session_no
FROM flagged;
```

7. Anti-join (customers with no X)

```
SELECT c.customer_id
FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o
                  WHERE o.customer_id = c.customer_id
                    AND o.status = 'COMPLETED');
```

Say: “NOT EXISTS is NULL-safe. NOT IN isn’t.”

8. Conditional aggregation (pivot)

```
SELECT customer_id,
  SUM(CASE WHEN channel = 'web'   THEN amount ELSE 0 END) AS web_revenue,
  SUM(CASE WHEN channel = 'store' THEN amount ELSE 0 END) AS store_revenue,
  COUNT(CASE WHEN status = 'REFUNDED' THEN 1 END)         AS refunds
FROM orders
GROUP BY customer_id;
```

9. Cohort retention

```
WITH first_order AS (
  SELECT customer_id, DATE_TRUNC('month', MIN(order_ts)) AS cohort_month
  FROM orders WHERE status = 'COMPLETED' GROUP BY customer_id
), activity AS (
  SELECT DISTINCT customer_id, DATE_TRUNC('month', order_ts) AS active_month
  FROM orders WHERE status = 'COMPLETED'
)
SELECT f.cohort_month,
  (EXTRACT(YEAR FROM a.active_month) - EXTRACT(YEAR FROM f.cohort_month)) * 12
  + EXTRACT(MONTH FROM a.active_month) - EXTRACT(MONTH FROM f.cohort_month) AS month_n,
  COUNT(DISTINCT a.customer_id) AS active_customers
FROM first_order f
JOIN activity a ON a.customer_id = f.customer_id
GROUP BY 1, 2;
```

Divide by the month-0 count per cohort to get the retention rate.

10. Funnel

```
WITH steps AS (
  SELECT user_id,
    MIN(CASE WHEN event_type = 'view'        THEN event_ts END) AS view_ts,
    MIN(CASE WHEN event_type = 'add_to_cart' THEN event_ts END) AS cart_ts,
    MIN(CASE WHEN event_type = 'purchase'    THEN event_ts END) AS buy_ts
  FROM events GROUP BY user_id
)
SELECT
  COUNT(view_ts) AS viewed,
  COUNT(CASE WHEN cart_ts > view_ts THEN 1 END) AS carted,
  COUNT(CASE WHEN buy_ts > cart_ts AND cart_ts > view_ts THEN 1 END) AS purchased
FROM steps;
```

11. Date spine (filling missing days)

```
SELECT d.date_day, COALESCE(s.revenue, 0) AS revenue
FROM dim_date d
LEFT JOIN daily_sales s ON s.order_date = d.date_day
WHERE d.date_day BETWEEN '2026-01-01' AND '2026-09-30';
```

Say: “Without a spine, days with zero sales simply don’t exist, which breaks moving averages and charts.”

12. Point-in-time join (SCD2)

```
SELECT t.txn_id, t.amount, c.risk_segment
FROM transactions t
JOIN accounts a ON a.account_id = t.account_id
JOIN customer_dim_scd2 c
  ON c.customer_id = a.customer_id
 AND t.txn_ts >= c.valid_from
 AND t.txn_ts <  COALESCE(c.valid_to, TIMESTAMP '9999-12-31');
```

Say: “Half-open intervals, >= start and < end, so a timestamp exactly on a boundary matches exactly one version.”

13. Percentiles and median

```
SELECT category,
  PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY amount) AS median_amount,
  PERCENTILE_CONT(0.9) WITHIN GROUP (ORDER BY amount) AS p90_amount
FROM order_amounts
GROUP BY category;
```

14. Self-join for hierarchies (employee vs manager)

```
SELECT e.name
FROM employees e
JOIN employees m ON m.emp_id = e.manager_id
WHERE e.salary > m.salary;
```

For hierarchies of unknown depth, use a recursive CTE.

15. Recursive CTE (org chart)

```
WITH RECURSIVE chain AS (
  SELECT emp_id, manager_id, name, 0 AS lvl
  FROM employees WHERE manager_id IS NULL
  UNION ALL
  SELECT e.emp_id, e.manager_id, e.name, c.lvl + 1
  FROM employees e JOIN chain c ON e.manager_id = c.emp_id
)
SELECT * FROM chain;
```

Say: “I’d add a depth limit as protection against cycles in bad data.” (Snowflake and BigQuery accept `WITH RECURSIVE`.)

Part 9: Twenty Analytical SQL Cases

This is the core of your preparation. For each case:

1. Read only the **prompt**.
2. Set a 15-minute timer and solve it **out loud**, on paper or in a plain editor, using the 7-step talk track.
3. Then compare with the model answer, and add your misses to your leak list.

The model answers show one good path, not the only one. If your assumptions differ but are stated and sensible, that is a good answer.

Case 1: Which customers are at risk of churning?

Prompt: "Marketing wants a list of customers at risk of churning so they can send offers. Write the query."

Clarify: "Is the goal to catch people before they're fully gone? Do we care about all customers or only valuable ones?"

Define and assume:

- Only completed orders count.
- "At risk" means active in the past, but no purchase for longer than usual. Simple version: no order in the last 90 days, but at least 2 orders in the prior year.
- `CURRENT_DATE` is the reference date.

```
WITH cust AS (
  SELECT customer_id,
         MAX(order_ts) AS last_order_ts,
         COUNT(*)      AS orders_last_year,
         SUM(amount)   AS revenue_last_year
  FROM orders
  WHERE status = 'COMPLETED'
    AND order_ts >= CURRENT_DATE - INTERVAL '365 days'
  GROUP BY customer_id
)
SELECT customer_id, last_order_ts, orders_last_year, revenue_last_year
FROM cust
WHERE orders_last_year >= 2
  AND last_order_ts < CURRENT_DATE - INTERVAL '90 days'
ORDER BY revenue_last_year DESC;
```

Alternative (say this, it scores well): "A fixed 90 days treats a weekly buyer and a yearly buyer the same. A better definition uses each customer's own rhythm: flag them if the time since their last order is more than twice their average gap between orders." Then compute the average gap with LAG:

```
WITH gaps AS (
  SELECT customer_id, order_ts,
    order_ts - LAG(order_ts) OVER (PARTITION BY customer_id ORDER BY order_ts) AS gap
  FROM orders WHERE status = 'COMPLETED'
), stats AS (
  SELECT customer_id, MAX(order_ts) AS last_order_ts, AVG(gap) AS avg_gap,
         COUNT(*) AS n_orders
  FROM gaps GROUP BY customer_id
)
SELECT customer_id
FROM stats
WHERE n_orders >= 3
  AND CURRENT_TIMESTAMP - last_order_ts > 2 * avg_gap;
```

Edge cases: customers with one order (no gap, excluded by `n_orders >= 3`), refunds, and new signups.

Follow-ups:

- "How would you validate this?" Backtest: apply the rule as of six months ago and check how many flagged customers actually stopped buying.
- "Which list goes to marketing first?" Sort by revenue, so the most valuable at-risk customers come first.

Case 2: Revenue dropped 15% last month. Find out why.

Prompt: "Revenue dropped 15% month over month. Use SQL to explain why."

Talk track: "Revenue breaks down as number of customers times orders per customer times average order value. I'll compute each component by month, and see which one moved. Then I'll slice the component that moved by region, channel and category."

```
WITH m AS (
  SELECT DATE_TRUNC('month', order_ts) AS month,
         COUNT(DISTINCT customer_id)   AS customers,
         COUNT(*)                      AS orders,
         SUM(amount)                   AS revenue
  FROM orders
  WHERE status = 'COMPLETED'
    AND order_ts >= DATE_TRUNC('month', CURRENT_DATE) - INTERVAL '3 months'
    AND order_ts <  DATE_TRUNC('month', CURRENT_DATE)
  GROUP BY 1
)
SELECT month, customers,
       1.0 * orders / customers AS orders_per_customer,
       1.0 * revenue / orders   AS avg_order_value,
       revenue,
       100.0 * (revenue - LAG(revenue) OVER (ORDER BY month))
             / NULLIF(LAG(revenue) OVER (ORDER BY month), 0) AS revenue_pct_change
FROM m
ORDER BY month;
```

Then drill into the driver, for example by channel:

```
SELECT channel, DATE_TRUNC('month', order_ts) AS month, SUM(amount) AS revenue
FROM orders
WHERE status = 'COMPLETED'
  AND order_ts >= DATE_TRUNC('month', CURRENT_DATE) - INTERVAL '2 months'
  AND order_ts <  DATE_TRUNC('month', CURRENT_DATE)
GROUP BY 1, 2
ORDER BY 1, 2;
```

Rule out data issues first (a partner will love this): “Before blaming the business, I’d check the data. Did a source load fail for a few days? Did refunds get reclassified? Is last month’s data complete? I’d compare daily row counts against the previous month.”

Other explanations to mention: fewer days in the month or a seasonal pattern (compare with the same month last year), a price change, a lost big customer, a marketing campaign ending.

Case 3: How much revenue comes from new versus returning customers?

Prompt: “Split monthly revenue into new and returning customers.”

Define: a customer is **new** in the month of their first completed order and **returning** in every later month.

```
WITH firsts AS (
  SELECT customer_id, MIN(order_ts) AS first_order_ts
  FROM orders WHERE status = 'COMPLETED'
  GROUP BY customer_id
)
SELECT DATE_TRUNC('month', o.order_ts) AS month,
  SUM(CASE WHEN DATE_TRUNC('month', o.order_ts) = DATE_TRUNC('month', f.first_order_ts)
           THEN o.amount ELSE 0 END) AS new_customer_revenue,
  SUM(CASE WHEN DATE_TRUNC('month', o.order_ts) > DATE_TRUNC('month', f.first_order_ts)
           THEN o.amount ELSE 0 END) AS returning_customer_revenue
FROM orders o
JOIN firsts f ON f.customer_id = o.customer_id
WHERE o.status = 'COMPLETED'
GROUP BY 1
ORDER BY 1;
```

Assumption to state: “The first order must be computed over **all history**, not just the reporting window. Otherwise everyone looks new in the first month of the window.” This is a classic trap, so say it proactively.

Alternative definition: “new” could mean signed up this month, using `customers.signup_date`, rather than first purchase. Ask which one the business uses.

Case 4: Where do users drop off in the purchase funnel, by channel?

Prompt: “Show conversion through view, add to cart and purchase, by marketing channel.”

Clarify: “Does the order of steps matter? Should a purchase count only if it happened after the cart? Which channel should a user belong to if they had several touches?”

Assume: steps must happen in order, and a user’s channel is their first touch (state this; Case 14 covers attribution).

```
WITH first_touch AS (
  SELECT user_id, channel
  FROM (
    SELECT user_id, channel,
           ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY touch_ts) AS rn
    FROM marketing_touches
  ) t WHERE rn = 1
), steps AS (
  SELECT user_id,
    MIN(CASE WHEN event_type = 'view'        THEN event_ts END) AS view_ts,
    MIN(CASE WHEN event_type = 'add_to_cart' THEN event_ts END) AS cart_ts,
    MIN(CASE WHEN event_type = 'purchase'    THEN event_ts END) AS buy_ts
  FROM events GROUP BY user_id
)
SELECT COALESCE(ft.channel, 'unknown') AS channel,
  COUNT(s.view_ts) AS viewed,
  COUNT(CASE WHEN s.cart_ts > s.view_ts THEN 1 END) AS carted,
  COUNT(CASE WHEN s.cart_ts > s.view_ts AND s.buy_ts > s.cart_ts THEN 1 END) AS purchased,
  ROUND(100.0 * COUNT(CASE WHEN s.cart_ts > s.view_ts THEN 1 END)
        / NULLIF(COUNT(s.view_ts), 0), 1) AS view_to_cart_pct
FROM steps s
LEFT JOIN first_touch ft ON ft.user_id = s.user_id
GROUP BY 1;
```

Edge case to name: using the **first** time each step happened can miss a user who viewed, left, came back, carted and bought days later. That's usually acceptable, but a session-level funnel is the stricter alternative.

Follow-up: "Why LEFT JOIN?" "So users with no marketing touch aren't silently dropped. They show as 'unknown'. An inner join would make conversion look different from reality."

Case 5: Top 3 products per category

Prompt: "Find the top 3 products by revenue in each category for last quarter."

Clarify: "Should ties share a place? Should products with very few sales count? Is revenue after refunds?"

```
WITH rev AS (
  SELECT p.category, p.product_id, p.product_name,
         SUM(i.quantity * i.unit_price) AS revenue
  FROM order_items i
  JOIN orders   o ON o.order_id = i.order_id
  JOIN products p ON p.product_id = i.product_id
  WHERE o.status = 'COMPLETED'
    AND o.order_ts >= DATE '2026-07-01' AND o.order_ts < DATE '2026-10-01'
  GROUP BY 1, 2, 3
)
SELECT category, product_name, revenue, rnk
FROM (
  SELECT r.*, DENSE_RANK() OVER (PARTITION BY category ORDER BY revenue DESC) AS rnk
  FROM rev r
) x
WHERE rnk <= 3
ORDER BY category, rnk;
```

Talking points:

- "With DENSE_RANK, a category could return more than 3 rows if there are ties. If the business wants exactly 3, I'd use ROW_NUMBER with a tiebreaker like product_id."
- "Revenue is computed at item grain, so there's no double counting from the join to orders."
- **Alternative:** rank by margin instead of revenue, `SUM(quantity * (unit_price - cost_price))`, if the business cares about profit.

Case 6: Detect possible duplicate transactions (banking)

Prompt: “Operations suspects duplicate card transactions. Find them.”

Clarify: “What counts as a duplicate? Same account, same amount, same merchant within a few minutes? Could legitimate repeat purchases look like this, like two coffees?”

Assume: a likely duplicate is the same account, merchant, amount and direction within 5 minutes of the previous such transaction.

```
WITH ordered AS (
  SELECT txn_id, account_id, merchant, amount, direction, txn_ts,
    LAG(txn_ts) OVER (PARTITION BY account_id, merchant, amount, direction
                      ORDER BY txn_ts) AS prev_ts,
    LAG(txn_id) OVER (PARTITION BY account_id, merchant, amount, direction
                      ORDER BY txn_ts) AS prev_txn_id
  FROM transactions
  WHERE status = 'POSTED'
)
SELECT txn_id, prev_txn_id, account_id, merchant, amount, txn_ts, prev_ts
FROM ordered
WHERE prev_ts IS NOT NULL
  AND txn_ts - prev_ts <= INTERVAL '5 minutes';
```

Why LAG instead of a self-join: “A self-join on account, merchant and amount with a time condition can explode for busy accounts. LAG compares each row only with its immediate predecessor, in one sort.”

Distinguish from exact duplicates: “If the same txn_id appears twice, that’s a pipeline duplicate, not a customer duplicate. That’s a data engineering problem, and I’d check it separately with `GROUP BY txn_id HAVING COUNT(*) > 1`. In my world at TD, telling those two apart matters for regulatory reporting.”

Follow-up: “What about three swipes in 10 minutes?” “Each one is compared with the previous one, so the second and third are both flagged.”

Case 7: End-of-day balance for every account, every day

Prompt: “We have transactions. Produce each account’s end-of-day balance for every day in September, including days with no transactions.”

Assume: opening balance is zero at `open_date`, CREDIT adds and DEBIT subtracts, and only posted transactions count.

```
WITH daily_net AS (
  SELECT account_id, CAST(txn_ts AS DATE) AS txn_date,
         SUM(CASE WHEN direction = 'CREDIT' THEN amount ELSE -amount END) AS net
  FROM transactions
  WHERE status = 'POSTED'
  GROUP BY 1, 2
), spine AS (
  SELECT a.account_id, d.date_day
  FROM accounts a
  JOIN dim_date d
    ON d.date_day >= a.open_date
   AND d.date_day <= COALESCE(a.close_date, DATE '2026-09-30')
  WHERE d.date_day <= DATE '2026-09-30'
)
SELECT account_id, date_day, balance
FROM (
  SELECT s.account_id, s.date_day,
    SUM(COALESCE(n.net, 0)) OVER (PARTITION BY s.account_id ORDER BY s.date_day
                                  ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS balance
  FROM spine s
  LEFT JOIN daily_net n ON n.account_id = s.account_id AND n.txn_date = s.date_day
) x
WHERE date_day >= DATE '2026-09-01';
```

Key explanation: “The spine starts at each account’s open date, not September 1st. Otherwise the running sum would miss all history before September and the balances would be wrong. I filter to September only **after** the running sum.”

Performance note: “For millions of accounts over years, recomputing from day one daily is wasteful. In production I’d store a daily balance snapshot and compute today’s as yesterday’s balance plus today’s net. That’s the incremental version.”

Follow-up: “What about time zones?” “End of day depends on the time zone. For a bank, it’s usually the bank’s business day and cut-off time, not midnight UTC. I’d confirm the business definition.”

Case 8: Loans with three or more consecutive missed payments

Prompt: “Find loans that have missed three or more installments in a row.”

Define: an installment is missed if `amount_paid < amount_due` by the due date. State the assumption about partial payments.

```
WITH status AS (
  SELECT loan_id, installment_no,
    CASE WHEN COALESCE(amount_paid, 0) < amount_due
           OR paid_date IS NULL OR paid_date > due_date
         THEN 'MISSED' ELSE 'PAID' END AS pay_status
  FROM loan_schedule
  WHERE due_date < CURRENT_DATE
), grouped AS (
  SELECT loan_id, installment_no, pay_status,
    installment_no
      - ROW_NUMBER() OVER (PARTITION BY loan_id, pay_status ORDER BY installment_no) AS grp
  FROM status
)
SELECT loan_id, MIN(installment_no) AS first_missed, MAX(installment_no) AS last_missed,
       COUNT(*) AS consecutive_missed
FROM grouped
WHERE pay_status = 'MISSED'
GROUP BY loan_id, grp
HAVING COUNT(*) >= 3;
```

Explain the trick: “installment_no increases by one each month. Within a run of consecutive missed installments, installment_no minus the row number among missed rows stays constant. That constant identifies each run. It’s the gaps-and-islands pattern.”

Assumptions to say: a late payment counts as missed; only installments already due are included; installment numbers have no gaps. If they could have gaps, use `ROW_NUMBER() OVER (PARTITION BY loan_id ORDER BY due_date)` instead of installment_no.

Business link: “Consecutive missed payments drive delinquency buckets like 30, 60 and 90 days past due, which feed regulatory credit risk reporting.”

Case 9: Reconcile source and target tables

Prompt: “After a migration, how would you prove the new table matches the old one?”

Talk track: “Three levels: counts, then totals, then row by row.”

```
-- Level 1 and 2: counts and control totals
SELECT 'source' AS side, COUNT(*) AS n, SUM(amount) AS total, COUNT(DISTINCT txn_id) AS keys
FROM source_txn
UNION ALL
SELECT 'target', COUNT(*), SUM(amount), COUNT(DISTINCT txn_id)
FROM target_txn;

-- Level 3: row-level differences
SELECT COALESCE(s.txn_id, t.txn_id) AS txn_id,
  CASE WHEN t.txn_id IS NULL THEN 'MISSING_IN_TARGET'
       WHEN s.txn_id IS NULL THEN 'EXTRA_IN_TARGET'
       WHEN s.amount IS DISTINCT FROM t.amount
         OR s.txn_ts IS DISTINCT FROM t.txn_ts THEN 'VALUE_MISMATCH'
       ELSE 'MATCH' END AS recon_status
FROM source_txn s
FULL OUTER JOIN target_txn t ON t.txn_id = s.txn_id
WHERE t.txn_id IS NULL OR s.txn_id IS NULL
   OR s.amount IS DISTINCT FROM t.amount
   OR s.txn_ts IS DISTINCT FROM t.txn_ts;
```

Why IS DISTINCT FROM: “Plain <> returns unknown when either side is NULL, so a NULL in one table and a value in the other would be silently treated as matching.”

At scale: “For billions of rows, compare hashes per partition first (for example, a hash of each row’s columns, summed per day). Only drill into days where the hashes differ.”

Your experience: this is exactly the parallel-run discipline behind your phased ADF to Databricks migration. Mention it.

Case 10: Exposure by customer risk segment at the time of each transaction

Prompt: “Risk wants last quarter’s transaction volume by customer risk segment. Customers change segment over time.”

Clarify: “Do they want the segment the customer had **when the transaction happened**, or the segment they have **today**?” This question alone shows senior thinking. Both are valid for different uses: point-in-time for regulatory accuracy, current for targeting.

```
SELECT c.risk_segment,
       COUNT(*)      AS txn_count,
       SUM(t.amount) AS txn_amount
FROM transactions t
JOIN accounts a ON a.account_id = t.account_id
JOIN customer_dim_scd2 c
  ON c.customer_id = a.customer_id
 AND t.txn_ts >= c.valid_from
 AND t.txn_ts <  COALESCE(c.valid_to, TIMESTAMP '9999-12-31')
WHERE t.txn_ts >= DATE '2026-07-01' AND t.txn_ts < DATE '2026-10-01'
  AND t.status = 'POSTED'
GROUP BY c.risk_segment;
```

Validation: “The total transaction count should equal the count without the dimension join. If it’s higher, the SCD2 table has overlapping versions. If it’s lower, there are gaps where a transaction matches no version. I’d run both checks.”

```
-- Overlap check on the SCD2 table
SELECT a.customer_id
FROM customer_dim_scd2 a
JOIN customer_dim_scd2 b
  ON a.customer_id = b.customer_id AND a.customer_sk < b.customer_sk
 AND a.valid_from < COALESCE(b.valid_to, TIMESTAMP '9999-12-31')
 AND b.valid_from < COALESCE(a.valid_to, TIMESTAMP '9999-12-31');
```

Case 11: Flag unusually large transactions

Prompt: “Fraud wants transactions that are unusual for that customer. Write the query.”

Clarify: “Unusual compared to the customer’s own history, or to all customers? How much history do we compare against?”

Assume: compare each debit with that account’s median debit over the previous 90 days, and flag it if it’s more than 5 times the median and above a minimum amount, to avoid flagging a $10 purchase from someone who usually spends $1.

```
WITH hist AS (
  SELECT account_id,
         PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY amount) AS median_debit,
         COUNT(*) AS n_txn
  FROM transactions
  WHERE direction = 'DEBIT' AND status = 'POSTED'
    AND txn_ts >= CURRENT_DATE - INTERVAL '90 days'
    AND txn_ts <  CURRENT_DATE
  GROUP BY account_id
)
SELECT t.txn_id, t.account_id, t.amount, h.median_debit,
       ROUND(t.amount / NULLIF(h.median_debit, 0), 1) AS times_median
FROM transactions t
JOIN hist h ON h.account_id = t.account_id
WHERE t.direction = 'DEBIT'
  AND t.txn_ts >= CURRENT_DATE
  AND h.n_txn >= 10
  AND t.amount > 5 * h.median_debit
  AND t.amount > 500;
```

Why the median, not the average: “One huge past purchase would inflate the average and hide new anomalies. The median resists outliers.”

Alternatives: a z-score (amount minus mean, divided by standard deviation), a percentile threshold such as above the account’s 99th percentile, or new-merchant plus high-amount rules. “In production, SQL rules are a first layer. A fraud team would combine them with a model.”

Edge case: new accounts with little history (excluded by `n_txn >= 10`; they need a different rule).

Case 12: DAU, MAU and stickiness

Prompt: “Product wants daily active users, monthly active users and stickiness (DAU divided by MAU).”

Define: active means at least one event that day. MAU for a day means distinct users active in the 30 days ending that day (rolling), not the calendar month. Ask which one they mean.

```
WITH daily_users AS (
  SELECT DISTINCT user_id, CAST(event_ts AS DATE) AS d
  FROM events
), days AS (
  SELECT DISTINCT d FROM daily_users
), dau AS (
  SELECT d, COUNT(*) AS dau FROM daily_users GROUP BY d
), mau AS (
  SELECT x.d, COUNT(DISTINCT u.user_id) AS mau
  FROM days x
  JOIN daily_users u
    ON u.d >  x.d - INTERVAL '30 days'
   AND u.d <= x.d
  GROUP BY x.d
)
SELECT dau.d, dau.dau, mau.mau,
       ROUND(1.0 * dau.dau / NULLIF(mau.mau, 0), 3) AS stickiness
FROM dau JOIN mau ON mau.d = dau.d
ORDER BY dau.d;
```

Why this is interesting: “You can’t do a rolling COUNT(DISTINCT) with a normal window function in most engines, so I join each day to the previous 30 days of **pre-deduplicated** user-days. Deduplicating to one row per user per day first shrinks the data a lot.”

Performance: “For large products this is still heavy: every day joins to 30 days of users. Options: APPROX_COUNT_DISTINCT; HyperLogLog sketches stored per day and merged over 30 days (BigQuery and Snowflake support mergeable sketches); or a table of each user’s last-active date, updated incrementally.”

Case 13: Do 20% of customers bring 80% of revenue?

Prompt: “Leadership asked whether the 80/20 rule applies to our customers.”

```
WITH cust AS (
  SELECT customer_id, SUM(amount) AS revenue
  FROM orders
  WHERE status = 'COMPLETED'
    AND order_ts >= CURRENT_DATE - INTERVAL '365 days'
  GROUP BY customer_id
), ranked AS (
  SELECT customer_id, revenue,
    SUM(revenue) OVER (ORDER BY revenue DESC, customer_id
                       ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
      / SUM(revenue) OVER () AS cum_share,
    ROW_NUMBER() OVER (ORDER BY revenue DESC, customer_id) AS rn,
    COUNT(*) OVER () AS total_customers
  FROM cust
)
SELECT MIN(rn) AS customers_for_80pct,
       MAX(total_customers) AS total_customers,
       ROUND(100.0 * MIN(rn) / MAX(total_customers), 1) AS pct_of_customers
FROM ranked
WHERE cum_share >= 0.8;
```

Explain: “Sort customers by revenue, compute the cumulative share, and find the first customer where the share reaches 80%. That rank divided by the total tells us what percent of customers drive 80% of revenue.”

Business framing: “If the answer is, say, 8%, then losing a handful of top customers is a real risk. That connects back to the churn case: prioritise those accounts.”

Case 14: First-touch vs last-touch attribution

Prompt: “Which marketing channel should get credit for purchases?”

Clarify: “There’s no single right answer, so this is a business choice. First touch rewards channels that bring people in. Last touch rewards channels that close. I’d show both, and mention multi-touch models.”

```
WITH purchases AS (
  SELECT user_id, event_ts AS purchase_ts
  FROM events WHERE event_type = 'purchase'
), touches AS (
  SELECT p.user_id, p.purchase_ts, m.channel,
    ROW_NUMBER() OVER (PARTITION BY p.user_id, p.purchase_ts ORDER BY m.touch_ts ASC)  AS first_rn,
    ROW_NUMBER() OVER (PARTITION BY p.user_id, p.purchase_ts ORDER BY m.touch_ts DESC) AS last_rn
  FROM purchases p
  JOIN marketing_touches m
    ON m.user_id = p.user_id
   AND m.touch_ts <= p.purchase_ts
   AND m.touch_ts >  p.purchase_ts - INTERVAL '30 days'
)
SELECT channel,
  SUM(CASE WHEN first_rn = 1 THEN 1 ELSE 0 END) AS first_touch_purchases,
  SUM(CASE WHEN last_rn  = 1 THEN 1 ELSE 0 END) AS last_touch_purchases
FROM touches
GROUP BY channel;
```

Assumptions to say: a 30-day lookback window; only touches **before** the purchase count; purchases with no touch don’t appear here, so report them separately as “direct / unattributed” so totals reconcile.

Alternative: linear attribution, giving each touch `1.0 / COUNT(*) OVER (PARTITION BY user_id, purchase_ts)` credit.

Case 15: Customers whose spending grew three months in a row

Prompt: “Find customers whose monthly spend increased for three consecutive months.”

```
WITH monthly AS (
  SELECT customer_id, DATE_TRUNC('month', order_ts) AS month, SUM(amount) AS spend
  FROM orders WHERE status = 'COMPLETED'
  GROUP BY 1, 2
), lagged AS (
  SELECT customer_id, month, spend,
    LAG(spend, 1) OVER w AS s1, LAG(spend, 2) OVER w AS s2, LAG(spend, 3) OVER w AS s3,
    LAG(month, 3) OVER w AS m3
  FROM monthly
  WINDOW w AS (PARTITION BY customer_id ORDER BY month)
)
SELECT DISTINCT customer_id
FROM lagged
WHERE spend > s1 AND s1 > s2 AND s2 > s3
  AND m3 = month - INTERVAL '3 months';
```

The trap you must mention: “LAG looks at the previous **row**, not the previous **month**. If a customer skipped a month, LAG would compare non-adjacent months. The check that the row three back is exactly three months earlier makes sure the months are consecutive.” (If your engine lacks the WINDOW clause, repeat the OVER clause.)

Clarify: “Three increases means four months of data. Or did they mean three months of data with two increases? I’d confirm.” Asking this shows precision.

Alternative: fill missing months with zero spend using a customer by month spine, which treats a skipped month as a drop.

Case 16: Dormant accounts that reactivated

Prompt: “Find accounts that were inactive for 180 days or more and then became active again.”

```
WITH t AS (
  SELECT account_id, txn_ts,
    LAG(txn_ts) OVER (PARTITION BY account_id ORDER BY txn_ts) AS prev_ts
  FROM transactions
  WHERE status = 'POSTED'
)
SELECT account_id, prev_ts AS last_activity_before_gap, txn_ts AS reactivated_at,
       CAST(txn_ts AS DATE) - CAST(prev_ts AS DATE) AS gap_days
FROM t
WHERE CAST(txn_ts AS DATE) - CAST(prev_ts AS DATE) >= 180;
```

Business link: “In banking, dormant accounts that suddenly reactivate are a known fraud and account-takeover signal, and dormancy rules are often regulatory. So I’d confirm the exact dormancy definition with compliance, including whether bank-initiated transactions like interest or fees count as activity. They usually don’t.”

Refinement: exclude fee and interest transactions from “activity,” which changes the answer.

Case 17: Data quality checks for a new table

Prompt: “A new source table landed. What checks would you run before anyone uses it?”

Talk track: “Six categories: volume, completeness, uniqueness, validity, consistency and freshness.”

```
-- Volume: compare with the expected range or the previous load
SELECT CAST(load_ts AS DATE) AS load_date, COUNT(*) AS n
FROM stg_transactions GROUP BY 1 ORDER BY 1 DESC;

-- Completeness: nulls in critical columns
SELECT COUNT(*) AS total,
  SUM(CASE WHEN txn_id     IS NULL THEN 1 ELSE 0 END) AS null_txn_id,
  SUM(CASE WHEN account_id IS NULL THEN 1 ELSE 0 END) AS null_account_id,
  SUM(CASE WHEN amount     IS NULL THEN 1 ELSE 0 END) AS null_amount
FROM stg_transactions;

-- Uniqueness: duplicate business keys
SELECT txn_id, COUNT(*) FROM stg_transactions
GROUP BY txn_id HAVING COUNT(*) > 1;

-- Validity: out-of-range or unexpected values
SELECT * FROM stg_transactions
WHERE amount <= 0 OR direction NOT IN ('CREDIT', 'DEBIT')
   OR txn_ts > CURRENT_TIMESTAMP;

-- Consistency: orphan foreign keys
SELECT s.account_id FROM stg_transactions s
LEFT JOIN accounts a ON a.account_id = s.account_id
WHERE a.account_id IS NULL;

-- Freshness: latest data timestamp
SELECT MAX(txn_ts) AS latest_txn, MAX(load_ts) AS latest_load FROM stg_transactions;
```

Senior point: “Checks should block bad data from moving to the next layer, not just log a warning. In regulatory reporting a wrong number is worse than a late one. I’d also agree thresholds with the business, for example that volume within 20% of the 30-day average is normal.”

Case 18: Incremental load with duplicates and late-arriving updates

Prompt: “Every hour we get a batch of customer updates. It can contain duplicates and late, older versions. Load it into the target correctly.”

```
MERGE INTO dim_customer AS t
USING (
  SELECT customer_id, name, country, segment, updated_at
  FROM (
    SELECT s.*,
           ROW_NUMBER() OVER (PARTITION BY customer_id
                              ORDER BY updated_at DESC, ingest_ts DESC) AS rn
    FROM stg_customer_updates s
  ) x
  WHERE rn = 1
) AS s
ON t.customer_id = s.customer_id
WHEN MATCHED AND s.updated_at > t.updated_at THEN
  UPDATE SET name = s.name, country = s.country,
             segment = s.segment, updated_at = s.updated_at
WHEN NOT MATCHED THEN
  INSERT (customer_id, name, country, segment, updated_at)
  VALUES (s.customer_id, s.name, s.country, s.segment, s.updated_at);
```

Explain the three protections:

1. “ROW_NUMBER deduplicates the batch, so MERGE never sees two source rows for one key. Snowflake and Delta would error or behave unpredictably otherwise.”
2. “`s.updated_at > t.updated_at` means a late-arriving **older** version can’t overwrite a newer one.”
3. “It’s idempotent: rerunning the same batch changes nothing, so retries are safe.”

Follow-ups:

- "What if you need history?" "Then it's SCD Type 2: close the current version by setting valid_to and is_current = false, and insert the new version."
- "What about deletes?" "Either a delete flag from the source with `WHEN MATCHED AND s.is_deleted THEN DELETE`, or a soft delete that sets a flag, which is usually preferred for auditability."

Case 19: Average session length and bounce rate

Prompt: "What's the average session duration, and what share of sessions are bounces?"

Define: session = events with less than 30 minutes between them. Bounce = session with exactly one event.

```
WITH flagged AS (
  SELECT user_id, event_ts,
    CASE WHEN event_ts - LAG(event_ts) OVER (PARTITION BY user_id ORDER BY event_ts)
              <= INTERVAL '30 minutes' THEN 0 ELSE 1 END AS new_session
  FROM events
), sessions AS (
  SELECT user_id, event_ts,
    SUM(new_session) OVER (PARTITION BY user_id ORDER BY event_ts
                           ROWS UNBOUNDED PRECEDING) AS session_no
  FROM flagged
), per_session AS (
  SELECT user_id, session_no,
         MIN(event_ts) AS start_ts, MAX(event_ts) AS end_ts, COUNT(*) AS n_events
  FROM sessions GROUP BY user_id, session_no
)
SELECT COUNT(*) AS sessions,
  AVG(end_ts - start_ts) AS avg_duration,
  ROUND(100.0 * SUM(CASE WHEN n_events = 1 THEN 1 ELSE 0 END) / COUNT(*), 1) AS bounce_rate_pct
FROM per_session;
```

Note on the CASE: "When LAG is NULL for the first event, the comparison is unknown, so the CASE falls to ELSE and correctly starts a new session. I'd still say it out loud, because NULL handling is where these queries break."

Assumption to flag: "A one-event session has zero duration, which pulls the average down. Some teams exclude bounces from the duration average. I'd report both."

Case 20: Design the table behind a CFO dashboard

Prompt: "The CFO wants a daily dashboard of revenue, orders and margin by region and product category, fast to load. How would you build it?"

This is an open design question. Structure it:

1. **Grain:** "One row per day, per region, per category. I'd confirm the lowest level the CFO needs to drill into, because the grain decides everything else."
2. **Source logic:** completed orders only, revenue from order items, margin as revenue minus cost, and currency converted at the day's rate.
3. **Build:** an aggregate table refreshed incrementally.

```
CREATE TABLE agg_daily_sales AS
SELECT CAST(o.order_ts AS DATE)                    AS sales_date,
       c.country                                   AS region,
       p.category,
       COUNT(DISTINCT o.order_id)                  AS orders,
       SUM(i.quantity * i.unit_price)              AS revenue,
       SUM(i.quantity * (i.unit_price - p.cost_price)) AS margin
FROM orders o
JOIN order_items i ON i.order_id = o.order_id
JOIN products   p ON p.product_id = i.product_id
JOIN customers  c ON c.customer_id = o.customer_id
WHERE o.status = 'COMPLETED'
GROUP BY 1, 2, 3;
```

1. **Refresh:** recompute only the last few days each run, to catch late orders and refunds, then MERGE or delete-and-insert those dates.
2. **Performance:** partition or cluster on sales_date. The dashboard reads thousands of rows instead of scanning billions.
3. **Trust:** reconcile daily totals against finance's numbers, and document definitions (what counts as revenue, when refunds apply).

The partner's likely follow-up: "Why COUNT(DISTINCT order_id) and not COUNT(*)?" *"Because after joining to order_items there's one row per item. COUNT(*)* would count items, not orders. That's the grain point again."

Second follow-up: "Orders by category can't be added up across categories. Why?" "One order can contain items from several categories, so it's counted once in each. The dashboard total for orders must come from a separate total, not the sum of the category rows. Revenue and margin are additive, but distinct counts are not." That answer shows real depth.

Part 10: Snowflake for a Databricks Engineer

Tim has spent years on Snowflake. You haven't used it in production. That's fine, **as long as you're honest about it and show that the concepts transfer.** This part gives you enough real understanding to hold a credible conversation and write Snowflake-flavoured SQL.

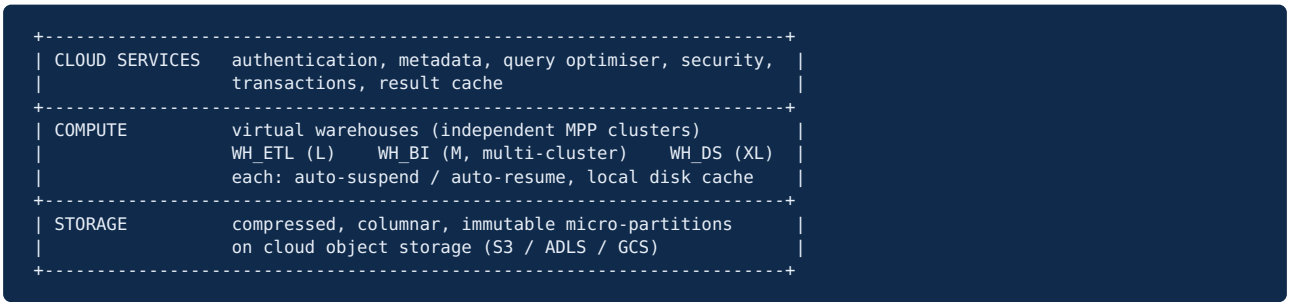
How to position yourself (say this if Snowflake comes up)

> "I should be upfront: my production work is on Databricks and Delta, not Snowflake. But the problems are the same (data layout for pruning, sizing compute, incremental processing, governance), and I understand how Snowflake approaches each of them. For example, micro-partition pruning and clustering keys are the same idea as Delta's file skipping and Liquid Clustering. I'd expect to be productive on Snowflake quickly, and I'd get certified early."

Never imply you've used it on a project. If he asks "have you worked with Snowflake?", the answer is "not in production," followed by what you know.

The architecture in one picture

Snowflake separates storage, compute and services, so each scales independently.



Say it like this:

> "Snowflake has three layers. Storage holds data as compressed, columnar micro-partitions on cloud object storage. Compute is virtual warehouses, independent clusters you size and start or stop separately, so the ETL workload never slows down the BI users. And a cloud services layer handles metadata, optimisation, security and transactions. Because storage and compute are separate, you can give each workload its own warehouse and pay only while it runs. Conceptually it's close to Databricks: Delta files on object storage, with separate job clusters and SQL warehouses on top."

The concepts, mapped to what you already know

Snowflake	What it is	Your Databricks equivalent
Micro-partitions	Immutable storage units (about 50–500 MB uncompressed each), columnar, with min/max and other metadata per column	Parquet files in a Delta table, with file-level statistics
Pruning	Skipping micro-partitions whose min/max can't match the filter	Data skipping
Clustering key	Tells Snowflake to keep a large table ordered by chosen columns so pruning works; maintained by the Automatic Clustering service (costs credits)	Liquid Clustering / Z-order
Virtual warehouse	A compute cluster, sized XS to 6XL; each size up roughly doubles compute and credits per hour	Job cluster / SQL warehouse
Multi-cluster warehouse	Adds clusters automatically for concurrency (many users), not for one big query	SQL warehouse scaling
Auto-suspend / auto-resume	Warehouse stops when idle and starts on the next query; billed per second (60-second minimum each resume)	Auto-termination
Result cache	Identical query on unchanged data returns instantly for 24 hours, no warehouse needed	Databricks query result cache
Warehouse (local disk) cache	Recently read data cached on the warehouse's local storage; lost when it suspends	Disk cache
Time Travel	Query or restore data as it was (default 1 day; up to 90 days on Enterprise edition). `AT(TIMESTAMP => ...)`, `BEFORE(STATEMENT => ...)`, `UNDROP`	Delta time travel, RESTORE
Fail-safe	7 more days after Time Travel for permanent tables, recoverable only by Snowflake support	No direct equivalent
Zero-copy clone	`CREATE TABLE dev_x CLONE prod_x` copies only metadata; storage is shared until data changes	Delta shallow clone
Streams	Change tracking on a table: shows rows inserted, updated or deleted since the stream was last consumed	Delta Change Data Feed
Tasks	Scheduled SQL, chainable into DAGs, can run only when a stream has data	Databricks Workflows
Dynamic Tables	Declarative pipeline: define the query and a target freshness ("target lag"); Snowflake refreshes incrementally	Delta Live Tables / materialised views
Snowpipe	Continuous file ingestion from a stage, triggered by cloud storage events	Auto Loader
Stages	Named locations for files (internal, or external on S3/ADLS/GCS) used by `COPY INTO`	External locations / volumes
VARIANT	Column type for semi-structured data (JSON, Avro, Parquet), queried with `:` paths and `FLATTEN`	Struct/map columns, `from_json`, `explode`

Snowflake	What it is	Your Databricks equivalent
Snowpark	DataFrame API in Python, Java or Scala that runs inside Snowflake	PySpark
RBAC, masking and row access policies	Role hierarchy; dynamic masking of columns by role; row-level filters	Unity Catalog grants, column masks, row filters
Secure Data Sharing	Share live data with another account without copying	Delta Sharing

Performance on Snowflake

The order to think in (same ladder as Part 7):

1. **Is pruning working?** In the Query Profile, compare *partitions scanned* with *partitions total*. Poor pruning is the most common cause of slow, expensive queries.
2. **Are filters sargable?** Functions on filter columns or implicit casts block pruning, exactly like on Databricks.
3. **Is it spilling?** "Bytes spilled to local storage" or, worse, "remote storage" means the warehouse doesn't have enough memory for this query. Reduce the data first; then consider a bigger warehouse.
4. **Is a join exploding?** The profile shows row counts per operator. A join outputting far more rows than its inputs is a grain problem.
5. **Is the warehouse right-sized?** Scale **up** (bigger size) for a single heavy query; scale **out** (multi-cluster) for many concurrent users. Neither fixes bad pruning.
6. **Clustering keys** only for very large tables (typically terabytes) that are filtered on a column the data isn't naturally ordered by, where pruning is measurably poor. Check with `SYSTEM$CLUSTERING_INFORMATION`. Automatic clustering costs credits, so justify it.
7. **Search Optimization Service** for highly selective point lookups (find one ID in billions of rows).
8. **Materialized views** for repeated expensive aggregations on one table.

Say it like this:

> "On Snowflake I'd open the Query Profile first. The biggest wins are usually pruning (partitions scanned versus total), spilling, and exploding joins. I'd fix the query and data layout before touching warehouse size, because a bigger warehouse just runs the same waste faster and at twice the price."

Cost on Snowflake

- **Compute is the main cost**: credits per second while a warehouse runs. So: auto-suspend after a short idle period, one warehouse per workload, right-sized, and **resource monitors** that alert or suspend at a credit limit.

- **Storage** is cheaper, but Time Travel and Fail-safe retain changed data, so very frequently rewritten tables can accumulate storage. Use **transient tables** (no Fail-safe) for staging data that can be recreated.
- **Serverless features** (Snowpipe, automatic clustering, search optimisation, serverless tasks) bill separately. Watch them.

> "Snowflake cost control is the same discipline as my Databricks right-sizing work: measure cost per workload, right-size compute to what the workload actually needs, suspend idle compute, and reduce the work itself."

Snowflake SQL you should be able to write

QUALIFY (filter on a window result)

```
-- latest record per customer
SELECT *
FROM stg_customer
QUALIFY ROW_NUMBER() OVER (PARTITION BY customer_id
                           ORDER BY updated_at DESC, load_ts DESC) = 1;
```

Semi-structured JSON with VARIANT and FLATTEN

```
-- raw_events(payload VARIANT), each payload has an array "readings"
SELECT
  e.payload:device_id::STRING           AS device_id,
  e.payload:event_ts::TIMESTAMP_NTZ     AS event_ts,
  r.value:metric::STRING                AS metric,
  r.value:val::FLOAT                    AS reading
FROM raw_events e,
     LATERAL FLATTEN(input => e.payload:readings) r;
```

Say: "The colon path navigates the JSON, `::` casts it, and FLATTEN turns each array element into its own row, like `explode` in Spark."

Dates

```
DATEADD('day', -30, CURRENT_DATE)             -- 30 days ago
DATEDIFF('day', start_date, end_date)         -- whole days between
DATE_TRUNC('month', event_ts)                 -- first day of the month
LAST_DAY(order_date)                          -- last day of the month
```

A date spine without a calendar table

```
SELECT DATEADD('day', SEQ4(), DATE '2026-01-01') AS date_day
FROM TABLE(GENERATOR(ROWCOUNT => 365));
```

Useful functions

Function	Use
`IFF(cond, a, b)`	Short CASE

Function	Use
`TRY_TO_NUMBER`, `TRY_TO_DATE`, `TRY_CAST`	Return NULL instead of failing on bad data. Great for data quality checks
`EQUAL_NULL(a, b)`	NULL-safe equality
`MEDIAN(x)`, `PERCENTILE_CONT(0.9) WITHIN GROUP (ORDER BY x)`	Medians and percentiles
`APPROX_COUNT_DISTINCT(x)`	Fast approximate distinct counts
`LISTAGG(x, ', ') WITHIN GROUP (ORDER BY x)`	Concatenate values in a group
`ARRAY_AGG`, `OBJECT_CONSTRUCT`	Build arrays and JSON objects
`CONDITIONAL_TRUE_EVENT(cond) OVER (...)`	Running count of when a condition was true. Perfect for sessionization
`CONDITIONAL_CHANGE_EVENT(col) OVER (...)`	Increments whenever a value changes. Perfect for gaps and islands on status changes
`MATCH_RECOGNIZE`	Pattern matching over ordered rows (advanced)

Sessionization with CONDITIONAL_TRUE_EVENT:

```
WITH g AS (
  SELECT user_id, event_ts,
         DATEDIFF('minute',
                  LAG(event_ts) OVER (PARTITION BY user_id ORDER BY event_ts),
                  event_ts) AS gap_min
  FROM events
)
SELECT user_id, event_ts,
       CONDITIONAL_TRUE_EVENT(gap_min > 30)
         OVER (PARTITION BY user_id ORDER BY event_ts) AS session_no
FROM g;
```

Say: “In standard SQL I’d flag each new session with a CASE and take a running SUM of the flags. Snowflake’s CONDITIONAL_TRUE_EVENT does the counting for me: it increases by one every time a gap exceeds 30 minutes, so session numbers start at 0 for each user.” Knowing the Snowflake shortcut impresses a Snowflake person, but **always know the portable version too** (Part 8, pattern 6). If you’re unsure of a function’s exact behaviour, write the portable version and mention the shortcut.

Incremental pipeline with a stream and a task

```
CREATE OR REPLACE STREAM stg_orders_stream ON TABLE stg_orders;

CREATE OR REPLACE TASK merge_orders
  WAREHOUSE = wh_etl
  SCHEDULE = '15 MINUTE'
  WHEN SYSTEM$STREAM_HAS_DATA('stg_orders_stream')
AS
MERGE INTO orders t
USING (
  SELECT * FROM stg_orders_stream
  WHERE METADATA$ACTION = 'INSERT'
  QUALIFY ROW_NUMBER() OVER (PARTITION BY order_id ORDER BY updated_at DESC) = 1
) s
ON t.order_id = s.order_id
WHEN MATCHED AND s.updated_at > t.updated_at THEN UPDATE SET
  status = s.status, amount = s.amount, updated_at = s.updated_at
WHEN NOT MATCHED THEN INSERT (order_id, status, amount, updated_at)
  VALUES (s.order_id, s.status, s.amount, s.updated_at);

ALTER TASK merge_orders RESUME;
```

Explain:

- “The stream records changes to the staging table since it was last consumed. When the MERGE reads from it inside a committed statement, the stream’s offset moves forward, so each change is processed once.”
- “Updates appear in a stream as a DELETE plus an INSERT pair, with METADATA$ISUPDATE set to true. Taking the INSERT rows gives the new values.”
- “The task only runs when the stream has data, so an idle pipeline costs nothing.”
- “Same three protections as my Databricks MERGE: dedupe the batch, only let newer versions overwrite, and make reruns safe.”

Alternative: “A Dynamic Table would do this declaratively. You write the SELECT and a target lag, like 15 minutes, and Snowflake maintains it incrementally. Simpler, less control.”

Loading files

```
COPY INTO raw_events
FROM @landing_stage/events/
FILE_FORMAT = (TYPE = 'JSON')
ON_ERROR = 'CONTINUE';
```

Say: “COPY INTO keeps load metadata, so it won’t reload the same file twice, which makes loads idempotent by default. For continuous arrival I’d use Snowpipe instead of scheduled COPY. ON_ERROR = CONTINUE skips bad rows; for regulated data I’d rather route them to a rejects table and alert, so nothing disappears silently.”

Governance and security on Snowflake

- **RBAC:** privileges are granted to roles, roles to users; roles form a hierarchy (ACCOUNTADMIN at the top, then SECURITYADMIN, SYSADMIN, and custom functional roles). Best practice: access roles per object set, functional roles per job, never work as ACCOUNTADMIN.
- **Dynamic data masking:** a masking policy on a column shows real values to authorised roles and masked values to others. Ideal for PII and health data.

- **Row access policies:** filter rows by role (for example, a clinic only sees its own patients).
- **Secure views:** hide the view definition and prevent certain optimisations from leaking data.
- **Object tagging:** tag columns as PII or sensitive, and attach masking policies by tag.
- **Constraints:** primary and foreign keys are **not enforced** (only NOT NULL is). Uniqueness must be checked in the pipeline. That's a great point to raise, because it's exactly why your DQ engine exists.

Snowflake versus Databricks: the balanced answer

> "They've converged a lot. Snowflake is SQL-first and very simple to operate: no clusters to tune, excellent concurrency for BI, strong governance and data sharing. It's often the fastest way for an analytics-heavy organisation to get value. Databricks is Spark-first and more flexible for heavy engineering, streaming, machine learning and open table formats, with more knobs to tune. Many organisations use both. Snowflake has added Python through Snowpark and supports Iceberg tables, and Databricks has strong SQL warehouses. I'd choose based on the client's workloads, skills and existing estate rather than a preference."

If he asks "which do you prefer?":

> "I know Databricks deeply, so I'm fastest there. But for a client whose main need is governed BI on structured data, with a SQL-skilled team, I'd happily recommend Snowflake. It's simpler to run and that matters for the client after we leave."

Snowflake interview questions with answers

1. "How does Snowflake decide which data to read?"

> "Each micro-partition has metadata: min and max values per column, and more. The optimiser compares the query's filters to that metadata and skips micro-partitions that can't contain matching rows. That's pruning. It works best when the data is naturally ordered by the filter column, like date on append-only data."

2. "When would you add a clustering key?"

> "Only when a large table is frequently filtered on a column that the data isn't ordered by, and the Query Profile shows poor pruning. Clustering keys have an ongoing cost (automatic reclustering uses credits), so I'd measure pruning before and after to prove it's worth it. For small tables it's rarely useful."

3. "A dashboard is slow in the morning when everyone logs in."

> “That’s concurrency, not a single heavy query. I’d look at queuing in the warehouse. A multi-cluster warehouse scales out for concurrency. I’d also separate BI from ETL warehouses, so a morning load doesn’t compete with users, and check whether the dashboard’s queries can hit the result cache or a pre-aggregated table.”

4. “A single report query is slow.”

> “Query Profile first: pruning, spilling, exploding joins. Fix the SQL or layout. Only if it’s genuinely compute-bound and spilling after that would I scale the warehouse up a size, and check that the runtime roughly halves, otherwise the extra cost isn’t buying anything.”

5. “How would you build incremental loads on Snowflake?”

> “Land files with Snowpipe or COPY INTO, track changes with a stream on the staging table, and MERGE into the target on a schedule with a task, deduplicating the batch and only letting newer versions overwrite. Or, for simpler cases, a Dynamic Table with a target lag.”

6. “Someone deleted rows from a production table an hour ago.”

> “Time Travel. Query the table as it was before the statement, for example with `BEFORE(STATEMENT => '<query_id>')` or `AT(OFFSET => -3600)`, and restore the rows, or clone the table at that point and swap. If the table was dropped, UNDROP. Then fix access, so that role can’t delete production data again.”

7. “How do you create a dev environment with production data safely?”

> “Zero-copy clone the production database. It’s instant and costs no extra storage until data changes. Then apply masking policies, so developers never see real PII.”

8. “How do you handle JSON from devices on Snowflake?”

> “Land it as VARIANT, so schema changes in the source don’t break loading. Then parse the fields I need into typed columns in the next layer with colon paths and casts, and FLATTEN arrays into rows. Typed columns in the curated layer give better pruning and simpler queries for BI.”

9. “How do you secure patient data on Snowflake?”

> “Least-privilege roles, masking policies on identifying columns (applied by tag, so new PII columns get protected automatically), row access policies so each clinic or region sees only its own data, and de-identified or pseudonymised keys in the analytics layer. Plus access auditing through the account usage views.”

10. “Primary keys aren’t enforced in Snowflake. Does that matter?”

> “Yes. It means duplicates can arrive silently, and joins can fan out. So uniqueness has to be tested in the pipeline: a check that fails the load if a key isn’t unique. That’s the same reason I built a DQ engine at TD, and it’s even more important on a platform that doesn’t enforce constraints.”

11. “What’s the difference between a view, a materialized view and a dynamic table?”

> “A view runs its query every time. A materialized view stores the result of a query on a single table and Snowflake keeps it up to date automatically. It suits repeated aggregations. A dynamic table can handle joins and complex transformations, and refreshes to a target freshness. It’s more like a declarative pipeline step.”

12. “Snowflake or Databricks for this client?” Use the balanced answer above, then ask about their workloads and team.

13. “What would you do in your first month on a Snowflake engagement?”

> “Understand the client’s goals and current pain first. On the technical side, get hands-on quickly: go through the account’s structure, roles and warehouses, look at the most expensive queries in the query history, and read the pipelines. I’d also get the SnowPro Core certification early. And I’d lean on the Snowflake practice’s standards rather than inventing my own.”

Part 11: Data Modeling Masterclass

Data modeling is on Tim's current engagement and in his whole background (BI, Power BI, enterprise data). A very likely problem shape is: **"Here's a business. Design the model. Now write queries on it."** This part teaches you to do that fluently, then gives you two full worked exercises in Tim's domains.

The method: seven steps for "design a model for X"

Say the steps out loud as you go. The structure is half the marks.

1. **Business questions first.** "What decisions will this data support? Give me three or four questions the business wants answered."
2. **Business processes.** Identify the events being measured: enrolling, ordering, using a device, servicing it.
3. **Grain for each process.** "One row per ___." Declare it explicitly and early. This is the most important sentence in the whole answer.
4. **Dimensions.** The who, what, where, when, how: the context you slice by.
5. **Facts (measures).** The numbers at that grain, and whether each is additive.
6. **History.** Which attributes change over time, and does the business need what was true then (SCD2) or just what's true now (SCD1)?
7. **Validate against the questions.** Walk each business question through the model and show it can be answered simply.

Opening line to use:

> "Before drawing anything, let me get three or four business questions, because they decide the grain. Then I'll pick the processes, declare the grain for each fact, and build the dimensions around them."

Kimball essentials you must explain clearly

Facts and dimensions

- **Fact table:** measurements of a business process at a declared grain. Mostly foreign keys to dimensions plus numeric measures. Long and narrow.
- **Dimension table:** descriptive context. Wide and relatively short. Business users filter and group by these attributes.
- **Star schema:** one fact surrounded by denormalised dimensions. Simple for users and BI tools, and fast.
- **Snowflake schema:** dimensions normalised into sub-dimensions (product → category → department). Saves a little storage, costs more joins. Usually not worth it for BI.

Why star schemas suit Power BI and other BI tools (Tim's world):

> “BI tools are built around star schemas: filters flow from dimensions to facts through one-to-many relationships, measures aggregate cleanly, and users understand it. Snowflaked or many-to-many models create ambiguous filter paths and slow, confusing reports. So even if the integration layer is something else, like Data Vault, I’d always present a clean star to the BI layer.”

The four fact table types

Type	Grain	Example	Behaviour
Transaction	One row per event	One row per enrolment in a course; one row per device error event	Insert-only; most detailed
Periodic snapshot	One row per entity per period	One row per student per term; one row per device per day	Regular, predictable; great for trends and balances
Accumulating snapshot	One row per process instance, updated as it moves through milestones	One row per student per degree program, with admit, first enrolment and completion dates; one row per service case, with opened, diagnosed, shipped and closed dates	Rows are updated; great for durations and pipeline stage analysis
Factless	One row per event or coverage, with no measures	Student attended a class; a device was eligible for a campaign	Counts and “what didn’t happen” analysis

Say: “I choose the fact type from the question. Trends over time want a periodic snapshot. How long things take wants an accumulating snapshot. Detailed analysis wants the transaction grain.”

Additivity

- **Additive:** sum across every dimension (revenue, credit points, usage hours).
- **Semi-additive:** sum across some dimensions but not time (account balances, enrolled headcount in a periodic snapshot). Across time you take the latest or the average, not the sum.
- **Non-additive:** ratios and percentages (pass rate, GPA). **Store the components** (passed count, attempted count) and compute the ratio at query time.

> “I never store a ratio as a measure to be summed. Averaging averages gives wrong answers. I store numerator and denominator and compute the ratio in the report.”

Dimension techniques

- **Surrogate keys:** integer keys generated in the warehouse, separate from source business keys. They handle SCD2 versions, unknown members and source key changes.

- **Conformed dimensions:** one shared dimension (date, student, product) used by many facts, so numbers line up across reports.
- **Role-playing dimension:** one dimension used in several roles, like the date dimension as enrolment date, census date and completion date.
- **Degenerate dimension:** an identifier kept in the fact with no dimension table, like an order number or case number.
- **Junk dimension:** a small table combining low-cardinality flags (is_online, is_international), instead of cluttering the fact with them.
- **Bridge table:** resolves many-to-many relationships, like a patient with several diagnoses, or a course taught by several lecturers, with an allocation weight if you need to split measures.
- **Mini-dimension:** rapidly changing attributes (like a student's current academic standing band) split out from a large dimension, to avoid exploding SCD2 history.
- **Unknown / inferred members:** a row like key -1 "Unknown," so facts with a missing dimension still load. For late-arriving dimensions, insert an inferred member with the business key and update it when the real record arrives.

Slowly changing dimensions

Type	Behaviour	Use when
0	Never changes	Original values, like date of birth or original enrolment date
1	Overwrite	Corrections, or when history doesn't matter (typo in a name)
2	New row per change, with valid_from, valid_to, is_current	History matters (residency status, program, device firmware, clinic)
3	Previous value kept in an extra column	Only "current and previous" needed, like a one-off reorganisation
4	History in a separate table	Rapidly changing attributes
6	1 + 2 + 3 combined: SCD2 rows plus a current-value column on every row	Reports need both "as it was" and "as it is now" easily

> "For each attribute I ask: if this changes, does the business need to report on the old value for past events? If yes, SCD2. If it's a correction or history doesn't matter, SCD1. Choosing per attribute avoids tracking history nobody needs."

Kimball, Inmon and Data Vault in one answer

> “Inmon builds a normalised enterprise warehouse first, then departmental marts from it: strong integration, slower to deliver. Kimball builds conformed dimensional marts directly: faster to deliver and easy to use, but integrating many changing sources can get painful. Data Vault is an integration layer designed for many changing sources and auditability: insert-only hubs, links and satellites, with star-schema marts on top for consumption. At TD we use Data Vault underneath and marts on top, because of regulatory audit needs and fifteen-plus sources. For most analytics clients with a manageable number of sources, Kimball directly is simpler and faster.”

One big table (OBT)

> “Some teams flatten everything into one wide table for BI. It’s simple for users and fast on columnar engines like Snowflake. The costs are duplicated dimension data, painful SCD handling, and it only serves the questions it was built for. I’d use it as a presentation convenience on top of a star, not as the core model.”

Exercise A: A university (Tim’s past engagement area)

Prompt: “A university wants to understand enrolments, student progression and course outcomes. Design the model.”

Step 1: business questions (say them)

1. How many students are enrolled by faculty, program and term, domestic versus international?
2. What proportion of first-year students continue into second year (retention)?
3. Which courses have the lowest pass rates?
4. How long do students take to complete their degree?
5. Which students are at risk (falling grades, reduced load)?

Steps 2–5: processes, grain, dimensions, facts

```
                        dim_term           dim_date (role-playing)
                           |                   |
dim_student ---- FACT_COURSE_ENROLMENT ---- dim_course_offering ---- dim_course ---- dim_faculty
(SCD2)       grain: one row per student          (course + term        (SCD1/2)
   |               per course offering            + campus + mode)
   |          measures: credit_points, grade_points, is_passed,
   |                    is_withdrawn, is_failed, enrolment_count (=1)
   |
   +---- FACT_STUDENT_TERM (periodic snapshot)
   |     grain: one row per student per term
   |     measures: credit_points_enrolled, credit_points_passed, term_gpa,
   |               enrolment_status (enrolled / leave / withdrawn), is_enrolled
   |
   +---- FACT_PROGRAM_PROGRESS (accumulating snapshot) ---- dim_program
         grain: one row per student per program
         milestones: admit_date, first_enrol_date, completion_date, lapse_date
         measures: terms_enrolled, credit_points_completed, days_to_complete
```

Key design decisions to say out loud:

- **Three facts, because the questions need three shapes.** Course outcomes need the enrolment transaction grain. Retention and headcount trends need a per-term snapshot. Time-to-degree needs an accumulating snapshot.
- **dim_student is SCD2** on attributes that change and matter historically: residency status (domestic/international), home campus. Name corrections are SCD1. **Personal data** (name, contact details) belongs in a restricted table or masked by policy, since reports rarely need it.
- **dim_course_offering** is the course in a specific term, campus and mode (online or on campus), because pass rates differ by offering.
- **Store components, not ratios.** is_passed and enrolment_count, so the pass rate is `SUM(is_passed) / SUM(enrolment_count)`. Term GPA in the snapshot is fine for display, but a cumulative GPA must be recomputed from grade points and credit points, never averaged.
- **Headcount is semi-additive:** summing FACT_STUDENT_TERM across terms double-counts students. Report headcount per term, or distinct students over a range.
- **Role-playing dates:** census date, admit date, completion date all point to dim_date.

Step 6: questions the partner will ask

“Why not one fact table?”

> “Because the grains are different. If I put term-level status into the course enrolment fact, a student enrolled in four courses would be counted four times in headcount. Separate facts at clean grains keep every measure additive where it should be.”

“How do you define retention?”

> “I’d confirm the university’s definition, since it’s often a regulatory metric with a formal definition. A common one: of students who commenced a program in year X, the proportion still enrolled, or who completed, in year X+1, excluding those who completed in the first year. The exact exclusions matter, so I’d agree them before building.”

“A student changes program. What happens?”

> “Program progress is at the student-program grain, so a transfer closes one row (with a lapse or transfer date) and opens a new one. Retention can then be measured at program level or institution level, depending on the question: a program transfer counts as retained by the university but not by the original program.”

“How would you handle international student fee data being sensitive?”

> “Keep fee amounts in a separate fact with restricted access, or mask them by role. Most enrolment reporting doesn’t need them.”

Exercise B: A medical device company (Tim’s current domain type)

Prompt: “A company makes hearing devices connected to a phone app. Devices send usage and error data. The company also runs a repair service through clinics. Design the model.”

Step 1: business questions

1. How much are devices being used, per day, by model and region?
2. Which devices have stopped being used, or show declining use (an early warning of patient drop-off)?
3. How quickly are new firmware versions adopted, and do errors fall after an upgrade?
4. How long does a repair take from report to return, by clinic and fault type?
5. Which models have the highest fault rates?

Steps 2–5

```
RAW LAYER: device events as JSON (VARIANT) landed via Snowpipe
     |  parse, type, deduplicate
     v
dim_device (SCD2: firmware_version, assigned_clinic, status)
dim_model, dim_clinic (region), dim_date, dim_fault_type
dim_patient_pseudo (pseudonymised key only; identity held in a restricted system)

FACT_DEVICE_EVENT        grain: one row per device event (error, connection, battery alert)
                         measures: event_count, error_code (degenerate), severity
FACT_DEVICE_USAGE_DAILY  periodic snapshot. grain: one row per device per day
                         measures: hours_used, sessions, errors, battery_min_pct
                         (rows exist even for zero usage, so "no usage" is visible)
FACT_SERVICE_CASE        accumulating snapshot. grain: one row per repair case
                         milestones: reported, received, diagnosed, repaired, shipped, closed
                         measures: days_report_to_close, days_in_repair, is_warranty, cost
```

Key decisions to say:

- **Land raw JSON as VARIANT**, so a new field in the device payload doesn’t break ingestion. Parse the fields we need into typed columns in the curated layer.
- **Deduplicate events** on device ID plus event ID (or device ID plus timestamp plus type), because devices and apps retry when connectivity drops. Assume at-least-once delivery.
- **The daily usage snapshot includes zero-usage days** (generated from a date spine per active device). Otherwise “stopped using” is invisible, because missing days just don’t appear.
- **dim_device is SCD2 on firmware version**, so errors can be attributed to the firmware installed at the time of the error, not the current one. This is the point-in-time join from Part 8.
- **Patient data is health data.** The analytics model uses a pseudonymised patient key. Identity lives in a restricted system. Masking and row access policies limit what clinics and analysts see, following the minimum-necessary principle.

- **Event time versus arrival time:** devices may upload hours later. Aggregate on event time, reprocess a recent window every run so late data lands in the right day, and track data freshness.

Step 6: partner follow-ups

“Why is the usage snapshot better than querying raw events?”

> “Raw events are huge and irregular. Daily usage per device is what the business asks about, it’s small enough for BI to be instant, and it makes zero-usage days explicit. Raw events stay available for detailed investigation.”

“Firmware versions change. How do you compare error rates before and after an upgrade?”

> “Join events to the device dimension version valid at the event time, so each error is attributed to the firmware installed then. Then compare error rates per device-day for the same devices before and after their upgrade, which controls for device differences. That’s a fairer comparison than comparing all old-firmware devices with all new-firmware devices.”

“Data arrives late from devices that were offline. What happens to yesterday’s numbers?”

> “They change as late data arrives, so I’d reprocess a rolling window, say the last seven days, with an idempotent MERGE by device and date, and show a ‘data complete up to’ date on the dashboard, so users know recent days may still move.”

“What would you check for data quality?”

> “Duplicate events, impossible values (negative usage, usage over 24 hours a day, timestamps in the future), devices sending events that aren’t registered (orphans, handled with an inferred member), freshness per region, and sudden volume drops, which might mean an app release broke telemetry rather than patients stopping use.”

Modeling rapid fire

“What’s the grain?” The meaning of one row. Always declare it first. **“Surrogate or natural keys?”** Surrogate keys in dimensions, natural keys kept as attributes for lineage. **“How do you handle a many-to-many?”** A bridge table, with weights if measures must be allocated. **“How do you handle a fact arriving before its dimension?”** Inferred member: insert a placeholder dimension row with the business key, and update it when the real record arrives. **“Why not just use the source system’s normalised model?”** It’s optimised for transactions, not analysis. Queries become deep join chains, and business users can’t navigate it. **“How do you model a status that changes over time?”** Either

SCD2 on the dimension, a periodic snapshot of status, or an accumulating snapshot with milestone dates, depending on whether the question is “what was it then,” “what was it each period,” or “how long did each stage take.” **“How do you make distinct counts work in BI?”** Distinct counts aren’t additive, so compute them in the query at the needed level, or provide a pre-aggregated table per grain. Never sum distinct counts across rows.

Part 12: Domain SQL Cases in Snowflake

Ten cases in Tim's world, built on the models in Part 11 and written in Snowflake SQL. Use the same routine as Part 9: read the prompt, set a 15-minute timer, solve out loud with the 7-step talk track (Part 3), then compare.

> **Dialect rule for the interview:** write Snowflake syntax where you're confident (QUALIFY, DATEADD, DATEDIFF, : paths, FLATTEN, MEDIAN). If you're unsure of a function, write standard SQL and say "in Snowflake there's a shortcut for this." Nobody fails for dialect; people fail for silent guessing.

The tables used in these cases

```
-- University (Exercise A)
dim_student       (student_sk, student_id, residency, home_campus,
                   valid_from, valid_to, is_current)          -- SCD2
dim_course        (course_sk, course_code, course_name, faculty)
dim_program       (program_sk, program_code, program_name, faculty, standard_terms)
fact_course_enrolment (student_id, course_sk, term_code, census_date,
                   credit_points, grade_points, is_passed, is_withdrawn)
fact_student_term (student_id, term_code, term_start_date, program_sk,
                   credit_points_enrolled, credit_points_passed, term_gpa, status)
fact_program_progress (student_id, program_sk, admit_date, first_enrol_date,
                   completion_date, lapse_date)

-- Medical devices (Exercise B)
raw_device_events (payload VARIANT, loaded_at TIMESTAMP_NTZ)
dim_device        (device_sk, device_id, model, firmware_version, clinic_id,
                   valid_from, valid_to, is_current)          -- SCD2
fact_device_event (device_id, event_id, event_ts, event_type, error_code)
fact_device_usage_daily (device_id, usage_date, hours_used, sessions, errors)
fact_service_case (case_id, device_id, clinic_id, fault_type,
                   reported_ts, received_ts, repaired_ts, shipped_ts, closed_ts)
```

Case U1: Headcount by faculty and residency

Prompt: "How many students were enrolled in each faculty in term 2026-T1, split by domestic and international?"

Clarify: "Enrolled as of when? Universities usually count at the census date, after which students are liable for fees. And residency can change, so do we want residency as of the census date?"

```
SELECT c.faculty,
       s.residency,
       COUNT(DISTINCT e.student_id) AS students
FROM fact_course_enrolment e
JOIN dim_course c
  ON c.course_sk = e.course_sk
JOIN dim_student s
  ON s.student_id = e.student_id
 AND e.census_date >= s.valid_from
 AND e.census_date <  COALESCE(s.valid_to, '9999-12-31'::DATE)
WHERE e.term_code = '2026-T1'
  AND e.is_withdrawn = FALSE
GROUP BY c.faculty, s.residency
ORDER BY c.faculty, s.residency;
```

Talk through:

- "COUNT(DISTINCT student_id), because the enrolment fact is one row per student per course. A student in four courses would otherwise count four times."
- "The residency join is point-in-time on the census date, using the SCD2 validity range."
- **Trap to mention proactively:** "A student taking courses in two faculties is counted in both. So the faculty rows won't add up to the university total. If they need a single faculty per student, I'd use the faculty of their program instead."

Case U2: First-year retention by cohort

Prompt: "What percentage of students who started in 2025 were still enrolled in 2026?"

Define: a commencing student is one whose first enrolment in their program was in 2025. Retained means enrolled in any 2026 term, or completed by then. I'd confirm whether completions count as retained. For most definitions, they're excluded from the denominator instead.

```
WITH cohort AS (
  SELECT student_id, program_sk, first_enrol_date, completion_date
  FROM fact_program_progress
  WHERE YEAR(first_enrol_date) = 2025
),
enrolled_2026 AS (
  SELECT DISTINCT student_id
  FROM fact_student_term
  WHERE YEAR(term_start_date) = 2026
    AND status = 'ENROLLED'
)
SELECT
  COUNT(*)                                          AS cohort_size,
  COUNT_IF(c.completion_date < '2026-01-01'::DATE)  AS completed_in_first_year,
  COUNT_IF(e.student_id IS NOT NULL)                AS retained,
  ROUND(100 * COUNT_IF(e.student_id IS NOT NULL)
        / NULLIF(COUNT(*) - COUNT_IF(c.completion_date < '2026-01-01'::DATE), 0), 1)
                                                    AS retention_pct
FROM cohort c
LEFT JOIN enrolled_2026 e ON e.student_id = c.student_id;
```

Talk through:

- "LEFT JOIN, so students who left are kept in the denominator. An inner join would make retention look like 100%."
- "I excluded students who completed within the first year from the denominator, which is common in retention definitions, but I'd confirm that."
- "COUNT_IF is Snowflake's shorthand for SUM(CASE WHEN … THEN 1 ELSE 0 END)."
- **Follow-up: "Retention by faculty?"** "Join the cohort to dim_program and group by faculty. Retention here is at university level; if they want program-level retention, a program transfer should count as a loss for the old program."

Case U3: Courses with the lowest pass rates

Prompt: "Which five courses had the lowest pass rates last year?"

```
WITH course_outcomes AS (
  SELECT c.course_code, c.course_name, c.faculty,
         COUNT(*)            AS attempts,
         COUNT_IF(e.is_passed) AS passes
  FROM fact_course_enrolment e
  JOIN dim_course c ON c.course_sk = e.course_sk
  WHERE e.term_code LIKE '2025-%'
    AND e.is_withdrawn = FALSE
  GROUP BY c.course_code, c.course_name, c.faculty
  HAVING COUNT(*) >= 30
)
SELECT course_code, course_name, faculty, attempts, passes,
       ROUND(100 * passes / attempts, 1) AS pass_rate_pct
FROM course_outcomes
QUALIFY DENSE_RANK() OVER (ORDER BY passes / attempts) <= 5
ORDER BY pass_rate_pct;
```

Talk through:

- "HAVING COUNT(*) >= 30 removes tiny courses where one student changes the rate by 50%. I'd agree the threshold with the business."
- "Pass rate is computed from components, not averaged from offering-level rates."
- "Withdrawals are excluded here. But whether a withdrawal counts as a fail is a policy question, and it changes the answer a lot, so I'd ask."
- "DENSE_RANK keeps ties, so there might be more than five rows."

Case U4: Median time to degree

Prompt: "How long do students take to finish each program?"

```
SELECT p.program_name,
       COUNT(*)                                                  AS completions,
       MEDIAN(DATEDIFF('month', f.first_enrol_date, f.completion_date)) AS median_months,
       PERCENTILE_CONT(0.9) WITHIN GROUP
         (ORDER BY DATEDIFF('month', f.first_enrol_date, f.completion_date)) AS p90_months,
       MAX(p.standard_terms)                                     AS standard_terms
FROM fact_program_progress f
JOIN dim_program p ON p.program_sk = f.program_sk
WHERE f.completion_date IS NOT NULL
  AND f.completion_date >= DATEADD('year', -3, CURRENT_DATE)
GROUP BY p.program_name
ORDER BY median_months DESC;
```

Talk through:

- "Median, not average, because a few students taking many years skew an average badly. P90 shows the slow tail."
- **The survivorship trap (say this, it's senior):** "This only includes students who finished. Students still enrolled after many years, or who dropped out, aren't in it, so the true time-to-degree is understated. For a complete picture I'd also report the share of each cohort that completed within the standard time plus one year."

Case U5: Students at risk

Prompt: "Find students whose results dropped sharply this term, so advisors can contact them."

Define: term GPA fell by 1.0 or more compared with their previous term, **or** they passed less than half of the credit points they enrolled in. Agree thresholds with the advisors.

```
WITH terms AS (
  SELECT student_id, term_code, term_start_date, term_gpa,
         credit_points_enrolled, credit_points_passed,
         LAG(term_gpa) OVER (PARTITION BY student_id ORDER BY term_start_date) AS prev_gpa
  FROM fact_student_term
  WHERE status = 'ENROLLED'
)
SELECT student_id, term_code, prev_gpa, term_gpa,
       credit_points_enrolled, credit_points_passed
FROM terms
WHERE term_code = '2026-T1'
  AND ( term_gpa <= prev_gpa - 1.0
     OR credit_points_passed < 0.5 * credit_points_enrolled );
```

Talk through:

- "LAG compares with the previous enrolled term, not the previous calendar term, so a term on leave doesn't create a false drop."
- "First-term students have no previous GPA, so the GPA test is unknown for them and they're only flagged by the pass-ratio test. That's reasonable, but I'd say it explicitly."
- **Privacy:** "This list contains student-level academic data, so access should be limited to advisors, using a row access policy by faculty."

Case D1: Parse and deduplicate device events

Prompt: "Device events arrive as JSON. Each payload has device_id, and an array of events, each with an event_id, a timestamp, a type and an optional error code. Devices retry uploads, so duplicates happen. Produce a clean event table."

```
SELECT
  r.payload:device_id::STRING              AS device_id,
  ev.value:event_id::STRING                AS event_id,
  ev.value:ts::TIMESTAMP_NTZ               AS event_ts,
  ev.value:type::STRING                    AS event_type,
  ev.value:error_code::STRING              AS error_code,      -- NULL if absent
  r.loaded_at
FROM raw_device_events r,
     LATERAL FLATTEN(input => r.payload:events) ev
WHERE ev.value:event_id IS NOT NULL
QUALIFY ROW_NUMBER() OVER (
          PARTITION BY r.payload:device_id::STRING, ev.value:event_id::STRING
          ORDER BY r.loaded_at) = 1;
```

Talk through:

- "FLATTEN turns each element of the events array into a row. The colon path reads fields, `::` casts them. A missing optional field just comes back NULL."
- "Deduplicate on device plus event ID, keeping the first load. If event IDs weren't reliable, I'd dedupe on device, timestamp and type."
- "Events without an ID go to a rejects table in a real pipeline, not silently dropped."
- **Performance:** "In production this runs incrementally: only newly loaded raw rows, via a stream on the raw table, merged into the clean table on device and event ID, so a retry arriving a day later is still deduplicated."

Case D2: Daily usage including zero-usage days

Prompt: "Build daily usage per device for September, including days a device wasn't used at all."

```
WITH days AS (
  SELECT DATEADD('day', SEQ4(), '2026-09-01'::DATE) AS usage_date
  FROM TABLE(GENERATOR(ROWCOUNT => 30))
),
active_devices AS (
  SELECT device_id, MIN(valid_from)::DATE AS active_from
  FROM dim_device
  GROUP BY device_id
),
spine AS (
  SELECT a.device_id, d.usage_date
  FROM active_devices a
  JOIN days d ON d.usage_date >= a.active_from
),
usage AS (
  SELECT device_id, event_ts::DATE AS usage_date,
         COUNT_IF(event_type = 'SESSION_START') AS sessions,
         COUNT_IF(event_type = 'ERROR')         AS errors
  FROM fact_device_event
  WHERE event_ts >= '2026-09-01' AND event_ts < '2026-10-01'
  GROUP BY 1, 2
)
SELECT s.device_id, s.usage_date,
       COALESCE(u.sessions, 0) AS sessions,
       COALESCE(u.errors, 0)   AS errors
FROM spine s
LEFT JOIN usage u
  ON u.device_id = s.device_id AND u.usage_date = s.usage_date;
```

Talk through:

- "Without the spine, days with no usage simply don't exist, and the business can't see devices that stopped being used."
- "The spine starts at each device's activation, so we don't create fake zero days before the device existed."
- "`SEQ4()` with GENERATOR is a quick date spine in Snowflake. Normally I'd use a proper date dimension."
- **Time zones:** "`event_ts::DATE` uses whatever time zone the timestamp is in. For patients in different time zones, 'a day' should probably be the patient's local day. I'd confirm how timestamps are stored."
- (The 'SESSION_START' event type is an assumption; say you'd confirm the real event types.)

Case D3: Devices unused for 7 or more consecutive days

Prompt: "Find devices that went seven or more days in a row without being used."

```
WITH flagged AS (
  SELECT device_id, usage_date,
         IFF(sessions = 0, 'IDLE', 'USED') AS state
  FROM fact_device_usage_daily
),
islands AS (
  SELECT device_id, usage_date, state,
         DATEADD('day',
                 -ROW_NUMBER() OVER (PARTITION BY device_id, state ORDER BY usage_date),
                 usage_date) AS grp
  FROM flagged
)
SELECT device_id,
       MIN(usage_date) AS idle_from,
       MAX(usage_date) AS idle_to,
       COUNT(*)        AS idle_days
FROM islands
WHERE state = 'IDLE'
GROUP BY device_id, grp
HAVING COUNT(*) >= 7
ORDER BY idle_days DESC;
```

Talk through:

- "It's gaps and islands. Within a run of consecutive idle days, the date minus its row number among idle days is constant, so that constant identifies each run."
- "This relies on the daily table having a row for every day, including zero-usage days, which is why Case D2 matters."
- **Alternative:** "Snowflake's CONDITIONAL_CHANGE_EVENT(state) gives a run number that increments whenever the state changes, which is another clean way to label the islands."
- **Business framing:** "A long idle stretch might mean a patient has stopped wearing the device, which is a clinical follow-up, or that the app stopped syncing, which is a technical issue. I'd check whether the phone app is still connecting before flagging it as a patient issue."

Case D4: Did errors fall after a firmware upgrade?

Prompt: "Firmware 5.2 was released last month. Did it reduce errors?"

Approach: attribute each error to the firmware installed at the time (SCD2 point-in-time), then compare error rates per device-day for devices that upgraded, before versus after their own upgrade.

```
WITH events_fw AS (
  SELECT e.device_id, e.event_ts::DATE AS event_date, e.event_type, d.firmware_version
  FROM fact_device_event e
  JOIN dim_device d
    ON d.device_id = e.device_id
   AND e.event_ts >= d.valid_from
   AND e.event_ts <  COALESCE(d.valid_to, '9999-12-31'::TIMESTAMP_NTZ)
  WHERE e.event_ts >= DATEADD('day', -60, CURRENT_DATE)
),
upgraded AS (
  SELECT DISTINCT device_id FROM dim_device WHERE firmware_version = '5.2'
),
per_device AS (
  SELECT ef.device_id,
         IFF(ef.firmware_version = '5.2', 'AFTER', 'BEFORE') AS period,
         COUNT_IF(ef.event_type = 'ERROR')  AS errors,
         COUNT(DISTINCT ef.event_date)      AS active_days
  FROM events_fw ef
  JOIN upgraded u ON u.device_id = ef.device_id
  GROUP BY 1, 2
)
SELECT period,
       COUNT(DISTINCT device_id)                          AS devices,
       SUM(errors)                                        AS errors,
       SUM(active_days)                                   AS device_days,
       ROUND(SUM(errors) / NULLIF(SUM(active_days), 0), 3) AS errors_per_device_day
FROM per_device
GROUP BY period;
```

Talk through:

- "Point-in-time join, so an error from last month is attributed to the firmware installed last month, not today's version."
- "Rate per active device-day, not raw counts, because the after-period may be shorter."
- "Comparing the same devices before and after controls for differences between devices."
- **Caveats (say them, this is the analytical maturity he's looking for):** "Early upgraders might be different people from late upgraders, and usage patterns change over time, so I'd present this as 'consistent with a reduction of X,' not proof. If the difference is small, I'd check it's more than normal week-to-week variation before calling it a result."

Case D5: Repair turnaround by clinic

Prompt: "How long do repairs take, by clinic, and which open cases are overdue against a 14-day target?"

```
-- Turnaround for closed cases in the last 90 days
SELECT clinic_id,
       COUNT(*)                                                  AS closed_cases,
       MEDIAN(DATEDIFF('day', reported_ts, closed_ts))           AS median_days,
       PERCENTILE_CONT(0.9) WITHIN GROUP
         (ORDER BY DATEDIFF('day', reported_ts, closed_ts))      AS p90_days,
       MEDIAN(DATEDIFF('day', received_ts, repaired_ts))         AS median_days_in_repair
FROM fact_service_case
WHERE closed_ts >= DATEADD('day', -90, CURRENT_DATE)
GROUP BY clinic_id
ORDER BY median_days DESC;

-- Open cases over the 14-day target
SELECT case_id, clinic_id, fault_type, reported_ts,
       DATEDIFF('day', reported_ts, CURRENT_TIMESTAMP) AS days_open,
       CASE WHEN received_ts IS NULL THEN 'AWAITING_DEVICE'
            WHEN repaired_ts IS NULL THEN 'IN_REPAIR'
            WHEN shipped_ts  IS NULL THEN 'AWAITING_SHIPMENT'
            ELSE 'IN_TRANSIT' END                  AS current_stage
FROM fact_service_case
WHERE closed_ts IS NULL
  AND DATEDIFF('day', reported_ts, CURRENT_TIMESTAMP) > 14
ORDER BY days_open DESC;
```

Talk through:

- “This is an accumulating snapshot: one row per case with milestone timestamps, so stage durations are just date differences.”
- “Median and P90 together: the median shows the typical experience, the P90 shows the patients who wait longest.”
- “Splitting total time into stages (waiting for the device, in repair, shipping) shows where the delay actually is, which is what operations can act on.”
- **Survivorship again:** “Measuring only closed cases hides the slowest ones, which are still open. That’s why the second query matters.”

Part 13: Analytical Discussions Without Code

A partner will often skip the keyboard entirely: “How would you approach this?” These are consulting conversations. Your SQL knowledge is the evidence, but the answer is a **plan**.

The framework for any vague analytical question

Use the same eight beats every time. Say them in order and the partner can follow your thinking.

1. **Objective:** what decision will this answer drive, and for whom?
2. **Metric:** define it precisely, including what’s in and out.
3. **Grain:** what is one row of the output?
4. **Data:** which tables, keys and filters, and what you’d check about them first.
5. **Analysis:** the steps (and the SQL pattern behind each).
6. **Edge cases:** duplicates, NULLs, late data, time zones, ties.
7. **Validation:** how you’d know the answer is right before sharing it.
8. **Recommendation:** what you’d tell the client, with appropriate caution about cause and effect.

Scenario 1: The reporting pipeline is slow and the numbers don’t match the source

This is the scenario the Reddit reply predicted, and it’s your daily world.

Model answer:

> “Two problems, and I’d take correctness first, because a fast wrong number is worse than a slow right one, and often the same root cause explains both.
>
> **Correctness.** First, agree on the definition: is the source figure gross or net, which statuses, which cut-off time and time zone? Many ‘mismatches’ are definition differences. Then reconcile top-down: row counts, then control totals, then by day or segment to narrow where the difference sits, then row-level on keys with a full outer join. The usual culprits are duplicates from retries, a join at the wrong grain inflating totals, missing late data, or a filter that excludes more than intended.
>
> **Performance.** Look at the plan or Spark UI for the step that dominates. A join fan-out, which would also explain inflated numbers, creates a huge intermediate result. Then scan volume: are filters pruning? Then shuffle and skew.
>
> **Fix and prove it.** Fix the grain or dedup, rerun, and reconcile against the source until it matches, then show the business owner the reconciliation, not just the new number. Finally, add a reconciliation check to the pipeline so this can’t recur silently.”

Follow-ups:

- *"The difference is only 0.3%. Does it matter?"* "For regulatory reporting, yes, because it must be explainable. For an internal trend dashboard, maybe not. I'd still find the cause, because small differences often signal a systematic bug that grows."
- *"You found duplicates. Where from?"* "Usually retries or overlapping incremental windows. Fix with dedup on the business key and an idempotent MERGE."

Scenario 2: Is this metric change a business event or a data problem?

"Rule out data first, because it's faster and more common than people think.

- **Completeness:** did every source load arrive? Compare daily row counts with normal.
- **Freshness:** is the latest data actually there, or is the last day partial?
- **Definitions and code:** did a pipeline, mapping or filter change recently? Check deployment history.
- **Distribution:** did one segment, region or product move while everything else is flat? A data problem is often concentrated in one source; a business change often shows across related metrics.
- **Corroboration:** do independent measures agree? If orders fell, did payments and shipments fall too?

If all of those are clean, it's likely a real business change, and I'd hand it over with the segments that drove it."

Scenario 3: Two departments define the same metric differently

"This is common, and it isn't a SQL problem first; it's a governance problem. I'd get both definitions written down precisely, then show both numbers side by side from the same data so the difference is visible and explainable. Usually both are valid for different purposes, like finance counting revenue at invoice and sales counting it at booking. The fix is naming: two clearly named metrics with documented definitions and a single owner for each, rather than one ambiguous 'revenue'. I'd bring it to whoever owns the data definitions for a decision, not pick a winner myself."

Scenario 4: "We need real-time reporting"

"I'd ask what decision the data drives and how fast that decision is made. If someone checks a dashboard twice a day, 15-minute or hourly freshness is enough, and much cheaper and simpler. Real streaming is justified when a delay costs money or risk: fraud blocking, stock-outs, operational alerts.

Then I'd show the trade-off in their terms: streaming costs more to build, run and support. Often the answer is a hybrid: a small streaming path for the one metric that truly needs seconds, and batch for everything else, with batch remaining the source of truth. That's exactly the design of my reconciliation project."

Scenario 5: “Why are customers leaving?”

> “First define ‘leaving’. For a subscription it’s cancellation. For retail it’s no purchase for longer than their usual gap. For a bank it could be closing accounts or balances draining.
>
> Then turn it into questions data can answer: 1. **Who is leaving:** churn rate by segment, tenure, product, channel and region. 2. **When:** after how long? Around which events, such as a price change, a service outage or the end of a promotion? 3. **What changed before they left:** fewer logins, smaller orders, complaints, failed transactions. 4. **Compare with stayers:** what’s different about similar customers who stayed?
>
> I’d be careful to present these as patterns, not proven causes. If the client wants to act, test it: try a retention offer on one group and compare with a control group.”

Scenario 6: Compare behaviour before and after a policy change

> “A simple before-and-after average is misleading, because other things change at the same time: seasonality, marketing, economic conditions.
>
> Better options: - **Control group:** compare affected customers with similar unaffected ones over the same period. The difference in their changes is the effect (difference-in-differences). - **Same period last year**, to remove seasonality. - **Enough time on both sides**, and exclude the transition period. - **Check mix shift:** if the customer mix changed, compare within segments.
>
> And I’d state the confidence honestly: ‘consistent with the policy having an effect of about X,’ not ‘the policy caused X.’”

Scenario 7: Duplicates suddenly increased

> “First, what kind of duplicates? Exact duplicate rows with the same ID point to the pipeline: a retry, a replayed batch, or an overlapping incremental window. Same business event with different IDs points to the source, such as a system resending.
>
> I’d check when it started and match it to deployments, source changes or incidents at that time, then check which source or partition they concentrate in. The fix is dedup on the business key plus idempotent loads, and a uniqueness check that fails the pipeline if it happens again.”

Scenario 8: Analyse failed transactions

> "Define failure first: declined by the bank, technical error, timeout, or customer-abandoned. They have very different causes.
>
> Then: failure rate over time, not just counts, since volume changes. Slice by channel, merchant, card type, region, time of day and error code. Look for a step change, which suggests an incident or release, versus a gradual trend. Check retries: one customer retrying five times inflates failure counts, so also measure the share of customers who ultimately failed.
>
> The output for the client: which failure types matter most in money or customers, and which are fixable."

Scenario 9: Is this pipeline output good enough?

> "I'd check four dimensions, each with a concrete test: - **Complete:** row counts versus source and versus normal volume, and no missing partitions or days. - **Accurate:** control totals reconciled with the source, valid ranges, and referential integrity. - **Timely:** landed before its expected-by time. Freshness is measured, not assumed. - **Unique:** no duplicate business keys.
>
> Critical failures block the data from moving downstream. Minor ones warn. That's how my DQ framework at TD works."

Scenario 10: "Why did you choose Data Vault over dimensional modeling?"

The Reddit reply predicted this one directly. It's also on your resume.

Technical version:

> "They solve different problems, and we use both. Data Vault is the integration layer. We have fifteen-plus regulated sources that change over time, and regulators need full history and lineage. Hubs hold business keys, links hold relationships, and satellites hold attribute history. It's insert-only, with record source and load time on every row. A new source mostly means new satellites instead of remodelling, and loads run in parallel because hash keys need no lookups.
>
> Dimensional modeling is the consumption layer. Raw Data Vault is join-heavy and hard for analysts, so star-schema marts sit on top for reporting.
>
> If a client had a few stable sources and mainly needed fast BI, I'd recommend Kimball directly. Data Vault's overhead only pays off with many changing sources and strict audit needs."

Executive version:

> “Data Vault lets us prove to regulators exactly where every number came from and what it looked like on any date, even as source systems change. We then present the data in a simpler shape for reporting.”

Follow-ups:

- *“Did you choose it?”* Be precise about whether the design predates you (see Part 17).
- *“What’s the cost of Data Vault?”* “More tables, more joins, a steeper learning curve, and you still need marts. It’s worth it for audit-heavy, many-source environments, not for everyone.”
- *“Kimball versus Inmon?”* “Inmon builds a normalised enterprise warehouse first, then departmental marts. Kimball builds conformed dimensional marts directly. Data Vault sits between them: a flexible, auditable integration layer feeding dimensional marts.”

Scenario 11: “How did you manage compute costs in Databricks?”

Predicted by the Reddit reply, and it’s on your resume.

30-second summary:

> “I right-size clusters from execution evidence, not from CPU alone. On one production workload I reviewed 170 runs: 99.4% success, about 25 minutes on average, 32 minutes at P95, with peak CPU under 19% and peak memory about 133 GiB on 32-core, 256 GB nodes. It was clearly over-provisioned on CPU, so I recommended a smaller profile that kept memory headroom, accepting 15 to 25 percent longer runtime for a projected 50 to 65 percent compute reduction. On a much heavier satellite workload, the same analysis told me the opposite: don’t downsize aggressively.”

90-second STAR:

> “**Situation:** Cluster sizes on our platform had mostly been set once and never revisited, and compute is a large share of cost.
>
> **Task:** Find genuinely over-provisioned workloads and resize them without putting the regulatory deadline at risk.
>
> **Action:** I pulled the execution evidence for each candidate: number of runs, success rate, average and P95 runtime, peak CPU and peak memory, plus what the job actually does. One workload stood out. Across 170 production runs it succeeded 99.4% of the time, averaged 24.6 minutes with a P95 of 32 minutes, and peaked at under 19% CPU but about 133 GiB of memory, on Standard_E32ds_v4 nodes with 32 cores and 256 GB each. It processes 12 source branches, current and history data, reference lookups and several sequential MERGEs. So CPU was heavily over-provisioned, but memory was genuinely used. I recommended a smaller profile that kept enough memory headroom, and was explicit about the trade-off: runtime expected to rise 15 to 25 percent, which still fits its SLA window, for a projected 50 to 65 percent compute reduction.
>
> **Result:** A clear, evidence-backed recommendation with the trade-off stated upfront. [Say what was implemented and the realised result, if known.] The lesson I apply every time: CPU alone should never drive a downsizing decision.”

The counter-case (this is what makes the story senior):

> “The same evidence stopped me on another workload: a very large retail-loan satellite. Across 51 runs it had a 94.1% success rate, a P95 of 112 minutes and peaked at 131 GiB of memory, and the job itself had a 5-way UNION ALL, 65 window functions and hundreds of joins. That’s heavy shuffle and memory pressure, and the success rate was already lower. Low CPU there did not mean spare capacity. So I rejected aggressive downsizing, kept the E32 node family, and only reduced the worker count conservatively. Saving money by making a critical regulatory job unstable would have been a bad trade.”

Follow-ups:

- *“Why not just look at CPU?”* > “Because Spark jobs can be memory-bound or shuffle-bound with low CPU. The satellite job is the proof: under-used CPU, but heavy memory and shuffle pressure. Downsizing it on CPU would have made it fail more.”
- *“How did you pick the new size?”* > “Keep peak memory comfortably below the new capacity with headroom for data growth, and check that the expected runtime still fits the job’s window in the dependency chain.”
- *“What does ‘projected’ mean? Did it save money?”* > “[Answer truthfully: implemented or not, and the realised result.] The 50 to 65 percent is the projection from node pricing and the recommended profile.”
- *“Your resume says $80K a year. Where does that come from?”* > “[Only if you can show it: the calculation and source.] If you can’t: ‘That’s an annualised estimate across the right-sizing recommendations. The case I can walk you through with full evidence is this one.’”

- *"How would you do this for a client?"* > "Same method: rank workloads by cost, pull runs, success rate, P95, CPU and memory for each, understand what the job does, then resize with the trade-off stated, and validate over several runs before moving on."

Business impact: lower compute cost without destabilising regulatory jobs; a repeatable, evidence-based method. **Technical depth:** CPU versus memory versus shuffle pressure, P95 versus average, Standard_E32ds_v4 (memory-optimised: 32 vCPU, 256 GB), worker count versus node family. **Don't overclaim:** "projected" is not "saved." Never state $80K as a realised saving unless you can show the calculation.

Executive version:

> "We were paying for computing power our jobs didn't use. We sized each job to what it actually needs, cutting cost by half or more on the over-sized ones, and deliberately left the critical, heavy jobs alone so the regulatory deadline was never at risk."

Scenario 12: Define dimensions and measures for a new dashboard

> "Start from the questions the dashboard must answer and who reads it. Measures are the numbers (revenue, orders, margin, customers). Dimensions are how they'll be sliced (date, region, product, channel). Then decide the grain: the lowest level anyone needs to drill into, say day by region by category.
>
> Two traps to call out: distinct counts like customers aren't additive across dimensions, so totals must be computed separately; and every measure needs a written definition agreed with the owner, so the dashboard and finance don't disagree later."

Scenario 13: A client's CFO challenges your number

> "Stay calm and curious. Ask what number they expected and where it comes from. Walk through the definition and the reconciliation. If they know something the data doesn't, like a manual adjustment, include it. If my number is right, show the evidence step by step. If I'm wrong, say so immediately and fix it. Credibility comes from how you handle being challenged."

Part 14: Spark, Databricks and Delta

Tim is a Snowflake person, but “hands-on technical” plus your resume means Spark questions are likely, especially if Subhash joins. These answers tie every concept to your own work, which is what makes them credible.

Spark fundamentals

1. “Explain how Spark executes a job.”

> “The driver takes my code and builds a logical plan. The Catalyst optimiser turns it into an optimised physical plan. Nothing runs until an action, like a write or a count, because transformations are lazy. When an action runs, the plan is split into stages at shuffle boundaries, and each stage runs as tasks, one per partition, on the executors. So: driver plans, executors do the work, stages split at shuffles, tasks map to partitions.”

2. “Narrow versus wide transformations?”

> “Narrow transformations, like filter, select or withColumn, work on each partition independently. No data moves. Wide transformations, like groupBy, join (without broadcast), distinct or repartition, need rows with the same key to end up together, so they shuffle data across the network. Wide transformations create stage boundaries and are where most of the cost lives.”

3. “What is a shuffle and why is it expensive?”

> “Redistributing data across executors by key. Each task writes its output to disk, split by target partition, and the next stage reads those files over the network. So it costs disk, network, serialisation, and a sync point where every task waits for all its inputs. At TD, cutting shuffle from about 500 GB to 200 GB was one of our biggest runtime wins.”

4. “Lazy evaluation: why does it matter?”

> “Because Spark sees the whole chain before running it, it can optimise across steps: push filters down to the source, prune columns, reorder operations. The practical consequence is that errors and slowness show up at the action, not at the line that caused them, so you debug from the plan and the Spark UI.”

5. “What does the Catalyst optimiser do?”

> “It turns a logical plan into an efficient physical one: predicate pushdown, column pruning, constant folding, choosing join strategies, and so on. It’s why DataFrame and SQL code usually beats hand-written RDD code, and why wrapping columns in functions or UDFs hurts: Catalyst can’t see inside a Python UDF, so it can’t optimise around it.”

6. “Repartition versus coalesce?”

> “Repartition does a full shuffle to create exactly N partitions, evenly distributed, and can increase or decrease the count. Coalesce only merges existing partitions without a full shuffle, so it’s cheaper, but it can only decrease the count and may leave partitions uneven. Before writing, coalesce is often enough to reduce file count. Before a heavy join on a key, repartitioning by that key can help.”

7. “How many partitions should you have?”

> “Enough to use all cores, with tasks that aren’t tiny. A common guide is partitions around 100 to 200 MB each, and a few times the number of cores. The default shuffle partition setting of 200 is often wrong in both directions: too many for small data, too few for huge data. Adaptive Query Execution fixes a lot of this by coalescing shuffle partitions at runtime.”

Joins

8. “What join strategies does Spark use?”

> “Broadcast hash join: the small side is copied to every executor and the big side isn’t shuffled, which is the fastest option when one side is small. Sort-merge join: both sides are shuffled and sorted by key, then merged, which is the default for two large tables. Shuffle hash join: both shuffled, one side hashed, used in some cases. And broadcast nested loop or Cartesian joins for non-equi joins, which are very expensive.”

9. “When does Spark broadcast, and what’s the risk?” (your story)

> “Automatically when one side’s estimated size is under the broadcast threshold, 10 MB by default, or when you add a broadcast hint. The risk is memory: the whole table has to fit on the driver and every executor. I learned that directly. I raised the threshold to about 80 MB on a large cluster, and a table bigger than expected got broadcast and caused an out-of-memory failure in production. I reverted it, and now I prefer explicit broadcast hints on tables I know are small, and I test against peak volumes.”

10. “What is data skew and how do you fix it?”

> “When a few keys hold most of the rows, the tasks for those keys take far longer than the rest, so the whole stage waits for them. You see it in the Spark UI as a few tasks with much longer durations and much more shuffle read. Fixes, in order: enable Adaptive Query Execution’s skew join handling, which splits skewed partitions; broadcast the other side if it’s small; handle the hot key separately; or salt the key by adding a random suffix to spread it, joining against a replicated other side.”

11. “What does Adaptive Query Execution do?”

> “It re-optimises during execution using real statistics from completed stages. It coalesces small shuffle partitions, switches a sort-merge join to a broadcast join if one side turns out small, and splits skewed partitions. It’s on by default in recent Spark and Databricks versions, and it fixes a lot of tuning problems automatically.”

Memory, caching and the Spark UI

12. “When do you cache?”

> “Only when the same DataFrame is reused several times and recomputing it is expensive, like a filtered dimension used in several joins. Caching costs memory that’s then unavailable for execution, so caching everything is counterproductive. And I unpersist when done. On Databricks the disk cache also helps automatically for repeated reads of the same files.”

13. “A job fails with out-of-memory. What do you check?”

> “Where: driver or executor. Driver OOM is usually collecting too much data to the driver, or a huge broadcast. Executor OOM is usually a skewed partition, a broadcast that’s too large, or too few partitions for the data. So I’d check the Spark UI for skew and partition sizes, check for collect() calls and broadcasts, then fix the cause: repartition, handle skew, remove the broadcast. Adding memory is the last step.”

14. “Walk me through debugging a slow job in the Spark UI.”

> “Jobs tab to find the slow job, then the stages. For the slowest stage, I look at: duration, shuffle read and write sizes, spill to memory and disk, and the task duration distribution. If the max task time is far above the median, it’s skew. If there’s spill, partitions are too big or memory too small. If shuffle is huge, I look at the SQL tab’s plan to see which join or aggregation causes it, and whether a broadcast or earlier filter would avoid it.”

15. “What’s spill?”

> “When a task’s data doesn’t fit in execution memory, Spark writes intermediate data to disk and reads it back. It’s slow. The fixes are more, smaller partitions, handling skew, or reducing the data before the expensive step.”

Delta Lake

16. “How does Delta provide ACID transactions?”

> “Through the transaction log, the _delta_log folder. Every write adds a new commit file listing which data files were added and removed. A table version is simply the set of files in the log at that point. Readers read a consistent snapshot. Writers use optimistic concurrency: they check at commit time whether a conflicting commit happened, and if so, one fails and retries. During our Liquid Clustering migration we saw exactly that as ConcurrentAppendException.”

17. “What does MERGE do and how do you make it efficient?”

> “It upserts: update matched rows, insert new ones, optionally delete. Under the hood Delta has to find the files containing matching keys and rewrite them. To make it efficient: deduplicate the source first (a source with duplicate keys errors), include a partition or clustering column in the match condition so Delta can skip files, keep the incremental batch small, and cluster the target on the merge key.”

18. “OPTIMIZE, Z-ORDER, VACUUM and Liquid Clustering?”

> “OPTIMIZE compacts small files into larger ones. Z-ORDER, used with OPTIMIZE, co-locates related values so file statistics skip more files, but it has to be re-run and it rewrites data. Liquid Clustering replaces both partitioning and Z-order: you declare clustering keys, can change them without rewriting the table, and clustering happens incrementally. VACUUM deletes files no longer referenced by the log after the retention period, seven days by default, which saves storage but limits how far back time travel works. We migrated our teams from date partitioning to Liquid Clustering.”

19. “Time travel: how would you use it?”

> “Query a table as of a version or timestamp: to debug what changed, reproduce a report as of a date, or RESTORE after a bad write. I built a small utility at TD that uses time travel to diff two versions of a shared reference table before each release, showing added, deleted and modified mappings, which replaced an error-prone manual check.”

20. “Change Data Feed?”

> “When enabled, Delta records row-level changes (inserts, updates with before and after images, deletes) per version, so downstream tables can process only what changed instead of rescanning. It’s the Databricks equivalent of a Snowflake stream.”

21. “Schema enforcement and evolution?”

> “Enforcement rejects writes whose schema doesn’t match the table, which protects consumers. Evolution, with mergeSchema or the auto-merge setting, allows adding new columns deliberately. For regulated data I’d keep enforcement strict and treat schema changes as reviewed changes. Schema mismatches are one of the likely causes of the UNRESOLVED_COLUMN failure we had, which is why I’d validate schemas before a job runs.”

22. “The small-files problem?”

> “Many small files mean many file opens, bigger metadata and slower reads. It comes from frequent small appends, streaming micro-batches or over-partitioning. Fixes: optimised writes and auto-compaction, periodic OPTIMIZE, sensible partitioning or Liquid Clustering, and coalescing before writes.”

Structured Streaming

23. “How does Structured Streaming work?”

> “It treats a stream as an unbounded table and runs the query incrementally in micro-batches. Each batch reads new offsets from the source, like Kafka, processes them, and writes to the sink. Progress and state are stored in a checkpoint, so after a failure it resumes where it left off.”

24. “Watermarks?”

> “For event-time aggregations, a watermark tells Spark how long to wait for late events: for example, a 5-minute watermark keeps a window open until the latest event time seen is 5 minutes past the window’s end. That bounds the state Spark must keep. Events later than that are dropped from the result. In my reconciliation project, I compare the streaming result with a batch recomputation precisely to measure what the watermark drops.”

25. “Exactly-once?”

> “End-to-end exactly-once needs a replayable source (Kafka offsets), checkpointed progress and state, and an idempotent or transactional sink. Delta and Iceberg commits are atomic, so a replayed micro-batch doesn’t create duplicates as long as the write is idempotent.”

26. “Output modes?”

> “Append writes only new, final rows (with aggregations, that needs a watermark so windows can be finalised). Update writes rows that changed. Complete rewrites the whole result each batch, only practical for small aggregates.”

Databricks platform

27. “Job clusters versus all-purpose clusters?”

> “Job clusters start for a scheduled job and terminate when it finishes, and they’re billed at a lower rate. All-purpose clusters are for interactive development and are shared. Running production jobs on all-purpose clusters is a common source of waste. It was part of what we looked at in our cost work.”

28. “What’s Photon?”

> “Databricks’ vectorised query engine written in C++. It speeds up SQL and DataFrame workloads, especially scans, joins and aggregations. It costs more per unit of compute, so I’d check that the runtime saving outweighs the higher rate for a given workload.”

29. “What’s Unity Catalog?”

> “Databricks’ governance layer: one metastore across workspaces for tables, files, models and functions, with fine-grained access control, lineage and audit logging. [Say whether you’ve used it hands-on.]”

30. “Databricks Workflows versus ADF?”

> “Workflows runs orchestration next to the compute: task dependencies across notebooks, Python, SQL and dbt tasks, job clusters per task, and ‘repair run’, which reruns only failed tasks. ADF is better for low-code copying from many external connectors. We migrated our orchestration from ADF to Workflows in phases, one source system at a time.”

31. “Auto Loader?”

> “Incremental file ingestion from cloud storage. It tracks which files are already processed, handles schema inference and evolution, and scales to millions of files, using either directory listing or file notifications. It’s the Databricks equivalent of Snowpipe.”

Quick PySpark you should be able to write

```
from pyspark.sql import functions as F, Window

# Latest record per key (dedup)
w = Window.partitionBy("customer_id").orderBy(F.col("updated_at").desc(), F.col("load_ts").desc())
latest = (df.withColumn("rn", F.row_number().over(w))
            .filter("rn = 1").drop("rn"))

# Broadcast a small dimension
joined = facts.join(F.broadcast(dim_product), "product_id", "left")

# Running total per account
w2 = Window.partitionBy("account_id").orderBy("txn_date") \
           .rowsBetween(Window.unboundedPreceding, Window.currentRow)
running = txns.withColumn("running_balance", F.sum("amount").over(w2))

# Upsert into Delta
from delta.tables import DeltaTable
(DeltaTable.forName(spark, "silver.customers").alias("t")
   .merge(latest.alias("s"), "t.customer_id = s.customer_id")
   .whenMatchedUpdateAll(condition="s.updated_at > t.updated_at")
   .whenNotMatchedInsertAll()
   .execute())
```

Say: “Same patterns as the SQL versions: a window with a deterministic tiebreaker for dedup, a broadcast hint for the small side, an explicit ROWS frame for running totals, and a MERGE that only lets newer versions overwrite.”

Part 15: Python and Live Coding

Anisha said SQL is the focus. But "hands-on problem-solving" could include a short Python exercise. You've said your weak spot in coding is spotting the pattern quickly and moving from a first attempt to an optimised one. So this part gives you a method and six data-engineering-flavoured exercises. Do it if Anisha confirms Python is in scope, or if you have time on Day 5.

The method: think out loud in five steps

1. **Restate with an example.** "So given these three records, I should return this."
2. **Name the pattern.** Counting → dictionary. Deduplication → dictionary keyed on the ID. Ordering or top-N → sort or heap. Ranges or overlaps → sort, then sweep. Sliding window → a deque or two pointers.
3. **Brute force first, out loud,** with its complexity: "The simple way is compare every pair, $O(n^2)$."
4. **Then optimise:** "A dictionary makes each lookup O(1), so it becomes O(n)."
5. **Test edge cases** out loud: empty input, one element, duplicates, ties, bad data.

The single most useful sentence: "Let me get a correct version first, then improve it." Partners prefer a working simple solution plus a clear optimisation over a half-finished clever one.

Exercise 1: Count log lines by status code

Prompt: "Given web log lines, count requests per HTTP status code."

```
from collections import Counter

def count_status(lines):
    counts = Counter()
    for line in lines:
        parts = line.split()
        if len(parts) < 9:          # malformed line: skip (or send to a rejects list)
            continue
        counts[parts[8]] += 1       # status code position in common log format
    return dict(counts)
```

Say: "One pass, O(n) time. A Counter is a dictionary built for counting. I skip malformed lines. In a real pipeline I'd count and log them rather than drop them silently."

Exercise 2: Keep the latest record per key

Prompt: "Given a list of customer records with an id and updated_at, return the latest record per id."

```
def latest_per_key(records):
    latest = {}
    for r in records:
        k = r["id"]
        if k not in latest or r["updated_at"] > latest[k]["updated_at"]:
            latest[k] = r
    return list(latest.values())
```

Say: “Dictionary keyed by ID, keeping the newer record. O(n). It’s the Python version of ROW_NUMBER over the key ordered by updated_at. For ties, I’d add a second field to compare, like a load timestamp, so the result is deterministic.”

Exercise 3: Merge overlapping time intervals

Prompt: “Given maintenance windows as start and end times, merge the overlapping ones.”

```
def merge_intervals(intervals):
    if not intervals:
        return []
    intervals = sorted(intervals)                # sort by start: O(n log n)
    merged = [list(intervals[0])]
    for start, end in intervals[1:]:
        if start <= merged[-1][1]:               # overlaps the last merged interval
            merged[-1][1] = max(merged[-1][1], end)
        else:
            merged.append([start, end])
    return merged

# merge_intervals([(1,3), (2,6), (8,10), (9,12)]) -> [[1,6], [8,12]]
```

Say: “Sort by start, then one sweep: either extend the last interval or start a new one. O(n log n) for the sort. The same idea as gaps and islands in SQL.”

Exercise 4: Moving average over the last k readings

Prompt: “Given a stream of sensor readings, return the moving average of the last k values after each reading.”

```
from collections import deque

def moving_average(values, k):
    window, total, out = deque(), 0.0, []
    for v in values:
        window.append(v)
        total += v
        if len(window) > k:
            total -= window.popleft()
        out.append(total / len(window))
    return out
```

Say: “Keep a running total and a queue of the last k values, so each step is O(1) instead of re-summing the window, which would be O(k). O(n) overall.”

Exercise 5: Find records in one source but not the other (reconciliation)

Prompt: “Given two lists of transaction IDs from source and target, report missing and extra IDs.”

```
def reconcile(source_ids, target_ids):
    s, t = set(source_ids), set(target_ids)
    return {
        "missing_in_target": sorted(s - t),
        "extra_in_target":   sorted(t - s),
        "matched":           len(s & t),
    }
```

Say: “Sets make membership O(1), so this is O(n) instead of comparing every pair. It’s the Python version of a full outer join. For duplicates, I’d compare counts per ID with a Counter instead of sets, because a set hides a duplicated ID.”

Exercise 6: Top N items by frequency

Prompt: “Return the k most frequent error codes.”

```
from collections import Counter
import heapq

def top_k(codes, k):
    counts = Counter(codes)
    return heapq.nlargest(k, counts.items(), key=lambda kv: kv[1])
```

Say: “Count in O(n), then a heap gets the top k in O(n log k), which beats sorting everything when k is small. `Counter.most_common(k)` does the same thing. I’d ask how ties should be ordered.”

Python concepts they might ask

- **List versus tuple versus set versus dict:** mutable ordered list; immutable tuple; unordered unique set with O(1) membership; key-value dict with O(1) lookup.
- **Generators:** produce values lazily with `yield`, so large files can be processed line by line without loading everything into memory. “For a 50 GB file, I’d stream it with a generator, never read it all at once.”
- **Exceptions:** catch specific exceptions, log with context, and don’t swallow errors silently.
- **Testing:** pytest with small hand-made inputs, including edge cases. Be honest: “I used pytest and Great Expectations some time ago and I’m refreshing them.”
- **pandas versus PySpark:** pandas for data that fits on one machine; PySpark for distributed data. The API ideas (filter, group, join, window) are similar.

Part 16: Rapid-Fire Questions

Partners often fire short questions between cases to test fundamentals. Cover the answer, say yours out loud, then check. Aim for 15 to 30 seconds each.

Fundamentals

1. WHERE vs HAVING? WHERE filters rows before grouping. HAVING filters groups after aggregation. Put non-aggregate filters in WHERE so fewer rows get grouped.

2. COUNT(*) vs COUNT(col) vs COUNT(DISTINCT col)? All rows; non-null values of col; distinct non-null values of col.

3. ROW_NUMBER vs RANK vs DENSE_RANK on values 100, 90, 90, 80? ROW_NUMBER: 1, 2, 3, 4. RANK: 1, 2, 2, 4. DENSE_RANK: 1, 2, 2, 3.

4. Table A has key values 1, 1, 2 and table B has 1, 1, 1, 3. How many rows from an INNER JOIN on key? Key 1 gives 2 × 3 = 6 rows, key 2 has no match, so 6 rows. LEFT JOIN: 6 + 1 (the 2 with NULLs) = 7. FULL OUTER JOIN: 7 + 1 (the 3) = 8. This tests whether you understand join multiplication.

5. What does a CROSS JOIN do, and when is it useful? Every combination of rows. Useful for building spines, like every account times every date, or every store times every product.

6. LEFT JOIN, but you add a filter on the right table in WHERE. What happens? Rows without a match have NULLs on the right, and the WHERE filter removes them, so it silently becomes an inner join. Move the filter into the ON clause.

7. Can you use a column alias in WHERE? No, WHERE runs before SELECT. Repeat the expression or wrap it in a CTE. Many engines allow aliases in GROUP BY and ORDER BY.

8. Why does `WHERE x NOT IN (SELECT y FROM t)` return nothing? If any y is NULL, `x <> NULL` is unknown for every row. Use NOT EXISTS.

9. What is `SELECT 10/4` in most engines? 2 in engines with integer division (Postgres, SQL Server). Snowflake and BigQuery return 2.5. To be safe, write `10 * 1.0 / 4` or CAST.

10. AVG of 10, NULL and 20? 15. NULLs are ignored. If NULL should mean zero, use `AVG(COALESCE(x, 0))`, which gives 10.

11. What does COALESCE do? And NULLIF? COALESCE returns the first non-null argument. NULLIF(a, b) returns NULL if a equals b, and is commonly used to avoid division by zero.

12. Difference between DELETE, TRUNCATE and DROP? DELETE removes chosen rows, is logged row by row and can be filtered. TRUNCATE removes all rows quickly and keeps the table. DROP removes the table itself.

13. What is a primary key vs a unique key vs a foreign key? Primary key: unique, not null, one per table. Unique key: unique values, may allow NULL depending on the engine. Foreign key: references another table's key to enforce relationships. Note that Snowflake and BigQuery don't enforce PK or FK constraints, so pipelines must check them. That's a good point to raise.

14. What is normalisation? Name the first three normal forms. Organising tables to remove redundancy and update anomalies. 1NF: atomic values, no repeating groups. 2NF: no partial dependency on part of a composite key. 3NF: no transitive dependency, meaning non-key columns depend only on the key.

15. Star schema vs snowflake schema? A star has a central fact table with denormalised dimensions, so there are fewer joins and faster reads. A snowflake normalises dimensions into sub-dimensions, saving storage but adding joins. Analytics usually prefers star.

16. Fact vs dimension? Facts are measurable events at a grain (transactions, orders) with numeric measures. Dimensions describe context (customer, product, date).

17. SCD Type 1 vs Type 2? Type 1 overwrites, so no history. Type 2 adds a new row per change with valid_from, valid_to and is_current, so full history is kept. Type 2 is standard for regulatory reporting because you must reproduce what was known at the time.

18. What is a surrogate key and why use one? A system-generated key, independent of source business keys. It's stable if the source key changes, handles SCD2 versions, and allows unknown or default members. Data Vault hubs use hash keys of the business key for a similar purpose.

Classic warm-ups (write these fast)

19. Second highest salary.

```
SELECT MAX(salary) FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
-- or DENSE_RANK() = 2, which generalises to Nth
```

20. Nth highest salary per department.

```
SELECT dept_id, name, salary FROM (
  SELECT e.*, DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS r
  FROM employees e) x
WHERE r = 2;
```

21. Employees earning more than their manager. Self-join on `e.manager_id = m.emp_id` and `e.salary > m.salary` (Part 8, pattern 14).

22. Find duplicate emails. `SELECT email, COUNT(*) FROM users GROUP BY email HAVING COUNT(*) > 1;`

23. Delete duplicates, keeping one. Use ROW_NUMBER to find rn > 1 and delete those. In Snowflake or Delta it's often easier to rebuild the table with `rn = 1` using CREATE OR REPLACE or INSERT OVERWRITE.

24. Departments with no employees. LEFT JOIN departments to employees WHERE emp_id IS NULL, or NOT EXISTS.

25. Cumulative salary by hire date. `SUM(salary) OVER (ORDER BY hire_date ROWS UNBOUNDED PRECEDING)`.

26. Percentage of total per row. `100.0 * salary / SUM(salary) OVER ()`, or `OVER (PARTITION BY dept_id)` for share within each department.

Deeper, partner-level

27. Why is `COUNT(DISTINCT)` expensive in distributed engines? Every distinct value has to be brought together in one place to be deduplicated, which requires a shuffle by that value, and memory to track them all. Approximate algorithms like HyperLogLog avoid that.

28. How would you design a query to be idempotent? Running it twice gives the same result: MERGE on business keys, delete-and-insert by partition, no blind appends, and deterministic dedup.

29. What is a watermark in incremental processing? The highest point already processed, such as a max updated_at or ingestion timestamp. The next run reads beyond it, often with a lookback to catch late data.

30. How do you handle late-arriving data? Reprocess a recent window every run (say, the last 3 days), use MERGE so reprocessing is safe, and guard against older versions overwriting newer ones.

31. How do you test SQL? Small hand-made inputs with known expected outputs, including edge cases: NULLs, duplicates, ties, empty groups and boundary dates. Plus data tests in the pipeline: uniqueness, not-null, referential integrity, reconciliation totals.

32. A query returns the right answer on your sample but the wrong one in production. Why? Usually data you didn’t anticipate: duplicates in a supposedly unique key, NULLs, time zones, late data, or a new category value. Check the grain and uniqueness assumptions first.

33. What is a semi-join and an anti-join? A semi-join returns rows from A that have at least one match in B, without duplicating them (EXISTS or IN). An anti-join returns rows from A with no match (NOT EXISTS).

34. How do time zones break analytics? Timestamps stored in UTC but reported by local day shift events across midnight, so daily totals differ between regions and daylight saving creates 23- and 25-hour days. Store in UTC, convert once at the reporting layer, and define the business day explicitly.

35. What is a materialized view’s main risk? Staleness and refresh cost. It’s only worth it if it’s read far more often than the base data changes.

36. When would you not use a window function? When a simple GROUP BY answers the question (it’s cheaper), or when a huge unpartitioned window would push all data to one worker.

37. How do you explain a complex query to a non-technical client? Describe what each step produces in business terms (“first I find each customer’s last purchase, then I compare it with today”), not the syntax.

38. What would you do if the numbers you produced disagree with finance’s numbers? Don’t assume either side is right. Align definitions first (gross vs net, completed vs all, time zone, cut-off date), then reconcile at the lowest grain to find exactly which rows differ.

39. What does EXPLAIN ANALYZE add over EXPLAIN? EXPLAIN shows the estimated plan. EXPLAIN ANALYZE actually runs the query and shows real row counts and timings, so you can spot bad estimates.

40. What’s your approach when you don’t know the answer? Say so, then reason from principles out loud: “I haven’t used that feature, but here’s how I’d expect it to work and how I’d verify it.” Partners respect this far more than bluffing.

Part 17: Resume Defence and Risk Audit

What Anisha's warning really means

"Be aware of everything on your resume" is a polite way of saying: **the partner will pick lines and push until they reach the edge of what you actually did.** At a consultancy this matters more than anywhere else, because Vivanti sells your credibility to clients. A partner is asking one question underneath all the others: *if I put this person in front of a bank's CTO and they say these things, will they hold up?*

The three kinds of resume question

Type	Example	What they're testing
Understanding	"Explain what Liquid Clustering does."	Do you know the technology behind the words?
Ownership	"Walk me through exactly what you did on the cluster right-sizing."	Did **you** do it, or were you nearby?
Evidence	"$80K a year. How did you calculate that? What was the baseline?"	Is the number real, and do you understand it?

Most candidates prepare only for the first type. The second and third are where offers are won and lost.

The evidence ladder (prepare this for every number)

For every metric on your resume, you must be able to answer six questions in under a minute:

1. **Baseline:** what was it before, and over what period?
2. **Change:** what exactly did **you** change?
3. **Measurement:** where did the number come from (Spark UI, run logs, billing, system tables, Jira)?
4. **Period:** over how long was it measured?
5. **Validation:** how did you confirm the change caused it, and not something else?
6. **Trade-off:** what got worse, riskier or more expensive?

If you cannot climb all six rungs for a number, **do not use that number in the interview**. Describe the work and the direction of the result instead.

The "I versus we" rule

Partners listen for pronouns. Use **"we"** for the team's outcome and **"I"** for your specific decisions and actions.

- Weak: "We migrated 150 pipelines." (What did you do?)
- Also weak: "I migrated 150 pipelines." (Alone? Unlikely, and they know it.)

- Strong: “The team migrated the pipelines. I proposed the phased approach by source system, owned the cutover plan for [X], and reviewed every migration PR.”

When the resume says more than you can defend

Your resume has already been submitted, so the partner may read a line aloud that you can’t fully stand behind. **Correcting yourself calmly is far better than defending a number you can’t support.** Partners respect precision. They distrust everything else you say once they catch one inflated claim.

Use this script:

> “Let me be precise about that one. The resume rounds it up. What I can stand behind is [the accurate version], and here’s how I know.”

That sentence costs you nothing, and it earns trust. Bluffing costs you the offer.

> **Never invent details under pressure.** If you don’t remember a number, say “I don’t have the exact figure in front of me, but the method was…” and describe the method. A partner can tell the difference between a forgotten number and a made-up one.

What NOT to do

- Don’t volunteer new numbers you haven’t prepared. Each one opens a new line of questioning.
- Don’t say “the team did most of it” to escape a hard question, then claim the credit elsewhere.
- Don’t use a technical word you can’t explain in the next sentence (“I used AQE” → be ready for “What does AQE do at runtime?”).
- Don’t argue with the partner if they doubt a number. Show the method instead.

The risk audit

I went through your resume against everything you have told me across our past sessions. Most of it is strong and defensible. But some lines will attract hard questions from a partner, and a few conflict with details you’ve given me before. **Settle every red and amber item before Tuesday.**

Summary

Risk	Claim	Why a partner will probe it
High	“4+ years of experience”	Your listed dates add up to about 3 years 8 months
High	“15+ Petabytes”	Very large for one reporting platform, and you’ve never broken it down with me

Risk	Claim	Why a partner will probe it
High	“99.9% SLA for 12 consecutive months”	The UNRESOLVED_COLUMN failure caused an SLA miss. When was it, and how is 99.9% calculated?
High	“Architected” the lakehouse and “Led enterprise transition to Data Vault 2.0”	If the design predates you, these verbs overclaim
High	“Spearheaded… 8,000+ legacy tables”	Huge scope. You’ve never given me details of this programme
High	“Training custom GitHub Copilot agents”	We agreed before: you grounded an agent on repos. You didn’t train a model
High	“$80K annually” and the Legendary Award	The Copilot search verified the right-sizing method and a projected 50–65% reduction on one workload, but not the $80K calculation
High	“SQL Server metadata-driven ingestion framework”	Your repo config lives in job_conf JSON and env_conf YAML. You must explain how both fit
High	Mandelbulb: “Data Engineer”, four months, seven metrics	Earlier we agreed to label it an internship. Seven precise metrics from four months invite scrutiny
Medium	60% runtime, 500 GB to 200 GB shuffle	You told me these came from run logs and the Spark UI. Know which pipelines and the baseline
Medium	“Migrated 150+ ADF pipelines… zero SLA breaches”	Is the migration complete? How many are migrated today?
Medium	“Accelerating Synapse-to-ADLS reads 30–40%”	Still unresolved: development only, or production too?
Medium	“Reducing data staleness by 25%”	How is staleness measured?
Medium	“~75% onboarding effort”	Days before versus after?
Medium	“Mentored 10+ engineers”	Names, levels, what you taught
Medium	“Stack Overflow Gold Medalist & Top Contributor”	Stack Overflow has badges, not medals. You hold one gold badge
Medium	DP-203 certification	Microsoft retired DP-203 on 31 March 2025, and it can no longer be renewed
Medium	Education dates and the 2019–2021 gap	Lambton ran to August 2023 but TD starts May 2023. There’s a gap after your BCA
Medium	Portfolio: “AWS S3 with Glue” and “MinIO/ S3, Iceberg REST catalog”	Which runs where?
Medium	Skills list breadth	Scala, Java, MongoDB, Hudi, Redshift, Lambda, Datadog, LangChain, RAG. Any may be picked
Solid	150 pipelines, 6 AM SLA, regulators, Pod Lead, liquid clustering, broadcast OOM, sprint resequencing, reconciliation project, Medium	Real, detailed and defensible. Lead with these

How to handle each red item

1. “4+ years”

May 2023 to today is about 3 years 4 months. Adding Mandelbulb (September to December 2021) gives about 3 years 8 months. A partner doing quick arithmetic will notice.

> **Say:** “About three and a half years in data engineering: over three years at TD, plus an earlier streaming role.”

If something else makes up the fourth year (a co-op or earlier work), know it precisely. **[YOU FILL: what makes up “4+”?]**

2. “15+ Petabytes”

A partner may ask: “What makes up 15 petabytes? How much does your nightly chain actually process?” You need to know:

- Is 15 PB the platform’s total storage, including all Delta versions and history, or the data your pipelines own?
- Roughly how much data moves each night?

[YOU FILL: source of the 15 PB figure, and nightly volume]

> **If you can’t source it, say:** “That’s the total historical footprint of the lakehouse. The part my pipelines process each night is [X]. The number I’d focus on is 150 pipelines against a 6 AM regulatory deadline.”

3. “99.9% SLA for 12 consecutive months”

99.9% of about 250 business days is fewer than one miss. **One miss in the period makes it roughly 99.6%.** Your UNRESOLVED_COLUMN incident caused a documented SLA miss. If it falls inside the 12 months, the partner may connect the two stories.

[YOU FILL: date of the UNRESOLVED_COLUMN SLA miss, and how 99.9% is calculated]

> **Safer phrasing if needed:** “We hit the 6 AM deadline on every night but [one], and after that miss I put in [the fix] so that failure class couldn’t recur.”

That’s actually a stronger story than a perfect record.

4. “Architected” and “Led enterprise transition to Data Vault 2.0”

In past sessions you described owning, running and improving the platform. You never said you designed it from scratch. A partner may ask: “What did the platform look like before you joined? What decisions did you make?”

[YOU FILL: did Data Vault exist when you joined? Which parts did you design?]

> **If the design predates you, say:** “I own the platform today and have evolved its architecture. The Data Vault foundation was in place. What I led was [source-system flagging, audit lineage, extending it to X sources].”

5. “Spearheaded… 8,000+ legacy tables”

This is a programme-level claim and you’ve never described it to me. Expect: “How many people were on it? What was your role? How many tables are done? How did you prove each migrated table matched?”

[YOU FILL: team size, your role, tables completed versus planned, timeline, validation method]

6. “Training custom GitHub Copilot agents”

Training means updating model weights. You configured and **grounded** agents on repositories (giving them the code and documents as context). A partner who knows AI will catch “trained” immediately.

> **Say:** “I built Copilot agents grounded on our legacy repositories. They didn’t learn new weights. We gave them the code as context and prompts to extract business rules and source-to-target mappings, then engineers verified every output.”

7. “$80K annually” and the Legendary Award

This is your most drillable claim. The Copilot search **verified the method and two strong cases** (a projected 50–65% compute reduction on an over-provisioned workload, and a heavy satellite workload where you rejected aggressive downsizing), but **not** the $80K figure or the 60+ cluster count.

How to handle it:

- Lead with the verified evidence (Part 22, story 22.2). It’s more convincing than a round number.
- If Tim asks about $80K and you can show the calculation (which clusters, cost before and after, annualisation), give it.
- If you can’t: “That’s an annualised estimate across the right-sizing recommendations. Let me show you the evidence on one workload instead.” Never defend a number you can’t derive.

Also, your Legendary Award came from Varun Khanna, tagged “Execute with Speed and Impact.” In earlier sessions it was linked to the Synapse and Spark optimisation work. Now the resume links it to cost savings. **Describe the award exactly as the citation describes it.**

[YOU FILL: the award citation wording; the $80K calculation only if it exists]

8. “SQL Server metadata-driven ingestion framework”

You’ve described SQL Server control tables keyed by pipeline ID, and a source cross-check found configuration in `job_conf/*.json` and `env_conf/*.yaml` in the repo. Both may be true. Know exactly which holds what, for example:

> “The SQL Server control tables hold the pipeline registry and run state: which pipelines exist, their dependencies and last run status. The JSON job configs in Git hold each pipeline’s transformation settings, and the YAML holds environment settings. That way config changes go through code review and CI/CD.”

[YOU FILL: confirm this split is accurate]

9. Mandelbulb

Four months, titled “Data Engineer”, with seven precise metrics (50,000 events per second, 45,000 vehicles, 12 to 48 partitions, 35%, 20%, 90%, 25%). In an earlier session we agreed to relabel it as an internship with the real telematics story. Expect: “You were there four months. How did you measure a 20% reduction in missed alerts?”

> **Say:** “It was a short early-career role, an internship, on a vehicle telematics platform. My piece was [X]. The numbers are what the team observed after the change, and I’m happy to walk through how.”

[YOU FILL: which metrics you can explain the measurement for; drop the others from your spoken answers]

Amber items: one line each

- **60% runtime and 500 to 200 GB:** know which pipelines, the before and after, and that they came from run logs and the Spark UI. Was the 60% an average across all pipelines or a subset?
- **“Zero SLA breaches” migration:** say how many pipelines are migrated today, and whether it’s still in progress.
- **Synapse 30–40%:** answer the open question. Development only, or production too?
- **Staleness down 25%:** define staleness (for example, time between source availability and data landing) and the before and after.
- **Onboarding down ~75%:** for example, “a new source went from about [N] days to about [M] days.”
- **Mentored 10+:** be ready to name what you taught (deployments, code review standards) and one person’s growth.
- **Stack Overflow:** say “one gold, five silver and 17 bronze badges, and about 53,000 people reached.” Don’t say “medalist”.
- **DP-203:** it’s retired. If Credly shows it as expired, say “I earned DP-203 before Microsoft retired it, and I’ve since earned DP-700, its replacement.” That’s a strong answer.

- **Dates:** be ready to explain the Lambton and TD overlap (final term or co-op?) and the 2019–2021 period in one calm sentence each. **[YOU FILL]**
- **Portfolio:** “It runs locally with MinIO as S3-compatible storage and an Iceberg REST catalog in Docker Compose, and [was also deployed / is designed to deploy] on AWS with S3 and Glue.” **[YOU FILL: which is true]**
- **Skills:** see Part 18, skills triage.

> **After this interview, update your resume** so every line matches what you can defend. For Tuesday, you control what you say. Precision on the red items will make the partner trust everything else.

Part 18: Line-by-Line Resume Questions

This part covers the summary, skills, Mandelbulb, education, certifications and achievements. The TD bullets and your portfolio project get full deep dives in Part 19.

The headline and summary

> "Senior / Lead Data Engineer with 4+ years… Lead engineer managing 150+ pipelines across 15+ PB…"

Likely questions:

- "Your title at TD is Data Engineer II. Why 'Senior / Lead'?"
- "What does 'lead engineer' mean in your team? Do you have direct reports?"

Model answer:

> "My title is Data Engineer II, and I'm the Pod Lead. That's technical leadership without direct reports. I own code reviews, releases and technical decisions for the pod, and I mentor the engineers on it. 'Senior' describes the scope I operate at: owning a regulatory platform end to end. I'm careful to say my actual title is Data Engineer II."

That honesty lands well with partners.

Skills triage: know where every skill came from

A partner may point at any skill and say "tell me about your experience with that." For each one, prepare **one honest sentence**: where you used it, and how deeply.

Level	Skills	What to prepare
Daily production use (defend deeply)	SQL, Python, PySpark, Spark, Databricks, Delta Lake, ADF, ADLS, Synapse, Key Vault, Data Vault 2.0, Unity Catalog (if used), RBAC, Git, CI/CD with digital.ai	Full deep dives (Part 19)
Project or earlier-role use (defend moderately)	Kafka, Structured Streaming, Schema Registry and Avro, watermarking, Iceberg, Airflow, Docker, MinIO, S3, Glue, Athena	"I used it in [project or role] for [purpose]." Know the core concepts
Unverified with me (decide now)	Scala, Java, MongoDB, Hudi, Redshift, Lambda, Great Expectations, Datadog, LangChain, RAG, Copilot Studio, SQL Server (as a developer)	**[YOU FILL]** For each: where did you use it? If the honest answer is "only in a course," say "I've worked with it in [course or project], not in production"

Model phrasing for a lighter skill:

> "I've used Hudi in a comparison project, not in production. My production table format is Delta. Conceptually Hudi's strength is record-level upserts with copy-on-write or merge-on-read tables, and I'd need a short ramp-up to use it on a client."

Be ready to compare the three table formats (they're all on your resume):

	Delta Lake	Apache Iceberg	Apache Hudi
Origin	Databricks	Netflix	Uber
Metadata	Transaction log (JSON commits plus checkpoints)	Metadata tree (metadata files, manifest lists, manifests)	Timeline of commits
Strength	Deep Databricks integration, Liquid Clustering, CDF	Engine-neutral, hidden partitioning, partition evolution	Record-level upserts, incremental pulls
Where you used it	TD production	Your portfolio project	**[YOU FILL]**

Mandelbulb Technologies (September to December 2021)

Your three bullets: 50,000+ Kafka events per second from 45,000+ vehicles; Kafka topics repartitioned from 12 to 48 partitions, with latency down 35% and missed alerts down 20%; inline schema validation filtering 90% of malformed records and on-call triage down 25%.

Likely questions:

1. "Four months. What was your role exactly?"
2. "Why did going from 12 to 48 partitions reduce latency?"
3. "What happens to message ordering when you increase partitions?"
4. "How did you size the consumers after repartitioning?"
5. "Where did malformed records go? Were they lost?"
6. "How did you measure 'missed maintenance alerts'?"

Model answers:

Q2: "In Kafka, a partition is the unit of parallelism. Each partition is read by at most one consumer in a group. With 12 partitions, at most 12 consumers could read in parallel, so under peak load a backlog built up. Going to 48 allowed more parallel consumers, so the lag dropped."

Q3: "Kafka only guarantees ordering within a partition. Messages with the same key go to the same partition, so if vehicles are keyed by vehicle ID, each vehicle's events stay in order. But repartitioning changes which partition a key maps to, so during the transition you can briefly get out-of-order events for the same vehicle. You handle that with event-time processing and watermarks, or by planning the change during low traffic."

Q5: "Invalid records shouldn't be silently dropped. They go to a dead-letter or quarantine topic with the error reason, so they can be inspected and replayed. **[YOU FILL: what was actually done]**"

Q1, Q6: [YOU FILL] Describe honestly what you personally built, and only quote metrics whose measurement you can explain.

> **Scale sanity check a partner might do:** 50,000 events per second from 45,000 vehicles is about one event per vehicle per second, which is plausible for telematics. Be ready to explain what an event contained (location, engine diagnostics) and the approximate message size.

Education

Likely questions:

- "Your diploma ran until August 2023 but you started at TD in May 2023. How did that work?"
- "What did you do between your BCA in 2019 and 2021?"

[YOU FILL: one calm, true sentence for each.] For example: "TD was my final-term placement that converted to full-time," if that's what happened. Gaps are normal. Hesitation is what draws attention.

Certifications

Certification	Likely question	Answer guidance
Databricks Data Engineer Professional	"What did the Professional exam cover that the Associate doesn't?"	Production concerns: incremental processing, Delta internals, optimisation, testing and deployment, monitoring and governance
Databricks Spark Developer	"How is a Spark job split into stages?"	Stages split at shuffle boundaries (wide transformations). Each stage runs as tasks, one per partition
DP-203	"Is this still active?"	Retired by Microsoft in March 2025, so it can't be renewed. "I earned it before retirement and have since earned DP-700, its replacement"
DP-700 (Fabric)	"Have you used Fabric in production?"	**[YOU FILL]** If not: "Certified, not yet in production. Its Lakehouse and OneLake map closely to what I do on Databricks and ADLS"
Astronomer Airflow	"Airflow versus ADF versus Databricks Workflows?"	Airflow: code-first, open source, rich dependency control. ADF: low-code, Azure-native copy and connectors. Databricks Workflows: native to Databricks jobs, simpler when everything runs on Databricks

The Credly link means a partner can check. Make sure every listed certification appears there with the right status.

Achievements

- **Legendary Award:** describe it exactly as the citation reads. Say who gave it (Varun Khanna) and what it recognised.
- **Stack Overflow:** “One gold, five silver and 17 bronze badges, and about 53,000 people reached over eight-plus years.” If asked about your gold badge, know which one it is and what earned it. **[YOU FILL]**
- **Medium:** “290+ articles and 8,500+ subscribers. Writing is how I learn: if I can’t explain something simply, I don’t understand it yet.” Be ready to name your two most-read articles. **[YOU FILL]**
- **$80K and 99.9%:** covered in Part 17 (risk audit) and Part 19 (deep dives).

Part 19: Project Deep Dives

Each project has a **2-minute overview** (what you say when asked “tell me about X”), the **5-minute deep dive** beats (when they say “go deeper”), and a question bank. Model answers use only what you’ve told me. Anything marked **[YOU FILL]** you must supply yourself. Don’t guess.

Deep dive 1: The EDF2 Financial Reporting Platform

Resume line: “Architected and operate a Databricks Lakehouse platform hosting 15+ Petabytes of data across 150+ pipelines and 12+ sources, sustaining 99.9% 6 AM regulatory SLA compliance for 12 consecutive months.”

2-minute overview

> “EDF2 is TD’s enterprise data foundation, and I own the Financial Reporting Platform for retail lending. It takes data from fifteen-plus regulated source systems, like loans, deposits and customers, and produces the numbers TD reports to OSFI, the Bank of Canada, CDIC and Statistics Canada.
>
> It runs on Databricks with Delta Lake on ADLS, orchestrated by ADF and increasingly Databricks Workflows. The core model is Data Vault 2.0. The platform is config-driven: pipelines are registered in SQL Server control tables and defined by JSON job configs, so adding a pipeline means adding configuration rather than writing new code.
>
> Every night about 150 pipelines run as a dependency chain that has to finish by 6 AM, because regulatory reports are due. My job is keeping that chain reliable, fast and correct, and leading the pod that builds on it.”

5-minute deep dive beats

1. **Sources and ingestion:** 15+ retail lending systems land via ADF into ADLS.
2. **Control plane:** SQL Server control tables keyed by pipeline ID (registry, dependencies, run state). JSON job configs and YAML environment configs in Git, promoted through CI/CD.
3. **Modeling:** Data Vault 2.0 hubs, links and satellites on Delta, with record source and load timestamps for audit.
4. **Quality:** a declarative DQ engine with a rules table (primary keys, null thresholds, referential checks), quarantine routing for failing rows, freshness alerts against expected-by times, and volume anomaly alerts on count changes.
5. **Observability:** run logs keyed by run ID, so every row can be traced to the run that produced it.
6. **Security:** RBAC through AD / Entra groups, with elevated access requested through ServiceNow.
7. **The deadline:** the nightly chain starts around 2 AM and must be done by 6 AM.

Question bank

Architecture: “Why a lakehouse and not a warehouse like Synapse for everything?”

> “Delta on ADLS gives us cheap, open storage with ACID transactions, full history via time travel, and one copy of data serving both heavy Spark processing and SQL. Synapse Dedicated Pool had concurrency limits that became a bottleneck for us, which is part of why we moved reads to ADLS.”

Architecture: “Why config-driven?”

> “With 150 pipelines, hand-coding each one means 150 slightly different implementations. Config-driven means one tested engine, consistent logging and quality checks everywhere, and new sources added in configuration.”

Troubleshooting: “The chain is running late at 4:30 AM. What do you do?”

> “Check the run log for which step is behind and whether it’s slow or failed. Work out whether it’s on the critical path to 6 AM. Tell reporting stakeholders early if the deadline is at risk. Then fix or rerun: our pipelines are idempotent, so reruns are safe.”

Business: “What happens if you miss 6 AM?”

> “Regulatory reports are delayed, which is a compliance issue for the bank, so there’s an escalation process. That’s why reliability work gets priority over new features on this platform.”

Governance: “How do you prove to an auditor where a number came from?”

> “Every Data Vault record carries its record source and load timestamp, and run logs tie each load to a run ID and config version. So from a reported number we can trace back through the vault to the source system and the load that brought it in.”

Alternatives: “Would you build it the same way today?”

> “Mostly yes. I’d move orchestration fully to Databricks Workflows sooner, and version the config with a full audit trail from day one (who changed a rule, when, and from what). That’s a gap we had to close later.”

Failure: “What’s the worst incident you’ve had?”

> The UNRESOLVED_COLUMN SLA miss (Part 22, story 22.13). Contain, communicate, then prevention, and be honest that the root cause wasn’t confirmed.

Sources: 15+ (confirmed). Use workload evidence instead of volume: 150+ production workloads, about 161 curation jobs, 48 downstream reporting outputs, 35 monitored FRP link jobs. **[YOU FILL]:** your exact role in the design.

Deep dive 2: Data Vault 2.0

Resume line: "Led enterprise transition to Data Vault 2.0, implementing source-system flagging and audit lineage to meet regulatory impact-analysis requirements across retail lending sources."

2-minute overview

> "Regulators expect a bank to show where every reported number came from and what's affected when a source changes. Data Vault 2.0 fits that. Hubs hold unique business keys, links hold relationships between them, and satellites hold the descriptive attributes and their full history. Every row records which source system it came from and when it was loaded.
>
> My part was [the transition work, YOU FILL], and in particular source-system flagging and audit lineage. Every record is tagged with its source, so when a source system changes, we can run an impact analysis showing exactly which vault tables and downstream reports depend on it."

Concepts you must explain without notes

- **Hash keys:** a hash of the business key (we use SHA-256), used as the hub's key. Deterministic, so the same key gets the same hash on any system, which allows parallel loading without lookups.
- **Hashdiff:** a hash of all a satellite's descriptive columns. Compare the incoming hashdiff with the latest stored one. If it differs, insert a new satellite row. That's how change detection works without comparing every column.
- **Insert-only:** raw vault tables are never updated, only inserted. History is never lost, which suits audit.
- **Record source and load date:** on every row, which is the basis of lineage.
- **Business vault:** derived rules on top of the raw vault.
- **PIT (point-in-time) and bridge tables:** helper tables that make querying satellites at a point in time fast.
- **Information marts:** star schemas built on top for reporting.

Question bank

"Why Data Vault instead of Kimball?"

> "Many sources, frequent source changes, and strict auditability. New sources mostly add satellites instead of forcing a remodel. It's insert-only, so history is preserved, and lineage is built into every row. The cost is complexity and join-heavy queries, so you still build star-schema marts on top for reporting. For a client needing quick BI on a few stable sources, I'd recommend Kimball directly."

"What does 'source-system flagging' mean exactly?"

> "Each record carries the source system that supplied it, the record source. If the same customer comes from two systems, you can see which system said what. For impact analysis you can query 'everything loaded from source X' and trace it to every downstream table and report."

"How do you handle a business key that exists in two sources with different formats?"

> "Standardise the key before hashing (trim, case, data type), otherwise the same customer gets two hub rows. If the systems use different keys entirely, you need a same-as link or a mapping to connect them."

"What's the downside you've experienced?"

> "Queries across many satellites are join-heavy and slow for analysts, and the model is hard for newcomers to understand. PIT tables and marts solve the first. Documentation and training solve the second."

[YOU FILL]: what "led enterprise transition" means: did you introduce Data Vault, extend it, or implement specific parts? (Hash function: SHA-256, confirmed.)

Deep dive 3: Evidence-based cluster right-sizing

Resume line: "Right-sized 60+ Databricks clusters via execution-log analysis, cutting cloud compute costs by $80K annually while eliminating processing bottlenecks."

> The Copilot search verified the **method and two detailed cases**, not the $80K figure or the 60+ cluster count. Lead with the cases below. Quote $80K only if you can show how it was calculated.

30-second summary:

> “I right-size clusters from execution evidence, not from CPU alone. On one production workload I reviewed 170 runs: 99.4% success, about 25 minutes on average, 32 minutes at P95, with peak CPU under 19% and peak memory about 133 GiB on 32-core, 256 GB nodes. It was clearly over-provisioned on CPU, so I recommended a smaller profile that kept memory headroom, accepting 15 to 25 percent longer runtime for a projected 50 to 65 percent compute reduction. On a much heavier satellite workload, the same analysis told me the opposite: don’t downsize aggressively.”

90-second STAR:

> “**Situation:** Cluster sizes on our platform had mostly been set once and never revisited, and compute is a large share of cost.
>
> **Task:** Find genuinely over-provisioned workloads and resize them without putting the regulatory deadline at risk.
>
> **Action:** I pulled the execution evidence for each candidate: number of runs, success rate, average and P95 runtime, peak CPU and peak memory, plus what the job actually does. One workload stood out. Across 170 production runs it succeeded 99.4% of the time, averaged 24.6 minutes with a P95 of 32 minutes, and peaked at under 19% CPU but about 133 GiB of memory, on Standard_E32ds_v4 nodes with 32 cores and 256 GB each. It processes 12 source branches, current and history data, reference lookups and several sequential MERGEs. So CPU was heavily over-provisioned, but memory was genuinely used. I recommended a smaller profile that kept enough memory headroom, and was explicit about the trade-off: runtime expected to rise 15 to 25 percent, which still fits its SLA window, for a projected 50 to 65 percent compute reduction.
>
> **Result:** A clear, evidence-backed recommendation with the trade-off stated upfront. [Say what was implemented and the realised result, if known.] The lesson I apply every time: CPU alone should never drive a downsizing decision.”

The counter-case (this is what makes the story senior):

> “The same evidence stopped me on another workload: a very large retail-loan satellite. Across 51 runs it had a 94.1% success rate, a P95 of 112 minutes and peaked at 131 GiB of memory, and the job itself had a 5-way UNION ALL, 65 window functions and hundreds of joins. That’s heavy shuffle and memory pressure, and the success rate was already lower. Low CPU there did not mean spare capacity. So I rejected aggressive downsizing, kept the E32 node family, and only reduced the worker count conservatively. Saving money by making a critical regulatory job unstable would have been a bad trade.”

Follow-ups:

- *“Why not just look at CPU?”* > “Because Spark jobs can be memory-bound or shuffle-bound with low CPU. The satellite job is the proof: under-used CPU, but heavy memory and shuffle pressure. Downsizing it on CPU would have made it fail more.”

- *"How did you pick the new size?"* > "Keep peak memory comfortably below the new capacity with headroom for data growth, and check that the expected runtime still fits the job's window in the dependency chain."
- *"What does 'projected' mean? Did it save money?"* > "[Answer truthfully: implemented or not, and the realised result.] The 50 to 65 percent is the projection from node pricing and the recommended profile."
- *"Your resume says $80K a year. Where does that come from?"* > "[Only if you can show it: the calculation and source.] If you can't: 'That's an annualised estimate across the right-sizing recommendations. The case I can walk you through with full evidence is this one.'"
- *"How would you do this for a client?"* > "Same method: rank workloads by cost, pull runs, success rate, P95, CPU and memory for each, understand what the job does, then resize with the trade-off stated, and validate over several runs before moving on."

Business impact: lower compute cost without destabilising regulatory jobs; a repeatable, evidence-based method. **Technical depth:** CPU versus memory versus shuffle pressure, P95 versus average, Standard_E32ds_v4 (memory-optimised: 32 vCPU, 256 GB), worker count versus node family. **Don't overclaim:** "projected" is not "saved." Never state $80K as a realised saving unless you can show the calculation.

Deep dive 4: Spark and Delta performance

Resume line: "Optimized Spark/Delta performance using OPTIMIZE, Z-Ordering, VACUUM, broadcast joins, skew handling, AQE, and Liquid Clustering, cutting runtimes 60% and shuffle volume from 500 GB to 200 GB."

2-minute overview

> "Our nightly chain was under pressure against 6 AM, so I worked on where time actually went, using the Spark UI and run logs.
>
> The biggest wins were: removing TRIM from join keys once the source team guaranteed clean, lowercase values, which let Spark join on raw columns; tuning broadcast joins so small dimension tables stopped being shuffled; handling skewed keys; and moving tables from fixed date partitioning to Liquid Clustering, with OPTIMIZE and VACUUM for file health.
>
> Shuffle dropped from about 500 GB to 200 GB, measured in the Spark UI, and average runtime fell about 60%, measured from run logs."

The one-liners you must be able to say

- **OPTIMIZE:** compacts many small files into fewer large ones, so reads open fewer files.
- **Z-Ordering:** co-locates related values across files, so file-level min and max statistics skip more files. Must be re-run as data changes.

- **Liquid Clustering:** replaces partitioning and Z-order. Clustering keys can change without rewriting the table, it clusters incrementally, and it handles skewed or high-cardinality data better.
- **VACUUM:** deletes files no longer referenced by the Delta log after the retention period (default 7 days). The trade-off: time travel can't go back further than the retention.
- **Broadcast join:** copies the small table to every executor so the big table isn't shuffled.
- **AQE (Adaptive Query Execution):** re-optimises at runtime using real statistics. It coalesces small shuffle partitions, switches sort-merge joins to broadcast when a side turns out small, and splits skewed partitions.
- **Skew handling:** AQE skew join, broadcast, or salting hot keys.

Question bank

"Walk me through how you found the problem."

> "Spark UI first: the slowest stages, shuffle read and write sizes, spill, and task time distribution. Large shuffles pointed at joins, and a few very long tasks pointed at skew. Then the query plans, which showed functions on join keys."

"Why does TRIM on a join key hurt?"

> "It runs on every row before the join. It also hides the raw column from optimisations like data skipping and dynamic pruning, which rely on comparing stored statistics to the raw values."

"Tell me about the broadcast change." (the failure story)

> "I raised the broadcast threshold from the 10 MB default to about 80 MB on a large cluster, 10 nodes at 128 GB each. A table larger than expected got broadcast and caused an out-of-memory failure in production. I owned it, reverted it, and now I use explicit broadcast hints for known-small tables instead of raising the global threshold, and I test against peak volumes."

> **Pick one version of the threshold numbers.** Your earlier materials say both "20 MB to 70–80 MB" and "10 MB to about 80 MB." Spark's default is 10 MB. Use whichever is true, and use it everywhere.

"Was the 60% across all 150 pipelines?"

> **[YOU FILL]** For example: "It was the average across the [N] pipelines I optimised, comparing [period] before and after from run logs." Don't let a partner assume it was all 150 if it wasn't.

“How did you know the improvement came from your change and not lower data volume?”

> “Compare runtime normalised by input volume, and compare the same pipelines over the same days of the week before and after. **[YOU FILL: what you actually did]**“

“Tell me about the Liquid Clustering migration.”

> “Other teams owned 50 to 70 pipelines each and had to remove their partition-by clauses. I had no authority over them. I explained the benefit, set up a shared tracker, and gave a realistic deadline instead of demanding it immediately. It finished ahead of schedule with no downtime.” Use one timeline consistently (you’ve said “a two-sprint window” and “a six-week deadline, done by week five”).

“Did Liquid Clustering cause any issues?”

> “Concurrent writes. We saw ConcurrentAppendException, which is Delta’s optimistic concurrency detecting two writers touching the same files. We handled it by [YOU FILL: retries, scheduling, or row-level concurrency].”

Deep dive 5: The PySpark data quality framework

Resume line: “Built a PySpark data quality framework with anomaly detection and freshness alerting, integrated into TD’s enterprise DQ suite, reducing data staleness by 25%.”

2-minute overview

> “In regulatory reporting a wrong number is worse than a late one, so quality checks need to block bad data, not just log warnings. I built a declarative, config-driven DQ framework in PySpark. Rules live in a rules table rather than in code: primary key uniqueness, null thresholds, referential integrity, and value ranges. Failing rows are routed to quarantine with the reason, so good data keeps flowing and bad data can be fixed and replayed.
>
> On top of that there’s freshness alerting, where each dataset has an expected-by time, and volume anomaly detection, which alerts when today’s row count moves more than a threshold percentage from normal. Results feed TD’s enterprise DQ suite. Staleness dropped about 25% because late data got caught and escalated earlier.”

Question bank

“What does ‘anomaly detection’ mean in your framework?”

> "Volume-based anomaly detection: compare today's row count with the recent norm, and alert if the change exceeds a threshold percentage. It's rule-based, not machine learning."

Say this plainly. If you let "anomaly detection" sound like ML and a partner asks which model, you're in trouble.

"Block or warn: how do you decide?"

> "By severity. A duplicate primary key in a hub or a failed referential check blocks, because it would corrupt downstream numbers. A small volume dip warns, because it may be genuine. Severity is configured per rule."

"What's in the quarantine, and who fixes it?"

> "The failing rows plus the rule that failed and the run ID. The owning team is alerted. Once fixed at source, the rows are reprocessed."

"How did you measure staleness?"

> **[YOU FILL]** For example, the time between the data being available at source and landing in the platform, averaged over a period, before and after.

"Why build your own instead of using Great Expectations?"

> "It had to plug into our config-driven engine and TD's enterprise DQ suite, and rules had to live in our control tables like everything else. Great Expectations is a good choice when you're starting fresh. **[YOU FILL: the real reason, and whether GE was used anywhere]**"

"What's the SQL behind a volume anomaly check?" See Part 20, exercise 5.

Deep dive 6: The metadata-driven ingestion framework

Resume line: "Designed a SQL Server metadata-driven ingestion framework cutting new-source onboarding effort ~75%."

2-minute overview

> "Before the framework, onboarding a source meant writing new pipeline code: connection, extraction, landing, loading and checks. With the framework, a source is described in metadata. The SQL Server control tables register the pipeline, its dependencies and its schedule, and the JSON config describes its transformations. A generic engine reads the metadata and runs it. Onboarding effort dropped by about 75%, because most of the work became configuration and testing rather than coding."

Question bank

"Show me what the control table looks like."

```
pipeline_registry (pipeline_id PK, source_system, target_table,
                   load_type,          -- FULL or INCREMENTAL
                   watermark_column, config_path, is_active)
pipeline_dependency (pipeline_id, depends_on_pipeline_id)
pipeline_run_log (run_id PK, pipeline_id, run_date, status,
                  start_ts, end_ts, rows_read, rows_written, config_version)
```

[YOU FILL: adjust to match your real tables.] Don't claim columns you don't have.

"How does the engine decide what can run?" Query the dependency table for pipelines whose upstream pipelines all succeeded today. The SQL is Part 20, exercise 3. This is a very likely live SQL question given your resume.

"What's the weakness of metadata-driven design?"

> "Debugging is harder because behaviour lives in configuration, and a bad config change can break many pipelines at once. So config needs the same discipline as code: version control, review, CI/CD promotion, and stamping each run with the config version it used."

"Where did 75% come from?"

> **[YOU FILL]** For example: "A new source used to take about [N] engineer-days, and now it takes about [M]."

Deep dive 7: ADF to Databricks Workflows migration, and Synapse to ADLS

Resume line: "…migrated 150+ ADF pipelines to Databricks Workflows with zero SLA breaches, accelerating Synapse-to-ADLS reads 30-40%."

2-minute overview

> “Orchestration was split: ADF triggered work that ran on Databricks. Moving to Databricks Workflows puts orchestration next to the compute, gives better visibility into task runs, and removes a layer.
>
> A senior colleague wanted to move everything at once. I argued for migrating one source system at a time, because with a regulatory deadline a big-bang cutover means everything breaks together. Each slice ran in parallel with the old path until outputs matched, then we cut over. The phased plan was adopted, and we didn’t breach the SLA during the migration.
>
> Separately, Synapse Dedicated SQL Pool had concurrency limits, so reads were queuing for 30+ minutes. Materialising the data to Delta on ADLS let Spark read files directly, and that cut read times by 30 to 40%.”

Question bank

“How did you prove a migrated pipeline was equivalent?”

> “Parallel runs, then reconciliation: row counts, control totals on key financial columns, and row-level comparison on keys for anything that differed. Cut over only when they matched for [N] consecutive runs. **[YOU FILL: N and the actual method]**“

“Is the migration finished?” [YOU FILL: honestly.] “In progress, with X of 150 done” is a perfectly good answer.

“Why was Synapse slow if the queries were fine?”

> “Dedicated SQL Pool limits concurrent queries by its service level and resource classes. With many pipelines reading at once, queries waited in a queue. The queries themselves weren’t slow; the waiting was.”

“Was the ADLS materialisation used in production?” [YOU FILL: dev only, or production too.] This has been an open question for months. Settle it.

“What does Workflows give you that ADF doesn’t?”

> “Native task dependencies across notebooks, Python and SQL tasks, repair runs that rerun only failed tasks, job clusters per task, and monitoring in the same place as the compute. ADF remains better for low-code copying from many external connectors.”

Deep dive 8: Legacy modernization and AI-assisted engineering

Resume lines: “Spearheaded an enterprise legacy modernization initiative… 8,000+ legacy tables and undocumented codebases from Hadoop HDFS, SAS, and SQL Server stored procedures into… PySpark pipelines and consumer data marts.” And: “Pioneered AI-assisted workflows by training custom GitHub Copilot agents on legacy repositories…”

This is the area where I have the least detail from you. **Prepare it carefully, because a partner will find it interesting.** Consultancies do exactly this work for clients.

Structure your answer (fill every bracket)

> “The problem: [N] legacy tables and jobs across Hadoop, SAS and SQL Server stored procedures, with almost no documentation. Nobody fully knew what the business logic was.
>
> My role: [YOU FILL: e.g., technical lead for the analysis and migration approach, with a team of N].
>
> The approach: first, understand the logic. I built Copilot agents grounded on the legacy repositories. They read the code and drafted business rules, source-to-target mappings and documentation, and engineers verified every output. Second, re-implement in PySpark by pattern, not table by table: [YOU FILL: e.g., templates for common SAS or stored-procedure patterns]. Third, prove equivalence by reconciling the old and new outputs.
>
> Result: [YOU FILL: tables migrated so far, time saved in analysis, current status].”

Question bank

“How do you know the AI’s interpretation of the logic was right?”

> “We didn’t trust it blindly. Every mapping was reviewed by an engineer, and the real proof was reconciliation: the new pipeline’s output had to match the legacy output. The AI sped up understanding. Reconciliation proved correctness.”

“Did you train a model?”

> “No. The agents were grounded, meaning they were given the repositories as context with specific instructions. No weights were trained.”

“What was hardest about SAS or stored-procedure logic?”

> **[YOU FILL]** Common real answers: row-by-row procedural logic that needs rethinking as set-based Spark; SAS data-step behaviour (implicit retain, by-group processing) with no direct equivalent; hidden dependencies between procedures.

“8,000 tables: how many did you personally touch?” [YOU FILL, honestly]

“How would you run this for a Vivanti client?”

> “Inventory and classify first: which tables are used, by whom, and how complex. Retire unused ones instead of migrating them. Group the rest by pattern, build templates, use AI-assisted analysis for documentation, and prove each wave with reconciliation.”

Deep dive 9: Pod leadership

Resume line: “Led pod delivery via Jira sprint planning, code reviews, and CI/CD (digital.ai, GitHub); mentored 10+ onshore/offshore engineers; awarded TD’s Legendary Award for performance engineering.”

2-minute overview

> “As Pod Lead I’m the technical lead of the pod, without direct reports. I run sprint planning in Jira, review code, own releases through our CI/CD pipeline with GitHub and digital.ai, and I’m the person the pod comes to with technical decisions.
>
> The thing I’m proudest of is building capability, not dependence. I trained the pod to run deployments themselves by teaching why each step exists, not just the steps, and that was recognised with an award from Chantal Paquette. I’ve mentored more than ten engineers across onshore and offshore teams.”

Question bank

“Tell me about prioritisation as a lead.” The sprint resequencing story: 50 open defects, fix upstream hub and link defects first, about 20 fixed and 12 to 13 downstream defects closed automatically. It then became standard process.

“How do you run a code review?”

> “Correctness first, then readability and consistency with our framework, then performance. I explain why, not only what, so reviews teach. For recurring issues I add them to our standards so they’re caught automatically.”

“How do you work with offshore colleagues across time zones?”

> “Clear written handovers, decisions documented in Jira rather than in calls, and overlapping hours for anything needing discussion. That’s directly relevant to Vivanti’s Bengaluru team working with Sydney.”

“Tell me about someone you mentored.” [YOU FILL: one real person, what they struggled with, what you did, and how they grew.]

“What does digital.ai do in your pipeline?” [YOU FILL: e.g., release orchestration and deployment approvals across environments.]

Deep dive 10: Real-Time Transaction Reconciliation (portfolio project)

2-minute overview

> “I built this to go deep on streaming correctness. A synthetic generator produces card transactions from a pool of 50,000 merchants into Kafka. A Spark Structured Streaming job reads them, applies a 5-minute watermark and 5-minute tumbling windows, and writes aggregates to Apache Iceberg tables.
>
> Separately, an Airflow-orchestrated batch job recomputes the same metrics independently from the raw bronze data. A reconciliation step full-outer-joins the streaming and batch results per window and flags each as MATCH or MISMATCH. That surfaces exactly the problems streaming systems hide: late events dropped by the watermark, window-boundary timing, and drift. It all runs in Docker Compose, and the code and architecture docs are on GitHub.”

Question bank

“What is a watermark and what does 5 minutes mean?”

> “It tells Spark how long to wait for late events. With a 5-minute watermark, Spark keeps a window open until the maximum event time seen is 5 minutes past the window’s end. Events arriving later than that are dropped from the streaming result. That’s exactly why I reconcile against batch: the batch sees everything, so mismatches reveal what the watermark dropped.”

“Event time versus processing time?”

> “Event time is when the transaction happened. Processing time is when Spark saw it. Aggregating by processing time gives wrong windows when events arrive late or out of order, so for financial correctness you use event time.”

“Why Iceberg and not Delta?”

> “To learn an engine-neutral table format with hidden partitioning and partition evolution, and because it integrates with AWS Glue and Athena. At TD I use Delta, so this broadened my range.”

“What throughput?”

> “The resume says 10,000+ transactions a minute. On a single node it sustained about 1,500 events a second, so the resume figure is conservative.”

“Why is this a ‘Lambda’ architecture?”

> “Two paths: a fast streaming path and a slower, complete batch path over the same raw data, compared against each other. Normally Lambda serves both to users. Here I use the batch path as the source of truth to check the streaming path.”

“Exactly-once?”

> “Structured Streaming with checkpointing and an idempotent or transactional sink gives effectively exactly-once results. Iceberg commits are atomic, so a replayed micro-batch doesn’t double-write if the sink is idempotent. **[YOU FILL: how your sink handles replays]**“

[YOU FILL]: Does it run on real AWS (S3, Glue, Athena), locally on MinIO with an Iceberg REST catalog, or both?

The reconciliation SQL is Part 20, exercise 8. Expect it: your resume invites it.

Part 20: SQL Straight From Your Resume

The round is SQL-heavy, and a smart partner will turn your resume into SQL questions: “You built a metadata-driven framework. Write the query that decides which pipelines can run.” These eight exercises are the most likely. Solve each out loud before reading the answer.

Exercise 1: Load a Data Vault hub

Prompt: “Write the SQL that loads new customers into a hub from staging.”

```
-- hub_customer (customer_hk, customer_id, load_dts, record_source)
-- stg_customer (customer_id, name, segment, record_source, load_dts)
INSERT INTO hub_customer (customer_hk, customer_id, load_dts, record_source)
SELECT MD5(customer_id), customer_id, load_dts, record_source
FROM (
  SELECT UPPER(TRIM(customer_id)) AS customer_id, load_dts, record_source,
         ROW_NUMBER() OVER (PARTITION BY UPPER(TRIM(customer_id))
                            ORDER BY load_dts, record_source) AS rn
  FROM stg_customer
  WHERE customer_id IS NOT NULL
) s
WHERE rn = 1
  AND NOT EXISTS (SELECT 1 FROM hub_customer h
                  WHERE h.customer_hk = MD5(s.customer_id));
```

Talk through:

- “Standardise the business key once, at load time, before hashing. Otherwise ’ abc’ and ‘ABC’ become two customers.” (Doing TRIM here, once per load, is right. Doing it inside every join each night is what you removed.)
- “ROW_NUMBER keeps one row per key if staging has several. The tiebreaker makes it deterministic.”
- “NOT EXISTS makes it insert-only and idempotent: rerunning the same batch inserts nothing.”
- “Null business keys are excluded. In practice they’d go to quarantine, not disappear.”

Exercise 2: Load a satellite with change detection

Prompt: “Insert a new satellite row only when a customer’s attributes changed.”

```
-- sat_customer (customer_hk, load_dts, hashdiff, name, segment, record_source)
INSERT INTO sat_customer (customer_hk, load_dts, hashdiff, name, segment, record_source)
SELECT s.customer_hk, s.load_dts, s.hashdiff, s.name, s.segment, s.record_source
FROM (
  SELECT MD5(UPPER(TRIM(customer_id))) AS customer_hk, load_dts, record_source,
         name, segment,
         MD5(COALESCE(TRIM(name), '') || '|' || COALESCE(TRIM(segment), '')) AS hashdiff
  FROM stg_customer
  WHERE customer_id IS NOT NULL
) s
LEFT JOIN (
  SELECT customer_hk, hashdiff,
         ROW_NUMBER() OVER (PARTITION BY customer_hk ORDER BY load_dts DESC) AS rn
  FROM sat_customer
) cur
  ON cur.customer_hk = s.customer_hk AND cur.rn = 1
WHERE cur.customer_hk IS NULL          -- brand new customer
   OR cur.hashdiff <> s.hashdiff;      -- attributes changed
```

Talk through:

- "The hashdiff is one hash over all descriptive columns, so I compare one value instead of every column."
- "COALESCE and a delimiter matter. Without the delimiter, 'AB' + 'C' and 'A' + 'BC' hash the same. Without COALESCE, one NULL column makes the whole hash NULL."
- "It compares with the **latest** satellite row only. If a customer changes and later changes back, we still insert, which is correct: history records both changes."
- "It assumes one staging row per customer per batch. If not, dedupe first as in exercise 1."

Follow-up: "How is this different from SCD Type 2?"

> "Same idea, history by inserting new versions. SCD2 usually also updates the old row's valid_to and is_current flag. The raw vault is insert-only, and end dates are derived later, in PIT tables or views."

Exercise 3: Which pipelines are ready to run?

Prompt: "Using your control tables, return pipelines that haven't run today and whose upstream dependencies all succeeded today."

```
SELECT p.pipeline_id
FROM pipeline_registry p
WHERE p.is_active = TRUE
  -- not already run or running today
  AND NOT EXISTS (
        SELECT 1 FROM pipeline_run_log r
        WHERE r.pipeline_id = p.pipeline_id
          AND r.run_date = CURRENT_DATE
          AND r.status IN ('SUCCESS', 'RUNNING'))
  -- no dependency that lacks a successful run today
  AND NOT EXISTS (
        SELECT 1 FROM pipeline_dependency d
        WHERE d.pipeline_id = p.pipeline_id
          AND NOT EXISTS (
                SELECT 1 FROM pipeline_run_log r
                WHERE r.pipeline_id = d.depends_on_pipeline_id
                  AND r.run_date = CURRENT_DATE
                  AND r.status = 'SUCCESS'));
```

Talk through: "This is relational division: 'all dependencies succeeded' becomes 'there is no dependency that didn't succeed'. The double NOT EXISTS is the cleanest way to say that. Pipelines with no dependencies qualify automatically, which is right for the first layer of the chain."

Alternative:

```
SELECT d.pipeline_id
FROM pipeline_dependency d
LEFT JOIN pipeline_run_log r
  ON r.pipeline_id = d.depends_on_pipeline_id
 AND r.run_date = CURRENT_DATE AND r.status = 'SUCCESS'
GROUP BY d.pipeline_id
HAVING COUNT(*) = COUNT(r.pipeline_id);
```

“Counts all dependencies versus successful ones. But it misses pipelines with no dependencies, so I’d UNION those in. That’s why I prefer the NOT EXISTS version.”

Follow-up: “What if a dependency ran twice today, once failing and once succeeding?” “Both versions check for **a** successful run, so a retry that succeeded counts. But in the COUNT version, if a dependency succeeded twice, the join duplicates rows and COUNT(r.pipeline_id) could exceed the dependency count. I’d count distinct dependencies instead.”

Exercise 4: Freshness alerting

Prompt: “Each dataset has an expected-by time. List datasets that are late today.”

```
-- dataset_sla (dataset, expected_by TIME)
-- load_log (dataset, status, end_ts)
WITH today_success AS (
  SELECT dataset, MIN(end_ts) AS first_success_ts
  FROM load_log
  WHERE status = 'SUCCESS'
    AND end_ts >= CURRENT_DATE AND end_ts < CURRENT_DATE + INTERVAL '1 day'
  GROUP BY dataset
)
SELECT s.dataset, s.expected_by, t.first_success_ts,
  CASE WHEN t.first_success_ts IS NULL THEN 'NOT_LOADED'
       ELSE 'LOADED_LATE' END AS freshness_status
FROM dataset_sla s
LEFT JOIN today_success t ON t.dataset = s.dataset
WHERE (t.first_success_ts IS NULL AND CURRENT_TIMESTAMP > CURRENT_DATE + s.expected_by)
   OR (t.first_success_ts > CURRENT_DATE + s.expected_by);
```

Talk through: “LEFT JOIN so datasets that never loaded appear, which are the most important ones. The range filter on end_ts keeps it prunable. I’d confirm the business day and time zone, since ‘today’ at 2 AM is ambiguous for a nightly chain.”

Exercise 5: Volume anomaly detection

Prompt: “Alert when today’s row count differs more than 20% from the previous 7 loads.”

```
-- daily_counts (dataset, load_date, row_count)
WITH w AS (
  SELECT dataset, load_date, row_count,
    AVG(row_count) OVER (PARTITION BY dataset ORDER BY load_date
                         ROWS BETWEEN 7 PRECEDING AND 1 PRECEDING) AS avg_prev7,
    COUNT(*)       OVER (PARTITION BY dataset ORDER BY load_date
                         ROWS BETWEEN 7 PRECEDING AND 1 PRECEDING) AS n_prev
  FROM daily_counts
)
SELECT dataset, load_date, row_count, ROUND(avg_prev7) AS avg_prev7,
       ROUND(100.0 * (row_count - avg_prev7) / NULLIF(avg_prev7, 0), 1) AS pct_diff
FROM w
WHERE load_date = CURRENT_DATE
  AND n_prev = 7
  AND ABS(row_count - avg_prev7) > 0.20 * avg_prev7;
```

Talk through:

- “The frame ends at 1 PRECEDING so today isn’t in its own baseline.”
- “`n_prev = 7` avoids alerting on new datasets with little history.”

- **Improvement a partner will like:** “Business data is weekly-seasonal. Mondays after weekends look different. Comparing with the same weekday over the last four weeks gives fewer false alarms.” That’s `PARTITION BY dataset, EXTRACT(DOW FROM load_date)` with `ROWS BETWEEN 4 PRECEDING AND 1 PRECEDING`.

Exercise 6: Find right-sizing candidates

Prompt: “From job run logs, how would you find clusters worth downsizing?” (This mirrors your right-sizing work. Present it as how you’d find candidates, and adapt it to what you actually did.)

```
-- job_runs (run_id, pipeline_id, cluster_id, start_ts, end_ts,
--           dbu_used, dbu_rate, avg_cpu_pct, needed_by_ts)
SELECT pipeline_id, cluster_id,
  COUNT(*)                                              AS runs,
  ROUND(SUM(dbu_used * dbu_rate), 2)                    AS cost_30d,
  ROUND(AVG(avg_cpu_pct), 1)                            AS avg_cpu_pct,
  ROUND(AVG(EXTRACT(EPOCH FROM (needed_by_ts - end_ts)) / 60), 0) AS avg_slack_min
FROM job_runs
WHERE start_ts >= CURRENT_DATE - INTERVAL '30 days'
GROUP BY pipeline_id, cluster_id
HAVING AVG(avg_cpu_pct) < 40
   AND MIN(EXTRACT(EPOCH FROM (needed_by_ts - end_ts)) / 60) > 60
ORDER BY cost_30d DESC;
```

Talk through: “Two conditions together: the cluster is under-used, **and** the pipeline has slack before anything downstream needs it, so shrinking it can’t threaten 6 AM. Ranking by cost means we fix the biggest spenders first. MIN slack, not average, because one tight night matters.”

Exercise 7: A customer’s attributes as of a date

Prompt: “Regulators ask what segment each customer was in on 30 June. Query the vault.”

```
SELECT h.customer_id, s.segment, s.load_dts AS version_loaded_at
FROM hub_customer h
JOIN (
  SELECT sc.*,
         ROW_NUMBER() OVER (PARTITION BY customer_hk ORDER BY load_dts DESC) AS rn
  FROM sat_customer sc
  WHERE load_dts < TIMESTAMP '2026-07-01 00:00:00'
) s ON s.customer_hk = h.customer_hk AND s.rn = 1;
```

Talk through: “Filter to versions loaded before the date, then take the latest per customer. For frequent as-of reporting across many satellites, this is exactly what PIT tables precompute: one row per key per snapshot date, pointing at the right satellite version.”

Senior nuance: “Load date is when **we** learned about a change, not when it happened in the source. If the question is what was true in the business on 30 June, I’d need the source’s effective date. If it’s what we reported on 30 June, load date is right. I’d clarify which.”

Exercise 8: Streaming versus batch reconciliation (your portfolio project)

Prompt: “You have streaming aggregates and batch aggregates per 5-minute window and merchant. Flag matches and mismatches.”

```
-- stream_agg / batch_agg (window_start, merchant_id, txn_count, total_amount)
SELECT COALESCE(s.window_start, b.window_start) AS window_start,
       COALESCE(s.merchant_id,  b.merchant_id)  AS merchant_id,
       s.txn_count AS stream_count, b.txn_count AS batch_count,
       CASE WHEN s.merchant_id IS NULL THEN 'MISSING_IN_STREAM'
            WHEN b.merchant_id IS NULL THEN 'MISSING_IN_BATCH'
            WHEN s.txn_count = b.txn_count
             AND ABS(s.total_amount - b.total_amount) < 0.01 THEN 'MATCH'
            ELSE 'MISMATCH' END AS recon_status
FROM stream_agg s
FULL OUTER JOIN batch_agg b
  ON b.window_start = s.window_start AND b.merchant_id = s.merchant_id
WHERE COALESCE(s.window_start, b.window_start)
      < CURRENT_TIMESTAMP - INTERVAL '10 minutes';
```

Talk through:

- “FULL OUTER JOIN because a window can be missing on either side, and both are findings.”
- “Money comparisons use a tolerance, or better, exact decimals, never floating-point equality.”
- “I only reconcile windows old enough to be closed: window length plus watermark. Otherwise every open window looks like a mismatch.”
- “The interesting cases are stream counts **lower** than batch counts. Those are late events dropped by the watermark. The mismatch rate tells you whether the watermark is too tight.”

Part 21: Follow-Up Chains

Partners rarely ask one question. They start broad and keep asking "why?" until they find the edge of your knowledge. Practise these chains out loud, and answer each step in two or three sentences before moving on.

Chain 1: Your Spark optimisation

1. **"Tell me about your Spark optimisation."** *(Testing: can you summarise impact?)* Use the 2-minute overview in deep dive 4.
2. **"How did you find where the time went?"** *(Method)* "Spark UI: slowest stages, shuffle sizes, spill, task time distribution. Then the query plans."
3. **"What did the plan show?"** *(Can you read a plan?)* "Sort-merge joins with large exchanges, meaning shuffles, on both sides, and functions wrapping the join keys."
4. **"Why does a shuffle cost so much?"** *(Fundamentals)* "Data is written to disk, sent across the network and re-read, and every task waits for all its inputs. It's usually the most expensive part of a job."
5. **"How did you reduce it?"** *(Actions)* "Broadcast the small dimension tables, filter and project before joins, remove functions from join keys, and let AQE coalesce partitions."
6. **"What happens if a broadcast table grows?"** *(Risk awareness)* "It can exceed executor memory and fail. That happened to me." Tell the OOM story.
7. **"Why not just use a bigger cluster?"** *(Cost judgment)* "That pays more for the same waste. It doesn't fix shuffles or skew, and the cost scales with every run."
8. **"How do you know it'll stay fast as data grows?"** *(Sustainability)* "Track runtime per input volume over time, and alert on trends before they threaten the deadline."

Chain 2: SQL deduplication

1. **"How do you remove duplicates?"** "ROW_NUMBER over the business key, ordered by the latest timestamp, keeping rn = 1."
2. **"Why not DISTINCT?"** "DISTINCT only removes rows identical in every column. Real duplicates usually differ in a timestamp or load ID."
3. **"Two rows have the same updated_at. Which wins?"** "Add a tiebreaker (ingestion time or a sequence number) so the result is deterministic. Otherwise reruns can pick different rows."
4. **"Where do duplicates come from?"** "Retries in ingestion, at-least-once delivery, overlapping incremental windows, or the source sending corrections."
5. **"How do you stop them reaching the target?"** "Dedupe each batch, then MERGE on the business key, so reprocessing is idempotent."
6. **"On a billion rows, what does ROW_NUMBER cost?"** "A shuffle and a sort by key. Deduping only the incremental batch, not the whole table, keeps it cheap."

Chain 3: SCD Type 2

1. **“What is SCD Type 2?”** “Keep history by adding a new row per change, with valid_from, valid_to and an is_current flag.”
2. **“Walk me through the load.”** “Detect changes by comparing the incoming hashdiff with the current row. Close the old row by setting valid_to and is_current = false, and insert the new version.”
3. **“Can you do that in one MERGE?”** “Not directly, because one source row must both update the old version and insert a new one. The common trick is to stage the changed rows twice (once with a null merge key, so they insert) or to do an update followed by an insert in one transaction.”
4. **“A change arrives late, dated before the current version.”** “Then it must be inserted in the middle of history, which means splitting the existing interval. Most pipelines avoid this by using load time as the version time. If business effective dates matter, you need proper bitemporal handling.”
5. **“How do you check an SCD2 table is healthy?”** “Exactly one current row per key, no overlapping intervals, and no gaps. All checkable in SQL.”

Chain 4: Data Vault

1. **“Why Data Vault?”** Many sources, auditability, insert-only history, adding sources without remodelling.
2. **“What’s a hash key for?”** “A deterministic key from the business key, so loads run in parallel without lookups, and keys are consistent across systems.”
3. **“Hash collisions?”** “We use SHA-256 hash keys. Collisions are practically impossible at any realistic scale, at the cost of longer keys than MD5, which some teams use for speed. Deterministic generation means the same business key always produces the same hash key, so loads can run in parallel.”
4. **“How do analysts query it?”** “They don’t query the raw vault directly. PIT and bridge tables and information marts, usually star schemas, sit on top.”
5. **“What would make you not recommend it?”** “A small team, few stable sources, or a need for fast BI delivery. The model’s overhead isn’t worth it then.”

Chain 5: The data quality framework

1. **“How do you ensure data quality?”** Rules table, block or warn by severity, quarantine, freshness, volume anomalies.
2. **“Who defines the rules?”** “Engineers define technical rules like keys and nulls. Business owners define business rules like valid ranges. Both live in the same table, so they’re visible and versioned.”
3. **“A rule blocks the chain at 4 AM. What happens?”** “The failing rows go to quarantine and the owner is alerted. We decide with the business whether to hold the report or publish with a documented exception. For regulatory data, correctness usually wins.”
4. **“How do you avoid alert fatigue?”** “Tune thresholds against history, use severity levels, and treat every false alarm as a rule to fix.”

5. **“How do you know your checks cover enough?”** “Map checks to known failure modes and past incidents. Every incident should produce a new rule, so the same thing can’t pass twice.”

Chain 6: The ADF to Workflows migration

1. **“Why migrate?”** Orchestration next to compute, repair runs, visibility, one less layer.
2. **“Why phased?”** Limit blast radius under a regulatory deadline, and allow rollback per source.
3. **“How did you prove equivalence?”** Parallel runs and reconciliation: counts, control totals, then row-level differences.
4. **“What went wrong?” [YOU FILL: one real issue, such as a dependency ordering difference or a parameter mismatch.]**
5. **“How would you run this for a client?”** “Same playbook: inventory, pick a pilot slice, reconcile, cut over, and use the lessons to speed up the next waves.”

Chain 7: Streaming and watermarking

1. **“Batch or streaming, how do you choose?”** “By the latency the business actually needs. Most ‘real-time’ requests are satisfied by 15 minutes, and streaming costs more to build and run.”
2. **“What’s a watermark?”** How long to wait for late events before closing a window.
3. **“How do you choose its length?”** “From the observed lateness distribution. If 99.9% of events arrive within 4 minutes, 5 minutes is reasonable. It’s a trade-off between completeness and latency and state size.”
4. **“What happens to events later than the watermark?”** “They’re dropped from streaming aggregates. That’s why I reconcile against batch, and why regulatory numbers come from batch.”
5. **“How does restart work?”** “Checkpoints store offsets and state, so the query resumes where it left off. The sink must be idempotent or transactional for exactly-once results.”

Chain 8: Metadata-driven pipelines

1. **“What is a metadata-driven pipeline?”** One engine, and behaviour defined in config.
2. **“Show me the control table.”** Part 19, deep dive 6.
3. **“Write the query for what’s ready to run.”** Part 20, exercise 3.
4. **“A config change broke 20 pipelines. How do you prevent that?”** “Config goes through Git review and CI/CD like code, with validation tests on the config itself, and each run records the config version.”
5. **“When is metadata-driven the wrong choice?”** “When pipelines are few and very different. The framework’s overhead only pays off with many similar pipelines.”

Chain 9: A production incident

1. **“Tell me about an incident.”** The UNRESOLVED_COLUMN SLA miss (22.13).

2. **“How did you find the cause?”** Run logs, then the error. UNRESOLVED_COLUMN means a referenced column doesn’t resolve, so the candidates are a schema or source change, a deployment mismatch, a broken view, or a refactoring defect. “We needed development support, and the root cause was never definitively confirmed.”
3. **“Why didn’t you catch it earlier?”** “We weren’t validating the incoming schema before the load. That was the gap.”
4. **“What did you change?” [YOU FILL: the systemic fix.]**
5. **“Could it happen again?”** “That class of failure is now caught before the load. **[YOU FILL: how]**“

Chain 10: Regulatory reporting

1. **“What’s different about regulatory data?”** “Correctness, lineage and reproducibility matter more than speed. You must be able to reproduce what you reported months later.”
2. **“How do you reproduce a past report?”** “Insert-only vault history plus Delta time travel and versioned code and config. **[YOU FILL: how long time travel is retained, given VACUUM]**“
3. **“VACUUM deletes old files. Doesn’t that break reproducibility?”** “It limits time travel to the retention period, so long-term reproducibility comes from the insert-only vault history, not from time travel.”
4. **“A regulator finds an error in a report you sent. What happens?”** “Find the root cause through lineage, fix it, restate with the business, and add a control so it can’t recur.”

Part 22: Your Stories

Each story has a **30-second executive summary** (for "briefly" or an executive), a **90-second STAR answer**, likely follow-ups, the business impact, the technical depth to have ready, and what not to overclaim.

The story index (which story to use for which question) is on the revision sheets in Part 30.

22.1 The TD Legendary Award (performance engineering)

30-second summary:

> "I received TD's Legendary Award from Varun Khanna, under the value 'Execute with Speed and Impact', for performance work on our platform: removing a 30-plus-minute Synapse bottleneck and making our Spark pipelines significantly faster, without putting the 6 AM deadline at risk."

90-second STAR:

> "**Situation:** Our nightly chain was under growing pressure against the 6 AM regulatory deadline. Some steps were slow, and others were waiting in a queue in Azure Synapse for more than 30 minutes.
>
> **Task:** I took on finding where the time actually went and fixing it without risking the deadline.
>
> **Action:** I split the problem in two. For Synapse, the queries themselves were fine. They were waiting because Dedicated SQL Pool limits how many queries run at once. I moved the data our pipelines needed to Delta on ADLS, so Spark read files directly instead of queuing. For Spark, I used the Spark UI and run logs to find the heavy stages. The biggest issue was join keys wrapped in TRIM. I got a guarantee from the source team that the values were already clean, removed the TRIM, and tuned broadcast joins and skew handling.
>
> **Result:** The queue waits disappeared, shuffle dropped from about 500 GB to about 200 GB, and average runtime fell about 60%. That work was recognised with the Legendary Award."

Follow-ups:

- *"What exactly did the award citation say?"* > "It was under 'Execute with Speed and Impact.' [Quote one line from the citation.]"
- *"Was the Synapse change in production?"* > "[Answer precisely: production, or development first.]" This is still open from earlier sessions. Settle it.
- *"Who else was involved?"* > "I drove the analysis and the changes. The source team gave the data-quality guarantee that made the TRIM removal safe, and my manager supported rolling it out."

Business impact: faster, more predictable delivery to regulators, and less risk to the deadline. **Technical depth:** Synapse Dedicated SQL Pool concurrency limits; functions on join keys blocking optimisation; broadcast versus shuffle joins. **Don't overclaim:** the award is from one person with a specific tag. Don't call it "the bank's top award" unless you know that. And 500→200 GB and 60% are **candidate metrics**: use them only if you can give the baseline and how they were measured.

22.2 Evidence-based cluster right-sizing

30-second summary:

> "I right-size clusters from execution evidence, not from CPU alone. On one production workload I reviewed 170 runs: 99.4% success, about 25 minutes on average, 32 minutes at P95, with peak CPU under 19% and peak memory about 133 GiB on 32-core, 256 GB nodes. It was clearly over-provisioned on CPU, so I recommended a smaller profile that kept memory headroom, accepting 15 to 25 percent longer runtime for a projected 50 to 65 percent compute reduction. On a much heavier satellite workload, the same analysis told me the opposite: don't downsize aggressively."

90-second STAR:

> "**Situation:** Cluster sizes on our platform had mostly been set once and never revisited, and compute is a large share of cost.
>
> **Task:** Find genuinely over-provisioned workloads and resize them without putting the regulatory deadline at risk.
>
> **Action:** I pulled the execution evidence for each candidate: number of runs, success rate, average and P95 runtime, peak CPU and peak memory, plus what the job actually does. One workload stood out. Across 170 production runs it succeeded 99.4% of the time, averaged 24.6 minutes with a P95 of 32 minutes, and peaked at under 19% CPU but about 133 GiB of memory, on Standard_E32ds_v4 nodes with 32 cores and 256 GB each. It processes 12 source branches, current and history data, reference lookups and several sequential MERGEs. So CPU was heavily over-provisioned, but memory was genuinely used. I recommended a smaller profile that kept enough memory headroom, and was explicit about the trade-off: runtime expected to rise 15 to 25 percent, which still fits its SLA window, for a projected 50 to 65 percent compute reduction.
>
> **Result:** A clear, evidence-backed recommendation with the trade-off stated upfront. [Say what was implemented and the realised result, if known.] The lesson I apply every time: CPU alone should never drive a downsizing decision."

The counter-case (this is what makes the story senior):

> “The same evidence stopped me on another workload: a very large retail-loan satellite. Across 51 runs it had a 94.1% success rate, a P95 of 112 minutes and peaked at 131 GiB of memory, and the job itself had a 5-way UNION ALL, 65 window functions and hundreds of joins. That’s heavy shuffle and memory pressure, and the success rate was already lower. Low CPU there did not mean spare capacity. So I rejected aggressive downsizing, kept the E32 node family, and only reduced the worker count conservatively. Saving money by making a critical regulatory job unstable would have been a bad trade.”

Follow-ups:

- *“Why not just look at CPU?”* > “Because Spark jobs can be memory-bound or shuffle-bound with low CPU. The satellite job is the proof: under-used CPU, but heavy memory and shuffle pressure. Downsizing it on CPU would have made it fail more.”
- *“How did you pick the new size?”* > “Keep peak memory comfortably below the new capacity with headroom for data growth, and check that the expected runtime still fits the job’s window in the dependency chain.”
- *“What does ‘projected’ mean? Did it save money?”* > “[Answer truthfully: implemented or not, and the realised result.] The 50 to 65 percent is the projection from node pricing and the recommended profile.”
- *“Your resume says $80K a year. Where does that come from?”* > “[Only if you can show it: the calculation and source.] If you can’t: ‘That’s an annualised estimate across the right-sizing recommendations. The case I can walk you through with full evidence is this one.’”
- *“How would you do this for a client?”* > “Same method: rank workloads by cost, pull runs, success rate, P95, CPU and memory for each, understand what the job does, then resize with the trade-off stated, and validate over several runs before moving on.”

Business impact: lower compute cost without destabilising regulatory jobs; a repeatable, evidence-based method. **Technical depth:** CPU versus memory versus shuffle pressure, P95 versus average, Standard_E32ds_v4 (memory-optimised: 32 vCPU, 256 GB), worker count versus node family. **Don’t overclaim:** “projected” is not “saved.” Never state $80K as a realised saving unless you can show the calculation.

22.3 Spark and Delta optimisation

30-second summary:

> “I made our join-heavy Spark pipelines much more efficient by fixing causes rather than adding hardware: clean join keys, broadcast joins for small tables, skew handling and better data layout. Shuffle dropped from about 500 GB to 200 GB, measured in the Spark UI, and average runtime dropped about 60%, measured from run logs.”

90-second STAR:

> "**Situation:** Join-heavy pipelines on a large cluster (10 nodes, about 1.2 TB of memory) were taking longer than they should, and the 6 AM deadline had less margin every month.
>
> **Task:** Find the real bottlenecks and fix them.
>
> **Action:** I started in the Spark UI: the slowest stages, shuffle sizes, spill, and task time distribution. The query plans showed join keys wrapped in TRIM, which forces Spark to compute on every row and blocks optimisations. I confirmed with the source team that the values were guaranteed clean and lowercase, then removed the TRIM. I also broadcast the small dimension tables so the big tables weren't shuffled, handled skewed keys, and moved tables to Liquid Clustering with OPTIMIZE for file health.
>
> **Result:** Shuffle went from about 500 GB to about 200 GB, and average runtime fell about 60%."

Follow-ups:

- *"Was the 60% across all 150 pipelines?"* > "[Say exactly which pipelines and period it covers.]"
- *"How did you know it was your change and not lower volume?"* > "We compared the same pipelines over comparable days before and after. [Add your specific method.]"
- *"Why did TRIM matter so much?"* > "It ran on every row of both sides of the join, and it hid the raw column from optimisations like data skipping."

Business impact: more margin on the regulatory deadline; lower compute. **Technical depth:** the Spark UI, broadcast versus sort-merge joins, AQE, skew, Liquid Clustering, OPTIMIZE. **Don't overclaim:** the Copilot search treats 500→200 GB and 60% as **candidate metrics**. Only state them if you can give the baseline, the after state and how you validated them (Spark UI, run logs). Otherwise say "significantly reduced shuffle and runtime" and describe the method.

22.4 Supporting 150+ production pipelines

30-second summary:

> "I own the reliability of about 150 production pipelines that run every night as a dependency chain and must finish by 6 AM for regulatory reporting. What makes that manageable is that the platform is config-driven: one tested engine, consistent logging and quality checks, and pipelines defined in configuration."

90-second answer:

> "The key is not treating them as 150 separate things. They're registered in SQL Server control tables with their dependencies and run state, and configured in JSON and YAML in Git. A shared engine runs them, so improvements and fixes apply everywhere at once. Every run is logged with a run ID, so when something fails I can trace it quickly. Data quality checks sit between layers and block bad data. And I know which pipelines are on the critical path to 6 AM, so when something goes wrong I can prioritise immediately. My role is keeping that system healthy, handling escalations, reviewing changes, and improving it over time."

Follow-ups:

- *"How many did you build yourself?"* > "[Be precise.] I built [N] and I own the platform's reliability and changes for all of them."
- *"What fails most often?"* > "Upstream issues: late or changed source data. That's why schema checks and freshness monitoring matter."

Don't overclaim: "I own the reliability of" is accurate. "I built 150 pipelines" probably isn't.

22.5 Maintaining a critical regulatory SLA

30-second summary:

> "Our deadline is 6 AM every business day for regulatory reports. We've consistently met it. The one miss I've been involved with taught me to catch source problems before the load rather than during it."

90-second answer:

> "Four things keep us on time. Explicit dependencies, so we always know the critical path. Compute sized for that critical path. Fixing upstream problems first, because a problem in a hub breaks everything beneath it. And early communication: if we might be late, the reporting owners hear it from us early, with options, not at 6 AM.
>
> We did have a miss, when a consumption job failed with an UNRESOLVED_COLUMN error. I'll be honest about it, including that the root cause was never definitively confirmed, because how we handled it and what we learned is the most useful part of the story."

Don't overclaim: don't say "99.9%" or "never missed." One owned miss is more credible.

22.6 Production releases and release coordination

30-second summary:

> “As Pod Lead I own releases for the pod: reviewing code, deciding what’s ready, and running deployments through our CI/CD with GitHub and digital.ai. The thing I’m proudest of is that I trained the pod to run deployments themselves, which was recognised with a Win Together award.”

90-second STAR:

> “**Situation:** Deployment knowledge sat with very few people, including me, which made releases a bottleneck and a risk.
>
> **Task:** Make the pod able to release safely without depending on one person.
>
> **Action:** Instead of just writing steps down, I walked people through why each step exists: what the pipeline checks, what could go wrong, how to roll back. Then they ran releases while I watched, and eventually on their own.
>
> **Result:** The pod can release without me, releases stopped being a bottleneck, and that was recognised with a Win Together award from Chantal Paquette.”

Follow-ups:

- *“What does digital.ai do in your process?”* > “[Your answer, e.g., release orchestration and approvals across environments.]”
- *“How do you decide a release is ready?”* > “Code reviewed, tests and data checks passed in lower environments, a rollback plan, and no conflict with the nightly window.”

22.7 A data quality or reconciliation challenge

30-second summary:

> “In regulatory reporting, a wrong number is worse than a late one. So I designed our data quality engine to block bad data, not just log warnings, with failing rows quarantined for correction. And whenever we migrate anything, we prove equivalence with reconciliation, not assumptions.”

90-second answer:

> “Our DQ engine is rules-driven. Rules live in a table, not in code: primary key uniqueness, null thresholds, referential integrity between hubs and links. Each rule has a severity. Critical failures block the data and route failing rows to quarantine with the reason and run ID. Minor ones warn. On top of that, freshness monitoring checks every dataset lands by its expected time, and volume checks alert when a load’s row count moves more than a threshold from normal.
>
> For migrations, like moving orchestration to Databricks Workflows, we ran old and new side by side and reconciled row counts, control totals and row-level differences before cutting over.”

Follow-ups:

- *"Is the anomaly detection machine learning?"* > "No, it's rule-based: a percentage threshold on volume changes. I'd rather be clear about that."
- *"Who fixes quarantined rows?"* > "The owning team is alerted with the failing rule and run ID, fixes at source, and the rows are reprocessed."

22.8 RRMDM migration or decommissioning analysis

> **No details about RRMDM exist anywhere in our project history.** Don't use this story until you've filled in the structure below with real facts. An unprepared story is worse than no story.

Structure to fill, then practise:

> "**Situation:** RRMDM was [what the system was and who used it]. The bank wanted to [migrate or decommission it] because [reason].
>
> **Task:** My part was [your specific responsibility: e.g., analysing which consumers and pipelines depended on it].
>
> **Action:** I [how you found dependencies: lineage, access logs, code search], [how you classified them: migrate, replace, retire], and [how you proved nothing would break].
>
> **Result:** [What was decided or delivered, and any measurable outcome.]"

The consulting point to make regardless: "Decommissioning is mostly a dependency problem. The risk is the consumer nobody remembered. So I'd start from actual usage data, not from asking people."

22.9 SAS decomposition and lineage discovery

This is supported by your resume's legacy modernisation bullet, but you haven't given me the details. The version below uses only what's on the resume. Fill in the brackets.

30-second summary:

> "I worked on modernising legacy workloads (Hadoop, SAS and SQL Server stored procedures) into PySpark, where there was almost no documentation. To understand the business logic, I used GitHub Copilot agents grounded on the legacy code to draft business rules and source-to-target mappings, and engineers verified every output."

90-second STAR:

> "**Situation:** We had a large estate of legacy logic in SAS, Hadoop and stored procedures, with very little documentation, so nobody could say with confidence what it did.
>
> **Task:** My role was [your specific role].
>
> **Action:** The first problem was understanding. I set up Copilot agents grounded on the legacy repositories. They weren't trained; they were given the code as context with specific instructions to extract business rules, lineage and source-to-target relationships. Engineers reviewed every output. Then we re-implemented in PySpark and proved each migrated piece by reconciling its output against the legacy output.
>
> **Result:** [What was delivered: scope completed, time saved on analysis, current status.]"

Follow-ups:

- *"How do you know the AI got the logic right?"* > "We didn't rely on it for correctness. It sped up understanding. Reconciliation against the legacy output proved correctness."
- *"What's hard about SAS logic?"* > "[Your real example.] Commonly: procedural, row-by-row data steps that need rethinking as set-based Spark, and implicit behaviours with no direct equivalent."
- *"How many tables did you personally handle?"* > "[Honest number.]"

Don't overclaim: "spearheaded 8,000 tables" unless you can explain your role at that scale. "Trained" agents: always say "grounded."

22.10 Source-to-target mapping automation

Two real threads support this: the metadata control plane stores source-to-target mappings, and the grounded Copilot agents extracted mappings from legacy code.

30-second summary:

> "Source-to-target mappings used to be spreadsheets that drifted from the real code. We moved them into the metadata control plane, so the mapping that documents a pipeline is the same metadata that drives it. For legacy code with no documentation, I used grounded Copilot agents to draft mappings, which engineers then verified."

Key message: "Documentation that's generated from, or drives, the running system can't drift from it."

Fill before using: which part you built, and one example.

22.11 Rahona onboarding

> **No details about Rahona exist in our project history.** Fill the structure before using it. If it's an onboarding of a source or a team onto your platform, the metadata-driven framework is probably your strongest angle.

> "**Situation:** [What Rahona is, and why it needed onboarding.] **Task:** [Your role.] **Action:** [Steps: requirements, configuration, data quality rules, testing, reconciliation.] **Result:** [Time taken versus a normal onboarding, and what went live.]"

22.12 Data Vault and enterprise lakehouse work

30-second summary:

> "Our platform uses Data Vault 2.0 on Databricks because regulators need full history and lineage from fifteen-plus changing sources. Every record carries its source and load time, so we can trace any reported number back to where it came from and do impact analysis when a source changes."

90-second answer:

> "Hubs hold unique business keys, links hold the relationships between them, and satellites hold the descriptive attributes and their history. It's insert-only, so nothing is ever overwritten. That suits audit. Hash keys let us load in parallel without lookups. When a source system changes, we can find every table and report that depends on it, because every record is tagged with its source.
>
> The trade-off is that raw Data Vault is join-heavy and hard for analysts to query, so reporting marts sit on top. For a client with a few stable sources who mainly needs dashboards, I'd recommend a Kimball star schema directly. The overhead of Data Vault only pays off with many changing sources and strict audit needs."

Follow-up: "Did you design it?" > "[Precise answer.] The Data Vault foundation [was in place / was introduced when]. My part was [source-system flagging and audit lineage / extending it to X sources / etc.]."

22.13 Production incident: the UNRESOLVED_COLUMN SLA miss

30-second summary:

> “A consumption job failed with an UNRESOLVED_COLUMN error, which meant the query referenced a column that no longer resolved, and we missed the SLA that day. We needed development support to fix it, and we never definitively confirmed the root cause. What I took from it is how to run that kind of incident: contain the blast radius, communicate early, and structure the RCA so it leads to prevention even when the cause is uncertain.”

90-second STAR:

> “**Situation:** A consumption-layer job failed with UNRESOLVED_COLUMN, which blocked the output it produced, and we missed the SLA.
>
> **Task:** Get the output delivered as soon as possible, keep stakeholders informed, and understand why it happened.
>
> **Action:** First, blast radius: which downstream outputs depended on that job, and which parts of the chain could still run. Second, early communication to the reporting owners, so they weren’t surprised. Third, diagnosis. UNRESOLVED_COLUMN means the query can’t find a column, so the candidates are a schema or source change, a deployment mismatch between the job and the tables or views it reads, a broken view, or a refactoring defect. We brought in development support to fix the job.
>
> **Result:** The output was delayed and we missed the SLA, and I don’t hide that. The root cause was never definitively confirmed, which is itself a lesson: the most useful prevention is making this class of failure visible earlier, [what was actually put in place, or what you recommended], so we wouldn’t depend on reconstructing the cause after the fact.”

If asked “so what was the cause?”:

> “Honestly, it wasn’t definitively confirmed. The evidence pointed to [the most likely theme, if you know it]. Rather than guess in the RCA, we recorded it as unconfirmed and focused on prevention that covers all the likely causes: validating schemas before a job runs, checking that deployed code and table definitions match across environments, and testing views after refactoring.”

Prevention checklist to talk through (say “what I’d put in place”, unless you know it was done):

- Schema checks before the consumption job runs, so a missing column fails fast with a clear message.
- A deployment check that code, views and table definitions match in the target environment.
- Regression tests on views after refactoring.
- Dependency lineage, so a change in an upstream table flags every consumer.

Don’t: say “an upstream schema change caused it” as a fact, or say “3 AM” unless verified.

22.14 A difficult technical problem

Use the Synapse bottleneck, because the difficulty was diagnosis, not effort.

> "The hardest problems are the ones where the obvious explanation is wrong. Some of our steps were taking 30-plus minutes longer than they should, and everyone assumed the queries were slow. When I looked closely, the queries themselves ran quickly. They were waiting in a queue, because Azure Synapse Dedicated SQL Pool limits how many queries run at once, and many of our pipelines hit it at the same time. Tuning the queries would have done nothing. The fix was architectural: materialise the data to Delta on ADLS so Spark could read files directly and skip the queue. The lesson I took: measure where the time actually goes before optimising anything."

22.15 A failure or mistake

> "I raised Spark's broadcast join threshold from the 10 MB default to about 80 MB on a large cluster, to avoid shuffles on some joins. It worked in testing, but in production a table bigger than I'd expected got broadcast, exceeded executor memory, and the job failed with an out-of-memory error.
>
> I owned it straight away, in writing, reverted the change, and got the pipeline rerun. Then I changed how I work: I test configuration changes against peak data volumes, not typical ones, and I use explicit broadcast hints for tables I know are small, instead of raising a global setting that affects every join."

Key message: you own mistakes openly and change the process, which matches Vivanti's Accountability value. **Follow-up: "What was the impact?"** > "[Honest: delay length, and whether the deadline was affected.]"

22.16 A conflict or disagreement

> "A senior colleague wanted to migrate all 150 pipelines from ADF to Databricks Workflows in one cutover, to get it done quickly. I disagreed, respectfully, because of the 6 AM regulatory deadline. With a big-bang cutover, if anything went wrong, everything went wrong at once.
>
> I started by agreeing with the goal: speed mattered. Then I proposed migrating one source system at a time: run old and new in parallel, reconcile, cut over, then move to the next. That limits the blast radius, lets us learn from the first slice, and lets us roll back one source without touching the rest. I laid out the risk and the plan in writing. The phased approach was adopted, and we kept regulatory delivery uninterrupted."

Follow-up: "What if they'd overruled you?" > "Then I'd have supported the decision fully and made the big-bang as safe as possible: a tested rollback, a dry run, and extra monitoring. Disagreeing before a decision is healthy. Undermining it afterwards isn't."

22.17 A situation involving ambiguity

Option A, the 50 defects (ambiguous priorities):

> "Before a release we had about 50 open defects and no clear order to fix them in. The default was to work through the list. I looked at where the defects sat in the data flow and noticed many downstream ones were symptoms of upstream problems in hubs and links. So I proposed fixing the upstream defects first. We fixed about 20, and 12 or 13 downstream defects closed on their own. The team then adopted upstream-first as the standard way to triage defects."

Option B, late scope in the FRP RESL 1.1 release: use only if you fill in the details (what the scope was, and how you made room without slipping the release).

22.18 Mentoring or onboarding engineers

> "I've onboarded and mentored engineers on the pod, including an offshore team. My approach is to teach the why, not just the steps. For deployments, I walked people through what each step protects against, then had them run releases with me watching, then on their own. For onboarding, I explain the platform as a system first (the nightly chain, the control tables, where data quality sits) before any specific pipeline, because once people see the whole picture, the details make sense. The result was a pod that can deploy without me, and that was recognised with an award from Chantal Paquette."

Fill: one specific person's growth story, if asked "tell me about one person." **Don't:** "I managed" or "I led a team of 10."

22.19 Influencing without authority

> "We moved from date-based partitioning to Liquid Clustering, and it only worked if other teams changed their pipelines too. Some owned 50 to 70 pipelines each, and I had no authority over any of them.
>
> I started by explaining what was in it for them: fewer small-file problems and faster reads, with no rewrite later if query patterns changed. I set up a shared tracker so progress was visible, and agreed a realistic six-week deadline instead of demanding it immediately. I made myself available to help with the tricky cases. We finished by week five, with no downtime."

Follow-up: "What problems came up?" > "Concurrent writes. We saw ConcurrentAppendException, which is Delta's optimistic concurrency detecting two writers touching the same files. [How you handled it.]"

22.20 Process or documentation improvement

> "At least four teams depend on a shared reference table that maps source values to standard values. Before each release, someone manually compared the old and new versions of that table, which was error-prone, and mistakes had broken other teams' pipelines.
>
> I built a small Databricks utility that uses Delta time travel to compare any two versions of the table and shows every added, deleted and modified mapping side by side, like a code diff. That turned a risky manual check into a reliable, repeatable one, and made release reviews much faster."

Why it's a great consulting story: small tool, big risk reduction, reused by other teams. And it shows you notice problems nobody asked you to fix. **Technical depth:** Delta time travel (querying a table as of a version); a full outer join on the composite key to classify ADDED, DELETED and MODIFIED rows.

22.21 Rejecting aggressive downsizing (evidence over assumptions)

30-second summary:

> "On a very large retail-loan satellite workload, the easy call was to downsize because CPU looked low. The evidence said otherwise: 51 runs, 94.1% success, a 112-minute P95, 131 GiB peak memory, and a job with a 5-way UNION ALL, 65 window functions and hundreds of joins. I kept the E32 node family and only reduced workers conservatively. Low CPU doesn't mean low resource need."

Use it for: "a decision you pushed back on," "a time you used data to make a decision," "a trade-off between cost and reliability." Full detail is in 22.2.

22.22 The Delta optimiser framework (never optimise blindly)

30-second summary:

> "Rather than running OPTIMIZE on everything, I helped onboard a Delta optimiser framework that decides based on evidence. For each candidate table we collect size, file count and average file size, estimate the benefit, and classify it as optimise, review or skip. The target file size is 128 MB, and the configuration supports retries, batch sizing, optimising individual partitions and file comparisons."

90-second STAR:

> "**Situation:** Small files and unoptimised Delta tables slow reads, but optimising everything wastes compute and can collide with running jobs.
>
> **Task:** Bring in an optimiser framework safely and use it selectively.
>
> **Action:** I did the candidate analysis: table size, file count, average file size and an estimated benefit, then classified tables into optimise, review or skip. I onboarded the framework, tested it in DEV, troubleshot configuration issues, wrote the runbook, created the stories, and planned the deployment. The configuration handles retries, batch sizes, per-partition optimisation and before-and-after file comparisons, with 128 MB as the target file size.
>
> **Result:** [What's live, and any measured read or runtime improvement.] The principle I'd bring to any client: measure first, and optimise only where the benefit justifies the compute."

Follow-ups:

- *"Why 128 MB?"* > "It's a common target that balances fewer file opens against parallelism and write cost. Too small means many files; too large means fewer tasks and longer rewrites."
- *"Why not optimise everything nightly?"* > "It costs compute, and tables with few small files gain almost nothing. OPTIMIZE also rewrites files, which can conflict with concurrent writers. Selectivity is the point."

22.23 Concurrent Delta writes (ConcurrentAppendException)

30-second summary:

> "Our platform has seen ConcurrentAppendException when several jobs wrote to overlapping tables: dependency overlap, catch-up runs overlapping normal runs, and MERGEs touching the same files. Delta uses optimistic concurrency, so when two writers commit changes to the same files, one fails. The fixes are mostly about sequencing and design, not just retries."

90-second answer:

> "Delta checks at commit time whether another transaction changed the files this one read. If it did, one writer fails with an exception like ConcurrentAppendException. On our platform, that happened when jobs overlapped on the same tables: a catch-up run colliding with the regular run, or two MERGEs whose conditions touched the same files.
>
> Mitigation in order: first, sequencing, making the dependency graph prevent two writers on one table at the same time and blocking catch-up runs from overlapping scheduled ones. Second, narrowing what each write touches, like including partition or clustering columns in MERGE conditions so writers work on disjoint files. Third, retries with backoff for the occasional genuine collision. Retries alone just hide a design problem."

Don’t overclaim: say “our platform experienced” unless you personally fixed a specific instance. If you did, describe exactly what you changed.

A diagnosis framework for any production failure

Your platform has seen many failure types: TABLE_OR_VIEW_NOT_FOUND, AlreadyExistsException, DELTA_PATH_DOES_NOT_EXIST, DELTA_REPLACE_WHERE_MISMATCH, cluster termination, partial or empty source files, bad source data, trigger dependency failures, ingestion delays, PGP decryption failures, catch-up failures, duplicate job execution and schedule configuration issues. You didn’t fix them all, and you shouldn’t claim to. But you can show a senior way of handling any of them:

1. **Classify:** is it data (bad, partial or empty files), code or deployment (missing tables, schema mismatch), platform (cluster termination), or orchestration (triggers, duplicates, schedules)?
2. **Blast radius:** which downstream outputs depend on it, and what can still run?
3. **Communicate:** tell consumers early, with an expected time and options.
4. **Recover safely:** rerun only if the job is idempotent. A duplicate run or a partial file can double-load data.
5. **RCA:** timeline, trigger, root cause (or “unconfirmed”, honestly), why detection didn’t catch it earlier.
6. **Prevent:** a check, test or guard for that failure class, not just for this one instance.

Examples to use:

- **Empty or partial source file:** “Check file completeness (row count or control file) before loading. Loading an empty file can wipe a day’s data if the load overwrites.”
- **Duplicate job execution:** “Make loads idempotent with MERGE on keys or partition overwrite, and add a lock or run-state check in the control table.”
- **DELTA_REPLACE_WHERE_MISMATCH:** “The data written doesn’t match the replaceWhere condition. Validate the batch’s date range before writing, so a batch can only overwrite the partitions it actually contains.”

Part 23: Leadership and Behavioural Questions

Most behavioural questions are the same eight or nine stories told from different angles. For each question below, the script gives you the **angle**: how to open, which story to use, and how to close. Full STAR versions are in Part 22.

> **Rule for questions with no real story:** some topics (like handling an underperformer) have no specific story in your history. For those, the script gives your honest approach and marks where a real example goes. Never invent one. “I haven’t faced that exact situation, but here’s how I’d handle it, and here’s something close” is a good answer.

Ownership

1. Tell me about a time you took ownership beyond your role.

> “The shared reference-table diff tool (22.20). Nobody asked me to fix the manual comparison, but it kept breaking other teams’ pipelines. I built a Delta time-travel utility that showed every change between versions, and the risk went away. Ownership to me means fixing the problem you notice, not only the one you’re assigned.”

2. Tell me about something you owned end to end.

> “Cluster right-sizing (22.2). I did the analysis, designed the rollout from the least critical pipelines inward, validated runtimes over several nights, and stated the trade-off upfront: about 15 to 25 percent longer runtime for a projected 50 to 65 percent compute reduction on that workload.”

3. Describe a time you went the extra mile for quality.

> “The DQ engine (22.7). I pushed for checks that block bad data rather than warn, because in regulatory reporting a wrong number is worse than a late one. It’s more work upfront, but it protects the output that matters.”

Leadership without authority

4. Tell me about leading without formal authority.

> Use the Liquid Clustering migration (22.19). Close with: “No one had to follow me. They followed because the benefit was clear and I made it easy.”

5. How do you get buy-in from people who don't report to you?

> "Make the benefit to them clear, lower the effort, and make progress visible. For Liquid Clustering, I explained what they'd gain, offered help on hard cases, and used a shared tracker. And I set a deadline they could actually meet."

6. What does leadership mean to you as a Pod Lead without direct reports?

> "Making the team better and less dependent on me. Setting the technical standard through code reviews, making decisions when the team needs one, and teaching the reasoning behind things so people can decide well without me. The best sign I've done it is that the pod can release without me."

Conflict and disagreement

7. Tell me about a conflict with a colleague.

> Use the phased migration (22.16). Emphasise that you agreed with their goal and argued about the approach, not the person.

8. Tell me about an architecture disagreement.

> "Same example, because it was an architecture decision: big-bang cutover versus phased migration. I made the case in terms of risk to the regulatory deadline, proposed an alternative that still met the goal of speed, and put it in writing. The phased approach was adopted."

9. Tell me about a time you disagreed with your manager.

> "[Use a real example if you have one.] If not: 'I haven't had a significant disagreement with my manager. We've had different views on priorities, and I handle that by bringing data. The sprint resequencing is an example: I proposed a different order for the defect work, showed the reasoning, and it was adopted.'"

10. What do you do when you strongly disagree but get overruled?

> "Commit fully. I make my case before the decision, clearly and in writing. Once it's made, I support it and make it work as well as possible. If new evidence appears later, I raise it calmly then."

Stakeholder management

11. Tell me about managing a difficult stakeholder.

> “The TRIM removal (22.3) depended on another team: I needed the source team to guarantee their values were clean and lowercase. I explained how much that guarantee would save downstream and made the ask specific and small. They agreed, and it unlocked a big performance gain. [If you have a genuinely difficult stakeholder story, use it instead.]”

12. How do you keep stakeholders informed?

> “Early and regularly, in their language. If the 6 AM deadline is at risk, the reporting owners hear from us before it’s missed, with options. Bad news early is a courtesy; bad news late is a failure.”

13. Tell me about working with a partner team.

> “I explained our platform to a partner team led by Annamalai Chidambaram, so they could use it without relying on us for every question. I focused on what they needed to do, not everything the platform does. That reduced interruptions for my team and sped them up.”

Failure and learning from mistakes

14. Tell me about a failure.

> Use the broadcast OOM (22.15).

15. Tell me about a time you learned from a mistake.

> “Same story, focusing on the change: I now test config changes against peak data volumes and prefer targeted broadcast hints over global settings. A mistake is only wasted if the process doesn’t change.”

16. Tell me about a missed deadline.

> Use the UNRESOLVED_COLUMN SLA miss (22.13). Own the miss, be honest that the root cause wasn’t confirmed, then prevention.

Feedback

17. Tell me about critical feedback you received.

> “After my Lyft final round, which was six rounds, I didn’t get the offer, and the clearest feedback was about time pressure in the SQL coding round, not knowledge. It was hard to hear, but it was accurate. I changed how I prepare: I now practise solving out loud against a timer and narrating my approach, not just getting the right answer eventually. It’s directly helped me in the Vivanti rounds.”

18. How do you give difficult feedback?

> “Mostly through code reviews, which is where I give feedback most often. I’m specific, I explain why, and I focus on the code, not the person: ‘This join can duplicate rows if the key isn’t unique. Here’s how to check.’ For recurring issues, I add them to our standards so it isn’t personal. [If you have a specific harder conversation, use it.]”

19. What would your manager say you need to improve?

> “Probably that I sometimes go deep on a problem before stepping back to communicate where I am. I’ve been working on giving shorter, earlier updates, especially under pressure, and stepping back to ask ‘what does the business need to know right now?’ [Adjust to what your manager has actually said.]”

Ambiguity and decision-making

20. Tell me about working with ambiguous requirements.

> Use the 50 defects (22.17, option A). Close: “I turned an unordered list into a strategy by looking at where the problems sat in the data flow.”

21. Tell me about a decision you made with incomplete information.

> “During the UNRESOLVED_COLUMN incident I had to decide quickly which parts of the chain could keep running without knowing the cause. I isolated the pipelines that depended on the changed source and let the independent ones continue, because that protected as much of the output as possible while I investigated.”

22. How do you make technical decisions?

> “Start from the requirement and the constraints (deadline, cost, risk, who maintains it), then compare options on those. I prefer reversible decisions when I’m unsure, like phased migration over big-bang. And I write the reasoning down, so the decision can be revisited if the facts change.”

Mentoring and handling underperformance

23. Tell me about mentoring someone.

> Use 22.18, and add one specific person's story if you have it.

24. How do you onboard a new engineer?

> "System first, details second. I explain the nightly chain, the control tables, where data quality sits and what the deadline means, then give them a small real change to ship with me reviewing. Early success builds confidence faster than reading documentation."

25. How would you handle someone underperforming when you're not their manager?

> "I haven't had to escalate a performance issue, so I'll describe how I'd handle it. First, assume there's a reason: unclear expectations, a skills gap or something outside work. I'd talk to them privately and offer specific help: pairing, a clearer task, a code-review walkthrough. If it continued and affected delivery, I'd raise it with their manager, framed around the work and what I'd tried, not as a complaint about the person."

Prioritisation and pressure

26. How do you prioritise?

> "By impact on the business and the deadline, and by dependency. In our platform, upstream problems come first, because they break everything beneath them. That's exactly the logic behind the 50-defect resequencing."

27. Tell me about working under production pressure.

> Use the UNRESOLVED_COLUMN incident (22.13). Stress the sequence: contain, communicate, fix, prevent.

28. How do you stay calm under pressure?

> "Having a sequence to follow. Contain, communicate, fix, then prevent. When I know what step I'm on, the pressure is manageable. And I communicate early, which removes a lot of stress for everyone."

29. Tell me about a difficult deadline.

> “The Liquid Clustering migration had a six-week deadline across teams that didn’t report to me. I made it achievable by starting early, tracking it visibly, and helping with hard cases. We finished in week five.”

Technical debt and long-term thinking

30. How do you handle technical debt?

> “Make it visible and tie it to risk. Debt that threatens correctness or the deadline gets fixed first. Cosmetic debt can wait. I log deliberate shortcuts with the reason and a plan, so they don’t become invisible.”

31. How do you balance tactical delivery with long-term improvement?

> “I try to make each tactical fix also improve the system. When the UNRESOLVED_COLUMN incident happened, the tactical job was getting the chain running. The long-term job was making sure that failure class couldn’t break it the same way again. When I fixed the Spark joins, I also moved teams to Liquid Clustering, so the gain lasted.”

32. Tell me about a process you improved.

> Use the reference-table diff tool (22.20) or the upstream-first defect process (22.17).

Cross-team coordination

33. Tell me about coordinating across teams.

> Use Liquid Clustering (22.19): shared tracker, clear deadline, help offered.

34. How do you handle dependencies on other teams?

> “Make them explicit early, agree dates with the owners, and have a fallback. For the TRIM removal I depended on the source team’s guarantee, so I got it in writing before changing anything.”

Learning and growth

35. How do you learn a new technology quickly?

> “Build something small and real with it, then explain it in writing. I built a streaming reconciliation project to learn Iceberg and Airflow properly, and I’ve written 290-plus articles. If I can’t explain something simply, I don’t understand it yet.”

36. What’s the most important thing you’ve learned in your career so far?

> “Measure before you fix. Whether it was the Synapse queue, the TRIM on join keys or the cluster costs, the real problem was rarely what everyone assumed. Looking at evidence first has saved me from optimising the wrong thing many times.”

37. What are you working on improving now?

> “Speed under pressure, especially in live problem-solving. It’s why I practise narrating my approach out loud against a timer. And broadening beyond Azure and Databricks into Snowflake and GCP.”

Values and fit

38. Tell me about a time you did the right thing when it was harder.

> “Owning the broadcast OOM immediately and in writing. It would have been easy to call it a platform issue. Being straightforward about it built more trust than hiding it would have.”

39. Tell me about a time you helped someone else succeed.

> “Training the pod on deployments. The goal was for them not to need me, which was recognised with the Win Together award.”

40. What would your colleagues say about you?

> “I’d point to what they’ve actually written: ten recognition awards from six colleagues over about two years, including my manager. The common themes are speed with impact, reliability, and helping others learn the platform.”

Part 24: Consulting, Client and Partner Pressure Questions

How do you communicate with non-technical stakeholders?

> "I start from what they need to decide, not from how the system works. Then I use one simple picture and one real example, and I avoid jargon. If I have to use a technical term, I explain it in the same sentence. And I check understanding by asking what they'd do with the information, not 'does that make sense?' When I explained our platform to a partner team so they could use it on their own, that was exactly the approach: their goal first, the minimum detail, and a worked example."

How would you explain a technical risk to a client executive?

> "In terms of impact, likelihood and options. For example: 'There's a risk the nightly load finishes after your 6 AM reporting deadline as data grows. If that happens, the report is late. We think it's likely within six months at current growth. We have two options: tune the slowest steps, which is cheap and buys about a year, or re-architect the load, which costs more but fixes it long-term. I recommend the first now and planning the second.' Impact, likelihood, options, recommendation, in under a minute."

How do you handle an unhappy client?

> "First, listen fully and don't get defensive. Often the frustration is about something other than the stated issue: missed expectations, surprises, not being kept informed. Then acknowledge it specifically, agree what 'fixed' looks like, and give a clear next update time, then meet it. Rebuilding trust comes from small commitments kept. If it's our mistake, I say so plainly. If it's a misunderstanding, I clarify without making them feel wrong."

What do you do when a client rejects your recommendation?

> "I make sure they rejected it for the right reasons. I check they understood the trade-offs, and I ask what matters most to them, because they may know something I don't: a budget constraint, a political reason, a timeline. If they still choose differently, it's their decision. I document the risk clearly and make their choice work as well as it possibly can. Consultants advise; clients decide."

How do you manage unclear requirements?

> “I turn them into something concrete quickly. I write down what I think they’re asking, including definitions and assumptions, and play it back: ‘So when you say active customer, you mean someone who bought in the last 90 days?’ Then I build the smallest useful version and show it early. People are much better at reacting to something real than describing what they want in the abstract. It’s cheaper to correct an assumption in week one than week six.”

How do you handle changing scope?

> “Changes are normal. What matters is that they’re visible and deliberate. When scope changes, I make the trade-off explicit: ‘We can add this, and it moves the date by a week, or we drop X to keep the date.’ Then the client chooses. What I avoid is silently absorbing changes until quality or the deadline breaks.”

How do you balance delivery speed and technical quality?

> “I separate what can be simplified from what can’t. Scope, polish and nice-to-haves can flex. Correctness, security and the ability to rerun safely can’t, because shortcuts there come back as incidents. So under time pressure I’d deliver a smaller, solid slice rather than a complete but fragile one, and I’d log any deliberate shortcuts as known technical debt with a plan to fix them.”

How do you handle unrealistic deadlines?

> “I don’t say yes, and I don’t just say no. I make the trade-off visible. First, what decision does the deadline actually serve? Often the business needs one number by that date, not the whole system. Then I offer options: the most valuable slice by the deadline, the full scope by a realistic date, or more people with the risks that brings. The client chooses with full information. That’s more useful than a heroic promise that fails.”

How do you manage stakeholders with conflicting priorities?

> “Get the conflict into the open and in writing. Often two stakeholders each think their item is top priority, but neither knows about the other’s. I’d list both, with the cost and impact of each, and take it to whoever can make the call, not decide it quietly myself. My job is to make the trade-off clear and help them agree, then execute on what’s decided.”

How would you handle a production incident during a client presentation?

> "Two things at once: the incident and the audience. I'd acknowledge it calmly and briefly ('we've just had an issue with tonight's load, my colleague is on it'), make sure someone is actually handling it, and continue with a clear promise of when they'll hear more. Afterwards, a short, honest update: what happened, the impact, and what we're changing. Handled calmly, an incident can actually build trust."

How do you build client trust?

> "By doing what I say I'll do, and by being honest when things go wrong. Small promises kept consistently matter more than big ones. I also try to understand their business, not just their tickets, so my suggestions are relevant. And I'm careful with numbers: I'd rather say 'I'll confirm that' than guess."

How do you communicate bad news?

> "Early, directly and with a plan. I don't wait until I have a perfect answer. I say what happened, what the impact is, what we're doing about it, and when they'll next hear from me. At TD, when our chain was at risk of missing 6 AM, we told the reporting owners early with options. Surprises at the deadline are what damage trust, not the problem itself."

How do you handle a mistake made by your team?

> "Externally, the team owns it. I don't point at an individual in front of a client. Internally, I focus on the cause, not the person. Usually a mistake reaches production because a process let it through, so the fix is the process: a test, a review step, a check. That's how I handled my own broadcast-join mistake too."

How do you challenge a senior stakeholder respectfully?

> "Agree with their goal first, then disagree with the approach, with evidence. 'I agree we need this fast. My concern with doing it in one go is X. Here's an option that gets us there with less risk.' In private or in writing, not by embarrassing them in a meeting. That's how the phased migration was adopted at TD. The senior colleague's goal was speed, and the phased plan protected both speed and the deadline."

How do you estimate a data engineering engagement?

> “Bottom-up, from an inventory, with the risky parts called out. I’d count sources, pipelines and their complexity, and group them: simple, medium, complex. Integration is usually harder than ingestion: matching customers across systems takes longer than loading them. I’d add explicit time for testing, reconciliation and handover, which are the things most often underestimated. Then I’d give a range, not a single number, with the assumptions that drive it, and refine it after a short discovery phase.”

How do you identify opportunities beyond the immediate request?

> “By understanding why they asked. If a client asks for a faster pipeline, I also look at cost, reliability and whether they’re recomputing things unnecessarily. The EDF reference-table diff tool I built at TD came from noticing a risk nobody had asked me to fix. The key is to raise opportunities as suggestions with a clear benefit, not to expand scope without agreement.”

How do you make sure a solution is maintainable after handover?

> “Plan the handover from day one. Pair the client’s engineers with ours during the build, not at the end. Write documentation and runbooks as we go, based on real incidents. Keep the design as simple as the problem allows, because clever is hard to maintain. And before we leave, have the client’s team operate it while we watch. At TD I did exactly this with deployments: the pod could release without me.”

More partner pressure questions

1. Why should a client trust your recommendation?

> “Because I’d show my reasoning and the evidence, not just the conclusion, and I’d be clear about the risks and what I’m unsure of. At TD, when I argued for phased migration, I laid out the risk of big-bang against our deadline. People trusted it because they could check the reasoning.”

6. How do you quantify the value of a data engineering project?

> “In business terms: cost saved (like the projected 50 to 65 percent compute reduction from right-sizing), risk reduced (regulatory deadlines met, errors caught), time saved (onboarding faster), or revenue enabled (decisions the data now supports). If I can’t name the value, I question whether we should build it.”

7. When is a pipeline reliable enough?

> “When it meets the agreed service level for its consumers, and more reliability would cost more than the failures it prevents. A regulatory chain needs near-perfect reliability. An internal exploratory dashboard doesn’t. Agree it explicitly.”

9. How do you prevent over-engineering?

> “Build for today’s known requirements with clear extension points, not for imagined scale. Ask ‘what breaks if we don’t build this?’ If nothing, don’t build it yet.”

12. How do you communicate uncertainty?

> “State what I know, what I don’t, and what would change my answer. For example: ‘Revenue fell about 15%. Two-thirds is explained by fewer orders from returning customers. I’m still confirming whether a data delay affected the last three days.’”

14. How would you handle disagreement between Sydney stakeholders and the Bengaluru team?

> “Get both sides to agree on the facts and the goal before debating solutions, and write the decision down. Time zones make misunderstandings expensive, so written decisions and a short overlap call beat long email threads.”

Part 25: Technical and Architecture Conversations

A partner's architecture questions are conversations, not exams. The winning pattern:

1. **Clarify** two or three requirements that change the design.
2. **Open with a 30-second shape** of the answer.
3. **Draw it** left to right.
4. **Walk the layers**, naming trade-offs as you go.
5. **Cover the non-functionals**: reliability, governance, security, operations, cost.
6. **Invite challenge:** "Where would you like me to go deeper?"

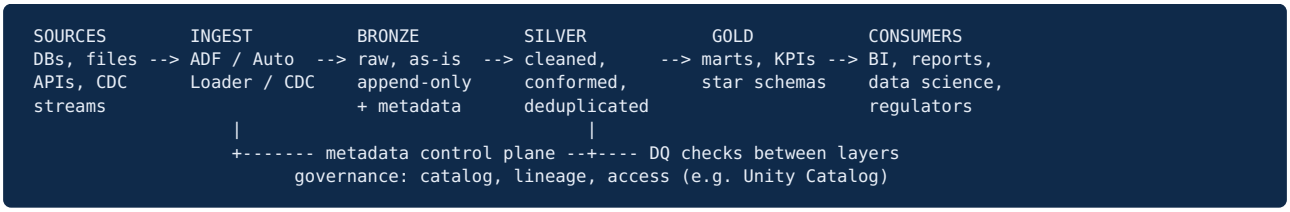
> **Honesty rule for this section:** say "at TD we…" only for things you've actually done. For everything else, say "I'd…" or "the standard approach is…". Tools you haven't used in production (Purview, Fabric, private endpoints, Snowflake, BigQuery) are fine to discuss, as long as you don't imply hands-on experience.

25.1 Design an enterprise lakehouse

Clarify: sources and volumes; freshness needed (daily, hourly, near real time); who consumes it (BI, data science, regulators); regulatory or audit requirements.

30-second opening:

> "I'd build it in layers on open table formats: raw data landed as-is, cleaned and conformed data in the middle, and business-ready data on top. The whole thing would be metadata-driven, so adding a source is configuration, and governed centrally for access, lineage and quality. That's essentially the platform I run at TD."



Deep dive points:

- **Storage:** Delta Lake (ACID, time travel, MERGE) on cloud object storage.
- **Ingestion:** batch copies, CDC for databases, streaming where latency needs it.
- **Modeling:** Data Vault or conformed dimensions in the middle for integration; star schemas on top for consumption.
- **Reliability:** idempotent loads, run IDs, data quality gates, reconciliation.
- **Governance:** central catalog, role-based access, lineage, PII masking.
- **Cost:** job clusters, right-sizing, incremental processing.

Partner challenges:

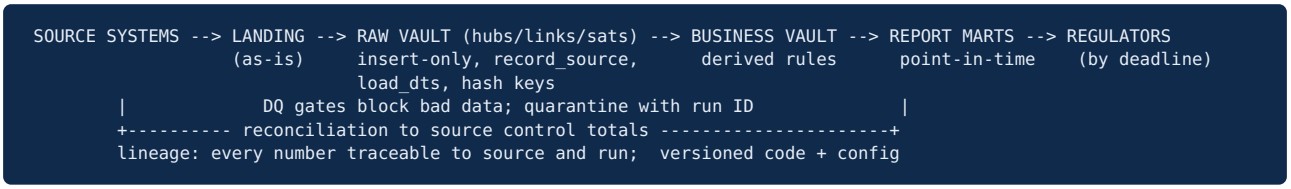
- *"Why not just a warehouse like Snowflake?"* > "For mostly structured BI with SQL users, a warehouse is simpler, and I'd happily recommend it. A lakehouse wins when there's heavy Spark processing, data science, streaming or very large volumes, or when the client wants open formats to avoid lock-in."
- *"How do you stop it becoming a data swamp?"* > "Governance from day one: every dataset has an owner, quality checks, a description and access rules. Nothing reaches the gold layer without them."

25.2 Design a regulatory reporting platform

Clarify: which regulators and reports; deadline; how far back reports must be reproducible; tolerance for late versus wrong.

30-second opening:

> "This is my home ground. The priorities are correctness, traceability and reproducibility, then timeliness, then cost. I'd integrate sources in an insert-only, auditable model like Data Vault, with quality gates that block bad data, reconciliation to source control totals, and the ability to reproduce any past report exactly."



Deep dive points:

- **Reproducibility:** insert-only history plus versioned code and configuration. Time travel helps, but VACUUM limits it, so long-term reproducibility comes from the vault history itself.
- **Deadline management:** explicit dependency chain, critical-path sizing, early warning.
- **Controls:** block-or-warn DQ rules by severity, control totals reconciled per source, sign-off before submission.

Partner challenges:

- *"What if the deadline and correctness conflict?"* > "Usually publish late rather than publish wrong. A late submission is a documented exception; a wrong one may need restating. But that's a business decision, so I'd escalate early with options rather than decide alone."
- *"How do you reproduce a report from a year ago?"* > "Query the vault as of that date and rerun the same versioned logic. Every row has a load date, and code and config are versioned, so the inputs and the rules can both be reconstructed."

25.3 Design Customer 360

Clarify: which systems hold customer data; what "360" is for (service agents, marketing, risk); freshness; privacy constraints.

30-second opening:

> "The hard part of Customer 360 isn't storage, it's identity: the same person exists in many systems with different keys. So I'd focus on identity resolution to create one customer key, then build a unified profile with history, strict privacy controls, and serve it differently for analytics and for real-time lookups."

```
CRM, core banking, --> INGEST --> STANDARDISE --> IDENTITY RESOLUTION --> CUSTOMER PROFILE --> BI / ML (lakehouse)
cards, web, support              names, emails   match rules + review     SCD2 history,       APIs / agents
                                 phones, keys    -> golden customer ID    consented attrs     (low-latency store)
                                   PII masked, consent flags, access by role, audit trail
```

Deep dive points:

- **Identity resolution:** deterministic matching first (exact IDs, emails), then fuzzy matching with confidence scores, and human review for the uncertain middle.
- **History:** SCD2 on key attributes, so you know what was true when.
- **Privacy:** consent flags, masking, purpose-based access.
- **Serving:** the lakehouse for analytics; a low-latency store for apps that need one customer instantly.

Partner challenges:

- *"Two records might be the same person. How do you decide?"* > "By confidence. High-confidence matches merge automatically, low ones don't, and the middle goes to review. Merges must be reversible, because a wrong merge is worse than a missed one."

25.4 Design a real-time fraud detection platform

Clarify: decision latency needed (block the transaction, or alert afterwards); transaction volume; how models are built and updated; false-positive tolerance.

30-second opening:

> "The first question is whether we must block a transaction in milliseconds or flag it seconds later, because that changes everything. I'd stream transactions through Kafka, compute features over windows (spend in the last hour, new merchant, distance from usual location), score them with rules plus a model, and act. Batch keeps the historical features and retrains models, and the batch path is the source of truth."

```
CARD TXNS --> KAFKA --> STREAM PROCESSOR ------------> SCORING (rules + model) --> ALLOW / BLOCK / ALERT
                        windowed features  <-- feature store (recent + historical)
                |
                +--> LAKEHOUSE (all txns, labels) --> batch features, model retraining, reconciliation
```

Deep dive points:

- **Latency:** in-path blocking needs millisecond scoring and a fast feature lookup. Post-transaction alerting can use Spark Structured Streaming micro-batches.
- **Event time and watermarks:** use event time; late events are handled by watermark, and batch catches what streaming missed. That's exactly what my reconciliation project tests.
- **Exactly-once:** checkpoints plus an idempotent or transactional sink.
- **Feedback loop:** confirmed fraud labels flow back for retraining.

Partner challenges:

- *"Spark Structured Streaming or Flink?"* > "For sub-second, event-at-a-time processing with large state, Flink is usually better. For seconds-level latency where the team already knows Spark and Databricks, Structured Streaming is simpler to run. I'd choose based on the latency requirement and the team's skills."
- *"What about false positives?"* > "They cost customers and money, so thresholds are a business decision. I'd measure precision and recall and let the business choose the trade-off."

25.5 Design a CDC-based data platform

Clarify: source databases; latency needed; whether deletes matter; history requirements.

30-second opening:

> "Capture changes from the database's log rather than querying tables, stream them through Kafka, and apply them to the lakehouse with MERGE. The important details are ordering, deletes, schema changes, and making the apply step idempotent so replays are safe."

```
SOURCE DB --> CDC (log-based, e.g. Debezium) --> KAFKA (+ schema registry) --> BRONZE (raw change events)
                                                                                          |
                                                                  SILVER: MERGE by key, ordered by log position (LSN)
                                                                  inserts / updates / deletes; SCD2 if history needed
```

Deep dive points:

- **Log-based capture:** low load on the source, and captures deletes.
- **Ordering:** apply by log sequence number per key, so an older change never overwrites a newer one.
- **Deletes:** apply as deletes or soft deletes; soft deletes suit audit.
- **Schema changes:** a schema registry, with compatibility rules.
- **Initial load:** snapshot first, then stream changes from that position.

Partner challenge: *"What if events arrive out of order?"* > "Keep the latest by log position per key within each batch, and guard the MERGE so an older change can't overwrite a newer row."

25.6 Migrate SAS or a legacy warehouse to Databricks

30-second opening:

> "Inventory from actual usage, not from asking people, because many legacy jobs turn out to be unused. Then migrate in waves by pattern, prove each wave with automated reconciliation against the legacy output, cut over per consumer, and decommission. The hardest part is understanding undocumented logic, and that's where AI-assisted analysis helps, as long as reconciliation, not the AI, proves correctness."

```
INVENTORY (usage logs) --> CLASSIFY (retire / migrate / rebuild) --> WAVES by pattern
     --> BUILD in PySpark --> DUAL RUN + RECONCILE --> CUT OVER per consumer --> DECOMMISSION
                        AI-assisted logic discovery, human-verified
```

Partner challenges:

- *"The numbers differ slightly between old and new."* > "Expect that: floating-point order, NULL handling, rounding, time zones. Explain every difference and agree a tolerance with the business. 'It matches' must be a query result, not an opinion."
- *"How long would it take?"* > "It depends on the inventory. I'd give a range after a short discovery, with the assumptions that drive it."

25.7 Cloud cost optimisation

30-second opening:

> "Measure first, then fix the biggest spenders. Cost follows compute time and data scanned. I'd rank workloads by cost, then reduce the work itself (less shuffle, better pruning, incremental instead of full recompute), right-size compute against the actual critical path, and use job clusters with auto-termination. That's the method I used at TD, where the evidence on one over-provisioned job supported a projected 50 to 65 percent compute reduction."

```
MEASURE cost per pipeline --> RANK --> REDUCE WORK (prune, incremental, fewer shuffles)
       --> RIGHT-SIZE (utilisation + critical-path slack) --> GUARDRAILS (policies, auto-terminate, budgets, alerts)
```

Partner challenge: *"An optimisation made it faster but more expensive."* > "Then it's only worth it if the speed has business value, like protecting a deadline. Otherwise I'd take the cheaper option. Faster isn't automatically better."

25.8 Data quality and reconciliation

30-second opening:

> "Four dimensions (complete, accurate, timely, unique), each with a concrete check, and a severity that decides whether failing data is blocked or just flagged. Reconciliation to the source's control totals is the final proof."

```
LOAD --> CHECKS (keys, nulls, ranges, referential, volume, freshness)
           | pass --> next layer
           | fail (critical) --> QUARANTINE + alert owner (rule, run ID) --> fix at source --> reprocess
         RECONCILE: counts and control totals vs source, per run
```

Partner challenge: *"Won't blocking data break the deadline?"* > "Sometimes, and that's a business decision. For regulatory data, correct and late usually beats on time and wrong. For less critical data, warn and continue."

25.9 Platform observability and SLOs

30-second opening:

> "Define what 'working' means for consumers (data landed by a time, with quality passing) as a service level objective, then measure it. Alert on symptoms consumers care about, like freshness and failed quality checks, with a named owner and runbook for every alert. And alert on the root cause, not the forty downstream failures it triggers."

Key metrics: freshness (landed by expected time), success rate, runtime trend on the critical path, data quality pass rate, cost per run. The best maturity signal: how often a consumer tells you about a problem before your alerts do.

Partner challenge: *"How do you avoid alert fatigue?"* > "Every alert must be actionable with an owner. If the response is 'ignore it,' delete the alert. Review alert volume regularly."

25.10 Disaster recovery and replay

30-second opening:

> "Two separate problems: losing the platform, and corrupting the data. For the platform, infrastructure as code, replicated storage and a tested failover procedure, sized to the business's recovery time and data loss tolerance. For corruption, which is far more common, idempotent pipelines, retained raw data and table history, so we can restore a table and replay from raw safely."

Key points:

- Raw landing data is kept, so anything downstream can be rebuilt.
- Delta time travel and RESTORE for recent mistakes, within the retention window.
- Idempotent loads (MERGE, partition overwrite), so replay doesn't duplicate.
- **Test it.** An untested recovery plan is a hope, not a plan.

Partner challenge: *"What recovery targets would you set?"* > "The business sets them, based on the cost of downtime and data loss. I'd make the cost of each option clear so they choose sensibly."

Comparisons and concepts (30-second answers plus a challenge)

Databricks versus Synapse versus Fabric

> "Databricks is the strongest for Spark-heavy engineering, data science and lakehouse at scale, and it runs on all three clouds. Synapse combines a SQL warehouse (Dedicated SQL Pool) with Spark in Azure. In my experience, Dedicated Pool's concurrency limits became a bottleneck for us. Fabric is Microsoft's newer all-in-one SaaS platform on OneLake, attractive for Microsoft-centric organisations wanting simplicity, and it's where Microsoft is investing. I'm certified in Fabric (DP-700) but haven't run it in production, so I'd say that openly."

Challenge: "Which would you recommend?" > "It depends on the client's existing estate, skills and workload. There isn't a universal answer, and I'd be wary of anyone who gives one."

Lakehouse versus warehouse

> "A warehouse is simpler and excellent for structured BI and SQL users. A lakehouse handles all data types, heavy processing, streaming and data science on open formats, at lower storage cost. The gap has narrowed: warehouses now handle semi-structured data, and lakehouses have strong SQL. So I'd choose based on the workloads and the team."

Batch versus streaming

> "Choose by the latency the business actually needs, not the latency it asks for. Most 'real-time' needs are met by 15-minute or hourly batch, which is cheaper and simpler. Streaming is justified when delay costs money or risk, like fraud or operational alerts. Often the right answer is a small streaming path with batch as the source of truth."

Data Vault versus dimensional modeling

> "They solve different problems. Data Vault is an integration layer for many changing sources with strict audit needs: insert-only history and lineage on every row. Dimensional modeling is for consumption: star schemas that analysts can query easily. At TD we use both: Data Vault underneath, marts on top. For a client with few stable sources, I'd go straight to Kimball."

Challenge: "Isn't Data Vault over-engineering?" > "For many clients, yes. It pays off with many sources, frequent change and regulators asking where numbers came from. Without those, it's overhead."

Kafka and event-driven architecture

> “Kafka is a durable, partitioned log. Producers write events, consumers read at their own pace, and events can be replayed. Partitions are the unit of parallelism, and ordering is only guaranteed within a partition, so you key events by what must stay in order, like account ID. I learned partition sizing the practical way in my internship, when under-partitioned topics caused consumer lag.”

Delta Lake and ACID transactions

> “Delta adds a transaction log on top of Parquet files. Every write is an atomic commit to the log, so readers see a consistent snapshot, never a half-written table. Concurrent writers use optimistic concurrency: if two conflict, one fails and retries. We saw that as ConcurrentAppendException during the Liquid Clustering migration. It also gives time travel, MERGE and schema enforcement.”

Spark performance optimisation

> “Measure first in the Spark UI: slow stages, shuffle size, spill, skew. Then work down the ladder: read less (prune and select fewer columns), move less (broadcast small tables, filter before joins), compute less (incremental, pre-aggregate), fix layout (clustering, compaction), and only then add compute. My biggest single win was removing TRIM from join keys.”

Data skew and partitioning

> “Skew is when a few keys hold most of the rows, so one task does most of the work. You see one task running far longer than the rest. Fixes: AQE’s skew join handling, broadcasting the other side, or salting the hot key. For partitioning, partition by low-cardinality columns most queries filter on, usually date, and cluster by higher-cardinality ones. Over-partitioning creates millions of tiny files, which is its own problem.”

Data lineage and governance

> “Lineage answers two questions: where did this number come from, and what breaks if this source changes? In Data Vault, every row carries its record source and load time, which gives row-level lineage. Table-level lineage comes from the catalog or from parsing the pipeline code. Governance is ownership, access control, quality standards and definitions. The goal is that every important dataset has an owner and a definition.”

Unity Catalog and Purview

> “Unity Catalog is Databricks’ governance layer: one place for access control, lineage and auditing across workspaces. Microsoft Purview is broader, cataloguing and governing data across the whole Azure and Microsoft estate, not just Databricks. They can be used together: Unity Catalog for fine-grained Databricks access, Purview for enterprise-wide discovery. [Say which you’ve used hands-on, if either.]”

Azure security and private connectivity

> “The principles: least-privilege access through Entra ID groups (at TD, access is granted through AD groups, with elevated access requested through ServiceNow), secrets in Key Vault rather than code, encryption at rest and in transit, and private networking so data traffic doesn’t cross the public internet, using private endpoints and virtual network integration. I’d be clear that network design is usually owned by a platform team, and I’d work with them rather than claim I designed it.”

CI/CD and release governance

> “Code and configuration both go through Git, pull requests with review, automated tests, and promotion through environments. At TD, we use GitHub and digital.ai. For data platforms, I’d add data tests (schema and quality checks) to the pipeline, not just code tests, and treat configuration changes with the same discipline as code, because in a config-driven platform a bad config can break many pipelines at once.”

Data products and Data Mesh

> “Data Mesh gives ownership of data to the domains that know it best, with each domain publishing data products: well-defined, documented, quality-checked datasets with an owner and a service level, on a shared self-service platform with common governance. It suits large organisations where a central team has become a bottleneck. For smaller organisations it’s often too much overhead. Even without full Data Mesh, the data product idea (owner, definition, quality, SLA) is worth adopting.”

Part 26: Difficult and Challenging Questions

The tone for all of these: **calm, honest, brief, then move forward.** Defensiveness is the only way to lose on these questions. Treat each as a fair question.

"You have about four years of experience. Why should we hire you for a senior role?"

> "That's a fair question. I'd say judge me on what I've owned, not the number of years. I own a regulatory platform with 150 production pipelines and a hard deadline, I've led changes across teams without authority, and I've delivered measurable savings. I'm not claiming to be an architect with fifteen years of experience. I'm saying I can own delivery on a client engagement from day one, and grow quickly into more. I'm also comfortable with whatever level you think fits. I'd rather be placed right and exceed expectations."

Avoid: arguing that years don't matter.

"Are you overstating your leadership responsibilities?"

> "I want to be precise, so thank you for asking. My title is Data Engineer II. Pod Lead is a technical leadership role with no direct reports: I own code reviews, releases, technical decisions and mentoring for the pod, but I don't manage people or do performance reviews. Where my resume says 'led,' it means I led the technical work, like the phased migration or the Liquid Clustering rollout, through influence rather than authority."

"Why didn't Lyft make an offer after six interviews?"

> "It was a strong loop and I reached the final round. The feedback I took from it was about time pressure in the SQL coding round, not knowledge. I solved the problems, but not fast enough. It was a useful lesson, and I've changed how I prepare: practising live problem-solving against a timer and narrating my approach. That's helped me through the Vivanti rounds so far."

Avoid: blaming the interviewer, or implying you declined.

"What is your biggest weakness?"

> "Under time pressure, I can go deep into solving a problem before I've communicated my approach, which makes me look slower than I am. The Lyft loop showed me that clearly. I've been working on it by stating my plan first (the grain, the steps, the assumptions) and then writing. It's made a real difference."

“What is your biggest failure?”

> Use the broadcast OOM (22.15). Keep it to 60 seconds. End on the process change.

“Why should we pay you more than another candidate with similar experience?”

> “Because of what I can deliver on day one: production Databricks and Spark performance work with measurable results, regulated banking experience that financial-services clients value, and the ability to explain it to their executives. But I’d rather prove that than argue it. I’m asking for a fair market rate for that combination, not a premium for its own sake.”

“What if you’re assigned work outside Databricks?”

> “I’d welcome it. That’s part of why I want consulting. The fundamentals transfer: modeling, performance, reliability, cost. On Snowflake, for example, clustering keys and micro-partition pruning are the same ideas as Liquid Clustering and data skipping. I’d need a short ramp-up on specifics, and I’d get certified quickly through the Academy.”

“What if the role is more consulting than coding?”

> “I’d enjoy that too. The parts of my job I find most rewarding are often explaining a platform, persuading a team, or making a trade-off clear. I’d want to stay hands-on enough to stay credible technically, but I understand that the value to a client is often the judgment and communication, not just the code.”

“What if you have to work with a demanding client?”

> “Demanding usually means they care, which is good. I’d make expectations very clear in writing, update them before they have to ask, and deliver small commitments reliably. Regulatory stakeholders are demanding by design, and I’ve worked with a 6 AM deadline every night, so pressure and high standards are normal for me.”

“What if the project isn’t in banking?”

> “That’s fine. My banking experience is a strength, not a limit. The disciplines (correctness, reconciliation, lineage, reliability) apply to any industry. I’d bring them and learn the new domain’s language quickly.”

"Why are you leaving a stable role at TD Bank?"

> "My Canadian work permit ends in October, and I've decided to build my career in India. I'm using that moment to move deliberately into consulting, which is the direction I want to grow in. TD has been a great foundation, and I'm leaving on good terms. My senior manager and AVP have actually supported my move."

Accurate note: your TD leadership referred you for the Cognizant rebadge, so "supported my move" is true. Don't overstate it.

"Are you only taking this because your work permit is expiring?"

> "The permit sets my timing, I won't pretend otherwise. But it doesn't decide where I go. I have other options, including a rebadge that would keep me on my current account. I'm choosing Vivanti because I want consulting and the platforms you work with. If the permit weren't a factor, I'd still want this kind of role. It would just have happened a bit later."

"What would make you leave Vivanti?"

> "If I stopped learning, or stopped doing work that matters to clients. Those are what I value most. Given what I've heard about Vivanti's growth and client work, I don't expect that to be a problem."

"How do we know you'll stay?"

> "I'm moving country to build my career in Bengaluru, and I'm choosing a consultancy deliberately, so I'm looking for somewhere to grow over years, not a stepping stone. At TD I've stayed and grown from engineer to Pod Lead. I'd rather show my commitment through my work than promise it."

"What if you disagree with the partner or the client's architect?"

> "I'd raise it privately first, with evidence, and offer an alternative, not just an objection. If the decision goes the other way, I'd support it fully in front of the client. We should never disagree in front of the client. And I'd document the risk so it's visible if it matters later."

“Tell me something you’re not good at.”

> “Hands-on experience outside the Azure and Databricks world is thinner. I’ve worked with AWS and Iceberg in my own project and I’m certified in Fabric, but I haven’t run Snowflake or GCP in production. I learn quickly, but I’d be honest with a client about that from the start.”

“What would your manager say you need to improve?”

> Use the answer in Part 23, question 19. Adjust it to real feedback you’ve received.

“Why should we choose you over someone with eight years of experience?”

> “You might not, for some roles, and that’s fair. Where I’d be the better choice is where you need someone hands-on, current and fast: I work in Databricks, Spark and Delta every day, I have measurable results, and I’m growing quickly. Someone with eight years may bring more architecture breadth. I’d bring depth, energy and a lower cost, and I’d grow into the broader role.”

Part 27: Compensation and Offer Discussion

Ground rules: never lie about a competing offer or its value. You may decline to share details; that's normal and ethical. Keep pay talk to the end of the interview or to HR, and let the partner raise it.

The facts you're working with

Item	Fact
Vivanti's posted band	₹20–35 LPA, Consultant – Data Engineering, Bengaluru, 5+ years (their own posting)
Reddit opinion	₹18–26 LPA for about 4 years, 85–90% fixed (unverified, promotional comment)
Cognizant	Rebadge in progress. Verbal 16.5 LPA fixed; you requested a grade review and anchored at 30
Your ask	About 30 LPA or above
Your broader target	50–80 LPA at GCCs and product companies. Well above Vivanti's posted band

Decide your floor before the call and keep it to yourself.

"What are your salary expectations?"

First, try to learn the band:

> "I'd like to understand the level you have in mind for me, and the range for that level. Then I can give you a number that fits."

If pressed for a number:

> "Based on the Bengaluru market for data engineers with production Databricks, Spark performance, cost optimisation and regulated-banking experience, I'm looking at around 30 to 35 LPA. I'm flexible on the structure between fixed and variable."

"What is your current compensation?"

> "I'm paid in Canadian dollars, so a straight conversion doesn't reflect the Bengaluru market. I'd rather anchor on the market for this role. I'm happy to share my current details during the formal process."

“Do you have another offer?”

> “Yes, I have a verbal offer from a large IT services firm, for a rebadge onto my current account. Terms are still being finalised. Vivanti is my preference because of the consulting work, so I’m keen to see if we can find something that works.”

Truthful: the Cognizant offer is verbal and under discussion. Don’t call it written or final if it isn’t.

“What has Cognizant offered?”

You’re not obliged to share the figure. Stay truthful:

> “I’d rather not share the specific figure while it’s still being finalised. What I can say is that it’s structured as a service-firm grade, and I’ve asked them to review the level. For Vivanti, I’d prefer we set the number on the value of this role.”

If pushed hard: don’t invent a number. If you do share it, share the real one, then explain it’s a grade-placement issue you’re contesting.

“Why are you asking for about 30 LPA or above?”

> “Three reasons. The role is client-facing and billable, and I can deliver on Databricks and Spark engagements from the start. I bring regulated banking experience, which is valuable to financial-services clients. And I have measurable results in performance and cost. 30 to 35 sits within your posted range for this role, so I think it’s fair for what I bring.”

“What is your minimum acceptable compensation?”

Don’t reveal it.

> “I’d rather not negotiate against myself. If you share what you have in mind, I’m confident we can find a number that works for both of us.”

“Would you accept less for the right opportunity?”

> “The opportunity matters a lot to me, and it’s why Vivanti is my preference. But I’d want the offer to reflect the value I bring. If there’s flexibility in the level, the fixed component or a joining bonus, I’m sure we can find a fair outcome.”

Fixed versus variable

> “Could you share how the package splits between fixed and variable, and how the variable part has paid out in practice?”

Evaluate on the **fixed** component. Variable pay isn’t guaranteed.

Joining bonus

> “Since I’m relocating from Canada, would a joining bonus or relocation support be possible? It would help bridge the move.”

Notice period and joining date

> “My work permit in Canada ends in mid-October, so I can join soon after that. [Confirm your exact TD notice period and last day.] I’d want a few days to relocate, so a joining date around [date] would work well.”

Check before saying it: your TD notice period and last working day.

Relocation timeline

> “I’m relocating to Bengaluru in [month]. I’ll be settled and available from [date].”

How to counter respectfully

> “Thank you, I really appreciate the offer, and I’m excited about the role. It’s a bit below what I was expecting, based on the scope and my experience. Would there be flexibility to bring the fixed component to [number]? If that’s possible, I’d be ready to accept.”

Name **one** specific number, slightly above what you’d accept, and explain it with value, not need.

How to ask for time to review an offer

> “Thank you, this is great to receive. Could I have until [two or three working days] to review the details? I want to make a considered decision.”

How to compare Vivanti against Cognizant

Factor	What to compare
Fixed pay	The real number. Ignore headline CTC inflated by variable pay and benefits
Level and title	Which grade, and how fast you’d be reviewed

Factor	What to compare
Growth	Variety of clients and stacks versus the same account
Learning	Academy, certifications, mentoring
Stability	Company size, bench policy, client pipeline
Five-year value	Which role makes your next move (to a GCC or product company) easier
Timing	Which one closes before your permit deadline

Cognizant is your safety net and floor, not your destination. Don’t let it anchor Vivanti’s number.

Part 28: Questions to Ask the Partner

Ranked from strongest to weakest. The strongest ones show business thinking and genuine interest in doing the job well.

Rank	Question	Why it works
1	“What would make someone in this role a clear success in their first six to twelve months, from your point of view?”	Shows you’re thinking about delivering, and tells you exactly how you’ll be judged
2	“What kinds of data problems are your clients bringing most often right now: migrations, AI readiness, cost, or something else?”	Business-minded; tells you what you’ll actually work on
3	“What separates consultants who do really well at Vivanti from those who are just good engineers?”	Shows self-awareness; the answer is your roadmap
4	“How does the Bengaluru team work with clients in Australia day to day?”	Practical and relevant to a Sydney partner
5	“How is the Databricks practice growing, and where do you see it going next?”	Plays to your strength; shows platform interest
6	“What’s an engagement you’re particularly proud of, and what made it work?”	Partners enjoy this; builds rapport
7	“How much of a typical engagement is hands-on building versus advising and architecture?”	Clarifies the role honestly
8	“Where does AI and data modernisation fit into Vivanti’s strategy over the next year or two?”	Forward-looking
9	“What opportunities are there to grow into architecture or technical leadership?”	Ambition, framed around contribution
10	“How are consultants matched to clients and engagements?”	Useful, lower impact
11	“How do you support learning: the Academy, certifications, mentoring?”	Fine, but partly answered on the website
12	“How is performance evaluated for consultants?”	Useful; slightly HR-flavoured
13	“Are there opportunities to work onsite with clients in Australia or elsewhere?”	OK late in the conversation; don’t lead with it
14	“What does utilisation look like, and how is time between engagements used?”	Important, but better for HR
15	“How is the team structured in Bengaluru today?”	Weakest; mostly factual

Save for HR, not the partner: bench policy details, leave, benefits, bonus timing, relocation policy, probation terms.

Pick your best three based on how the interview goes

- **If it was very technical:** ask 1, 2 and 5.
- **If it was business- and consulting-focused:** ask 1, 3 and 6.
- **If rapport is strong and time is short:** ask 1 and 3 only.

Always listen to the answer and respond briefly. For example: “That’s helpful. The migration work sounds very close to what I’ve done with our orchestration move.”

Questions tailored to Tim and Subhash

Your two interviewers have specific backgrounds (Part 2). These questions show you’ve thought about their world without being intrusive. Use one or two in place of the generic ones above.

For Tim (Snowflake, data modeling, BI, engagement lead):

1. “What kinds of problems are your Snowflake clients bringing to Vivanti right now: new platforms, migrations, or getting more value from what they have?”
2. “On your engagements, how much of the work is data modeling and making data usable for the business, versus platform and pipeline engineering?”
3. “What do you look for in an engineer you’d put in front of one of your clients?”
4. “With AI changing how clients use data, where do you see data modeling and engineering fitting in over the next couple of years?”

For Subhash (Bengaluru director, modernisation, delivery):

1. “How is the Bengaluru team structured today, and how does it work with engagements led from Sydney?”
2. “What does a typical first engagement look like for a new data engineer in Bengaluru?”
3. “Where do you see the biggest growth for the Bengaluru team over the next year?”

The best combination if both are present: ask Tim question 3 (“What do you look for in an engineer you’d put in front of a client?”), then Subhash question 1. You get Tim’s success criteria and Subhash’s view of your day-to-day.

Part 29: Mock Interviews

Run each mock with a 45-minute timer, out loud, without notes. Record yourself if you can. Score yourself afterwards with the scorecard.

Mock A (Retail analytics)

1. **Warm-up (5 min):** Explain the logical order of SQL execution. Why can't you filter a window function in WHERE?
2. **Case (15 min):** Case 2, revenue dropped 15%. Find out why.
3. **Follow-up (5 min):** "Your query takes 20 minutes on 2 billion orders. What do you do?"
4. **Case (12 min):** Case 3, new versus returning revenue.
5. **Rapid fire (5 min):** questions 4, 6, 8, 17 and 27 from Part 16.
6. **Close (3 min):** "What would you check before sending these numbers to a client?"

Mock B (Banking)

1. **Warm-up (5 min):** Hash join vs broadcast join. When does each go wrong?
2. **Case (15 min):** Case 7, end-of-day balances with missing days.
3. **Follow-up (5 min):** "Now do it incrementally for 10 million accounts."
4. **Case (12 min):** Case 8, three consecutive missed payments.
5. **Rapid fire (5 min):** questions 13, 17, 30, 34 and 38.
6. **Close (3 min):** "Tell me about a time your numbers didn't match someone else's." (Use your own experience.)

Mock C (Open-ended and performance)

1. **Warm-up (5 min):** "A query scans 5 TB but returns 100 rows. What's wrong?"
2. **Case (15 min):** Case 20, the CFO dashboard table.
3. **Case (12 min):** Case 12, DAU, MAU and stickiness, including the performance discussion.
4. **Design follow-up (5 min):** "How would you choose partition and cluster columns for the events table?"
5. **Rapid fire (5 min):** questions 3, 9, 23, 28 and 36.
6. **Close (3 min):** your two questions for the partner.

Scorecard for the SQL mocks (score each 1 to 5)

Dimension	What a 5 looks like
Clarified before writing	Asked about the goal and the grain, and stated definitions
Stated assumptions	Said them out loud, and they were sensible
Structure	Built CTE by CTE, narrating each step
Correctness	Grain, joins, NULLs, ties and duplicates handled

Dimension	What a 5 looks like
Validation	Walked through edge cases without being asked
Alternatives	Offered at least one other approach, with its trade-off
Performance	Mentioned how it scales and what to change
Communication	Never silent more than 20 seconds; business language

Anything scoring 3 or below goes on your leak list. Redo that case the next day.

Mock D (Resume cross-examination)

Run this out loud with a 30-minute timer. Better still, have someone read the partner's lines and interrupt you. Cover the guidance until you've answered.

Partner: "I've read your resume. Give me two minutes on what you actually do day to day."

> *Testing: clarity, and whether your summary matches your resume.* Use the EDF2 2-minute overview. Mention your real title, Data Engineer II and Pod Lead.

Partner (interrupting after 60 seconds): "OK, shorter. One sentence."

> *Testing: executive communication.* "I keep 150 regulatory pipelines on Databricks reliable, fast and correct by 6 AM every night, and lead the pod that builds them."

Partner: "Fifteen petabytes. What's in it?"

> *Testing: evidence.* Use your Part 17 answer. If you can't break it down, give the honest version and redirect to the nightly volume.

Partner: "You say you architected it. What was there before you joined?"

> *Testing: ownership versus overclaiming.* Be precise about what existed and what you changed.

Partner: "$80,000. Walk me through the maths."

> *Guidance:* if you can derive it, walk through it. If not, pivot honestly to the verified case (22.2): "That's an annualised estimate across the recommendations. Let me show you the evidence on one workload." *Testing: numeric evidence.* Your prepared calculation from the evidence ladder. **If you don't have it, don't improvise one.**

Partner: "Good. Now, you optimised a join. Write me the SQL that finds which pipelines can run next, given your control tables."

> *Testing: SQL tied to your own resume.* Part 20, exercise 3. Narrate the double NOT EXISTS.

Partner: “What if a dependency failed and then succeeded on retry?”

> “Both approaches check for a successful run, so the retry counts. I’d confirm the business rule, though. Some teams want a failed run investigated before downstream starts.”

Partner: “You trained Copilot agents. How?”

> *Testing: honesty about AI vocabulary.* “Grounded, not trained,” then the verification-by-reconciliation point.

Partner: “Eight thousand tables. How many did you personally migrate?”

> *Testing: I versus we.* Your honest number, and your role in the programme.

Partner: “Your Kafka role was four months. What did you actually build?”

> *Testing: early-career honesty.* The internship framing, and only the metrics you can explain.

Partner: “Tell me about a time a number you produced was wrong.”

> *Testing: accountability.* The broadcast OOM, or any real data error you caught. Own it and show the systemic fix.

Partner: “Last one. Why should I put you in front of an Australian bank next month?”

> *Testing: consulting readiness.* “Because I’ve spent three years making regulatory numbers correct under a hard deadline, and I explain trade-offs in business terms. I’d be honest with them about risk, the way I’ve been honest with you today about exactly what I did.”

Scorecard for Mock D

Signal	Strong	Weak
Precision	Exact about own role and numbers	Vague, or inflated then retracted under pressure
Pronouns	“I” for own decisions, “we” for team outcomes	“We” everywhere, or “I” for everything

Signal	Strong	Weak
Evidence	Baseline, method and source for numbers	“It was around that” with no method
Composure	Corrects the resume calmly when needed	Defends a number they can’t support
Brevity	Adjusts length when interrupted	Restarts the long version

Mock E (The full partner conversation, 75 minutes)

A realistic 75-minute conversation, written with both sides. Practise it out loud; better still, have someone read the partner’s lines and interrupt you. Your answers are summarised here where the full script already appears earlier; practise the full version.

Partner: “Thanks for making the time, Vishal. I’ve heard good things from the earlier rounds. I’m a partner here in Sydney. I’d like to understand your background, work through a problem or two together, and leave time for your questions. Sound good?”

You: “That sounds great. Thank you for making the time, especially at this hour in Sydney.”

Partner: “Let’s start with you. Tell me about yourself.”

You: Give the 2-minute version (Part 4).

Partner (interrupting): “Great. If I had to describe you to a client in one sentence?”

You: “A data engineer who makes a bank’s regulatory numbers correct, on time and cheaper to produce, and who can explain how.”

Partner: “Your resume says Pod Lead. How many people do you manage?”

You: “None directly, and I want to be precise about that. My title is Data Engineer II. Pod Lead is technical leadership: code reviews, releases, technical decisions and mentoring for the pod, without direct reports.”

Partner: “Appreciated. Tell me about the $80K.”

You: Lead with the verified evidence (22.2): the 170-run workload, CPU under 19% but 133 GiB memory, the projected 50–65% reduction, and the heavy satellite job you deliberately didn’t downsize. Quote $80K only if you can derive it.

Partner: “What got worse when you shrank the clusters?”

You: “Headroom. Smaller clusters leave less room for data growth, so we watch runtime trends on critical-path pipelines and resize before they threaten the deadline.”

Partner: “Let’s do a problem. A client, a mid-size Australian bank, says their monthly finance report is slow and doesn’t match their source system. How would you approach it?”

You: “Can I check two things first? Is the mismatch consistent every month, and roughly how big is it?”

Partner: “Every month, about half a percent. Revenue is overstated.”

You: “Overstated and consistent points to something systematic, like duplicates or a join at the wrong grain inflating totals. I’d take correctness first, because the same cause often explains the slowness too. First, agree the definitions: which statuses, what cut-off time, which time zone. Then reconcile top-down: totals, then by day or product, to narrow where the difference is, then row by row with a full outer join. If I find a many-to-many join, fixing the grain fixes both problems at once: the numbers match, and the query stops creating a huge intermediate result. Then I’d add a reconciliation check to the pipeline so it can’t recur silently.”

Partner: “Explain that to their CFO in two sentences.”

You: “The report was counting some transactions twice because two data sets were combined at the wrong level of detail. We fixed it, the numbers now match your ledger, and the report runs faster.”

Partner: “Good. Now write me a query. Transactions and accounts. I want each account’s end-of-day balance for every day in September, including days with no transactions.”

You: Clarify (opening balance, credit and debit signs, posted only, time zone), then build the date spine from each account’s open date, a running sum, and filter to September at the end. *(See Part 9, Case 7.)*

Partner: “Now it’s 10 million accounts and it runs nightly.”

You: “Then I wouldn’t recompute history every night. I’d store a daily balance snapshot and compute today’s balance as yesterday’s plus today’s net movement. That processes one day of transactions instead of all history, and the date filter prunes partitions.”

Partner: “Why did your team choose Data Vault instead of a dimensional model?”

You: Give the Data Vault versus dimensional answer (Part 25), and end with: “For a client with a few stable sources, I’d recommend Kimball directly.”

Partner: “Did you choose it?”

You: “[Your precise answer about what existed before you and what you shaped.]”

Partner: “Tell me about a time you got something wrong.”

You: The broadcast OOM (22.15).

Partner: “Suppose a client insists on a tool you think is the wrong choice. What do you do?”

You: “Make sure they’re choosing with full information: the trade-offs in cost, risk and time. Ask what’s driving the preference, because they may know something I don’t. If they still choose it, it’s their call. I document the risk and make their choice work as well as possible.”

Partner: “Why Vivanti, and not staying in banking?”

You: Give the “Why Vivanti?” answer (Part 5), in 60 seconds.

Partner: “What are your salary expectations?”

You: “I’d like to understand the level you have in mind first. Broadly, for the Bengaluru market with my Databricks and banking experience, around 30 to 35 LPA, and I’m flexible on the structure.”

Partner: “HR will pick that up. What questions do you have for me?”

You: “What would make someone in this role a clear success in their first six to twelve months, from your point of view?”

(Listen, and respond briefly to the answer.)

You: “And what separates the consultants who really thrive at Vivanti from those who are just good engineers?”

Partner: “Anything else you’d like me to know?”

You (closing statement): “Just that I’m genuinely excited about this. I’ve spent close to four years making a regulated platform reliable, fast and cheaper, and I want to bring that to Vivanti’s clients. I’m honest about what I know and what I don’t, and I learn quickly. Thank you for your time today. I’ve really enjoyed the conversation.”

Scoring Mock E

Question	Yes / No
Did every answer start with a clear headline?	
Were you precise about your title and role?	
Did you use only numbers from your fact sheet?	
Did you clarify before solving the problem?	
Did you give the executive version in two sentences?	
Did you shorten when interrupted?	
Did you avoid going silent for more than 20 seconds?	
Did you ask two strong questions and respond to the answers?	

Mock F (Tim’s style: modeling plus Snowflake SQL, 45 minutes)

This is the most likely shape of the real round. Run it out loud with a timer, sharing your screen in a plain editor. Better still, have someone read Tim’s lines and interrupt.

Tim: “Let’s do something hands-on. Imagine a university client. They want to understand enrolments and how students progress. Where would you start?”

You: Business questions first (Part 11, Exercise A, step 1). Ask him for two or three that matter most to the client. Then declare the processes and grains.

Tim (interrupting): “Keep it simple. One fact table.”

You: “If we only have one, I’d pick the course enrolment grain, one row per student per course offering, because it answers the most questions. But headcount and retention need a distinct count, and time-to-degree needs a different shape, so I’d flag those as the first things to add.” *(He’s testing whether you’ll defend grain under pressure without being rigid.)*

Tim: “Residency can change. How do you model that?”

You: SCD2 on dim_student, and a point-in-time join on the census date (Part 12, Case U1).

Tim: “Write me the query: headcount by faculty and residency for this term.”

You: Case U1, narrating the COUNT(DISTINCT) and the faculty double-counting trap.

Tim: “Now retention for last year’s commencing cohort.”

You: Case U2. Ask about the definition first: “Do students who completed in their first year count as retained, or are they excluded?”

Tim: “The query’s slow. It’s on Snowflake. What do you do?”

You: Query Profile: partitions scanned versus total, spilling, exploding joins. Check the date filters are sargable. Fix the query before resizing the warehouse (Part 10).

Tim: “Explain what you’ve built to the university’s Deputy Vice-Chancellor, in thirty seconds.”

You: “We’ve built one trusted source for enrolment and progression. It shows how many students you have each term, how many continue into second year, and which courses they struggle in, all from the same definitions finance and faculties agree on. So discussions become about what to do, not whose numbers are right.”

Tim: “Last thing. Device telemetry arrives as JSON with duplicates. How do you land it?”

You: VARIANT landing, FLATTEN, dedupe on device and event ID with QUALIFY, incremental via stream and task (Part 12, Case D1; Part 10).

Scoring Mock F

Question	Yes / No
Did you ask for business questions before designing?	
Did you declare the grain of every fact out loud?	
Did you handle “one fact only” calmly, naming the trade-off?	
Did you use COUNT(DISTINCT) for headcount and explain why?	
Did you clarify the retention definition before writing?	
Did you go to the Query Profile before warehouse size?	
Was the executive version under 30 seconds and jargon-free?	
Did you narrate throughout, never silent more than 20 seconds?	

Part 30: Revision Sheets, Numbers Card and Interview Day

Sheet 1: Personal introduction

One sentence: “A data engineer who makes a bank’s regulatory numbers correct, on time and cheaper to produce, and who can explain how.”

30 seconds: Data Engineer II and Pod Lead at TD → own a regulatory platform, about 150 pipelines, 6 AM deadline, regulators → relocating to Bengaluru → want to take production depth to many clients.

2 minutes, in five beats:

1. Identity: DE II and Pod Lead, close to four years, started in streaming.
2. Platform: config-driven Databricks lakehouse, Data Vault, 15+ sources, 150+ workloads, 6 AM, OSFI and others.
3. Proud of: evidence-based right-sizing (projected 50–65% less compute on an over-provisioned job; refused to downsize a heavy one), Spark tuning, Legendary Award; cross-team migrations.
4. Leadership: no direct reports; reviews, releases, mentoring; pod deploys without me.
5. Why here: consulting, breadth, Snowflake and GCP, Bengaluru.

Sheet 2: Achievements and numbers (only these)

Claim	Say it like this
Pipelines	“about 150 production pipelines”
Sources	“fifteen-plus source systems”
Deadline	“hard 6 AM regulatory deadline, nightly chain from about 2 AM”
Right-sizing	“170 runs, 99.4% success, P95 32 minutes, CPU under 19%, memory 133 GiB on 32-core, 256 GB nodes → smaller profile, +15–25% runtime, −50–65% compute (projected)”
Counter-case	“51 runs, 94.1% success, P95 112 minutes, 131 GiB memory, 65 window functions → kept E32, reduced workers conservatively”
Workloads	“150+ production workloads, about 161 curation jobs, 48 reporting outputs, 35 FRP link jobs”
Performance	“shuffle from about 500 GB to 200 GB (Spark UI); average runtime down about 60% (run logs)”
Synapse	“30-plus-minute queue waits removed”
Onboarding	“roughly 75% less effort for a new source”
Defects	“about 20 fixed upstream, 12 or 13 closed on their own”
Liquid Clustering	“teams with 50 to 70 pipelines each; six-week deadline, done in week five, no downtime”
Awards	“ten recognition awards from six colleagues, including a Legendary Award”
Writing	“290-plus articles, about 8,500 readers”
Never say	15 PB, 99.9%, “architected” (unless true), “trained” agents, the 30% QE figure, “manage a team”

Sheet 3: Story index (which story for which question)

If they ask about…	Use
Achievement, impact, award	22.1 Legendary Award, or 22.2 Right-sizing
Using evidence, pushing back, cost vs reliability	22.21 Rejecting aggressive downsizing
Process improvement, Delta optimisation	22.22 Delta optimiser framework
Concurrency, platform failures	22.23 ConcurrentAppendException
Technical depth, hardest problem	22.3 Spark optimisation, or 22.14 Synapse
Failure, mistake, accountability	22.15 Broadcast OOM
Incident, pressure, missed deadline	22.13 The UNRESOLVED_COLUMN SLA miss
Conflict, disagreement, challenging seniors	22.16 Phased migration
Influence without authority, cross-team	22.19 Liquid Clustering
Ambiguity, prioritisation	22.17 The 50 defects
Mentoring, leadership, teamwork	22.18 and 22.6 Deployment training
Initiative, process improvement	22.20 Reference-table diff tool
Quality, correctness	22.7 The DQ engine
Feedback received, growth	Lyft feedback (Part 23, question 17)
Legacy modernisation, AI use	22.9 Grounded Copilot agents (fill first)

Sheet 4: Technical revision

- **Spark performance ladder:** measure → read less → move less → compute less → fix layout → scale compute last.
- **Joins:** broadcast small tables; sort-merge for two large ones; watch memory on broadcasts.
- **Skew:** one long task; fix with AQE, broadcast or salting.
- **Delta:** transaction log, ACID, time travel, MERGE, OPTIMIZE, VACUUM (limits time travel), Liquid Clustering.
- **Data Vault:** hubs (keys), links (relationships), satellites (history); hash keys, hashdiff, insert-only, record source.
- **DQ:** complete, accurate, timely, unique; block or warn by severity; quarantine.
- **Streaming:** event time, watermarks, checkpoints, idempotent sinks; batch as source of truth.
- **CDC:** log-based capture; ordering by log position; deletes; MERGE.
- **Cost:** job clusters, right-sizing to the critical path, auto-termination, incremental processing.

Sheet 5: Consulting revision

- Understand the goal before the solution.
- Clarify only what changes the answer, then commit to assumptions out loud.
- Options with trade-offs, then a recommendation. The client decides.
- Bad news early, with a plan.
- Scope changes are fine if they’re visible and chosen.
- Never disagree with a colleague in front of the client.
- Plan the handover from day one.
- Impact, likelihood, options, recommendation: the four-part risk explanation.

Sheet 6: Partner-round cheat sheet

- **Five messages:** production correctness · business outcomes · influence and teaching · honesty · consulting-ready.
- **Answer shape:** headline → impact → depth → your role → stop.
- **SQL talk track:** clarify → define → assume → plan (grain) → write CTE by CTE → validate → extend (alternative and scale).
- **Executive switch:** two sentences, no jargon, lead with the outcome.
- **Never silent for more than 20 seconds.** Narrate your thinking.
- **If you don’t know:** say so, reason from principles, say how you’d verify.

Sheet 7: Compensation script

1. Level first: “What level do you have in mind, and the range for it?”
2. Number: “Around 30 to 35 LPA, flexible on structure.”
3. Current pay: “Paid in CAD; a conversion doesn’t reflect this market.”
4. Other offer: “A verbal offer from a large IT services firm, being finalised. Vivanti is my preference.”
5. Minimum: never reveal.
6. Counter: one specific number, justified by value.
7. Time: “Could I have until [date] to review?”

Sheet 8: Questions to ask

1. “What would make someone in this role a clear success in six to twelve months?”
2. “What problems are clients bringing most often right now?”
3. “What separates consultants who thrive here from those who are just good engineers?”

Backup: Bengaluru and Australia collaboration · Databricks practice direction · an engagement they’re proud of.

Sheet 9: The final 15 minutes before the call

1. Read Sheet 2 (your numbers) once, out loud. **3 min**
2. Say your one-sentence and 30-second introductions. **2 min**
3. Read the SQL talk track and the answer shape (Sheet 6). **2 min**
4. Say the broadcast OOM story in 60 seconds. **2 min**
5. Glance at your three questions (Sheet 8). **1 min**
6. Water, camera, microphone, plain editor open, notifications off. **3 min**
7. Breathe slowly. You've cleared every round so far. **2 min**

Sheet 10: Snowflake and modeling on one page

Positioning: “My production work is Databricks, not Snowflake. The concepts transfer, and I’d be productive quickly.”

Snowflake	= Databricks
Micro-partitions + pruning	Delta files + data skipping
Clustering key	Liquid Clustering / Z-order
Virtual warehouse (scale up = bigger; scale out = multi-cluster)	Job cluster / SQL warehouse
Time Travel (+ Fail-safe)	Delta time travel
Zero-copy clone	Shallow clone
Stream + task / Dynamic Table	Change Data Feed + Workflows / DLT
Snowpipe	Auto Loader
VARIANT + : + FLATTEN	from_json + explode
Masking and row access policies	Unity Catalog masks and row filters

Performance order: Query Profile → pruning (scanned vs total) → sargable filters → spilling → exploding joins → then warehouse size.

Modeling in seven steps: business questions → processes → **grain** → dimensions → facts (additivity) → history (SCD1 or 2 per attribute) → validate against the questions.

Fact types: transaction (events) · periodic snapshot (trends, balances) · accumulating snapshot (durations, milestones) · factless (coverage, “what didn’t happen”).

Traps to name out loud: distinct counts don’t add up · never average averages · survivorship (only finished cases) · zero rows missing without a spine · point-in-time joins for SCD2 · PK/FK not enforced on Snowflake.

The numbers card

Write this on one card and memorise it. **Only numbers you can defend go on it.** Every number you say in the interview must come from this card.

Claim	Number to say	Source / method	Confident?
Pipelines	150+	Platform inventory	Yes
Sources	15+	Copilot search	Yes
Deadline	6 AM, nightly chain from about 2 AM	Platform design	Yes
Regulators	OSFI, Bank of Canada, CDIC, Statistics Canada	Platform scope	Yes
Data volume	[YOU FILL: 15 PB total? nightly volume?]	[YOU FILL]	Settle
SLA record	[YOU FILL: exact misses in the period]	[YOU FILL]	Settle
Right-sizing	Verified case: projected −50–65% compute; counter-case kept E32	Copilot search	Yes
$80K a year	Resume figure	Not verified	Only with a calculation
Runtime	About 60% average reduction	Run logs. [YOU FILL: which pipelines]	Settle scope
Shuffle	About 500 GB to 200 GB	Spark UI. [YOU FILL: which job]	Settle scope
Broadcast threshold	[10 or 20] MB to about [70–80] MB	Your change	Pick one
Onboarding	About 75% less effort	[YOU FILL: days before and after]	Settle
Staleness	25% reduction	[YOU FILL: definition]	Settle
Defects	50 open, about 20 fixed upstream, 12–13 closed automatically	Jira	Yes
Liquid clustering	Teams with 50–70 pipelines each	Tracker	Yes. Pick one timeline
Synapse	30+ minute queue waits removed, reads 30–40% faster	[YOU FILL: dev or prod]	Settle
Mentoring	10+ engineers	[YOU FILL: names and roles]	Settle
Awards	Ten TD awards from six colleagues, including the Legendary Award (Varun Khanna)	Award records	Yes
Medium	290+ articles, 8,500+ subscribers	Medium	Yes
Stack Overflow	1 gold, 5 silver, 17 bronze badges; about 53,000 reached	Profile	Yes

Claim	Number to say	Source / method	Confident?
Experience	About 3 years 8 months by listed dates	Dates	Say it precisely
Never claim	The 30% QE pipeline improvement (Radhey's work)		Never

The fill-in worksheet (do this first on Day 1)

Answer each of these in one or two sentences. Most take minutes, because you already know the answers. **This hour is the highest-value hour of your preparation.**

1. What makes up "4+ years"?
2. Your TD start date, and how it fits with Lambton ending in August 2023.
3. What you did between April 2019 and September 2021.
4. What the 15 PB figure measures, and how much data the nightly chain processes.
5. The date of the UNRESOLVED_COLUMN SLA miss, how 99.9% is calculated, and whether any prevention was actually put in place afterwards.
6. The systemic fix after that incident.
7. What existed before you joined, and what you designed.
8. Your exact role in the Data Vault 2.0 "transition", and which hash function you use.
9. The $80K calculation, if one exists (the Copilot search didn't find it). Also: was the right-sizing recommendation implemented, and what was the realised result?
10. The Legendary Award citation, in its actual words.
11. Which pipelines the 60% and 500-to-200 GB figures cover, and the baselines.
12. The broadcast threshold before and after (one version).
13. The Liquid Clustering timeline (one version), and how you handled ConcurrentAppendException.
14. How SQL Server control tables and JSON/YAML configs divide the work.
15. The onboarding "75%": days before and after.
16. How staleness was measured for the 25%.
17. How many ADF pipelines are migrated today, and whether the migration is complete.
18. Whether the Synapse-to-ADLS materialisation was used in production.
19. The legacy modernization: team size, your role, tables done, validation method.
20. What you personally built at Mandelbulb, and which metrics you can explain.
21. Where the portfolio project runs: AWS, local MinIO, or both.
22. For each skill in the "unverified" row of the skills table in Part 18, where you used it.
23. One mentee story, with a name and outcome.
24. What digital.ai does in your release process.
25. Your gold Stack Overflow badge, and your two most-read Medium articles.

> Send me your answers to this worksheet and I'll turn them into final, polished answers for every deep dive, with no brackets left. That's the single most useful thing we can do next.

How the round will likely run

- A video call, probably with a shared document or code editor where you type SQL. **You probably won't be able to run the query**, so practise writing SQL in a plain editor without autocomplete this weekend.
- The partner may describe tables verbally. Write the schema at the top of the document as you hear it. It shows structure and saves you from mistakes.
- Expect interruptions and "what if" changes mid-query. That's the test, not a problem.

If you get stuck

1. **Say where you are:** "I have the per-customer totals. Now I need to compare each month with the previous one."
2. **Simplify:** "Let me first solve it for a single customer, then generalise."
3. **Use a tiny example:** write three rows and the output you expect.
4. **Ask for a hint the smart way:** "I'm choosing between a self-join and LAG. Is there a constraint I should know about?" The recruiter said hints are welcome, so it isn't a penalty.
5. **Never go silent.**

Interview-day execution plan

What to review before the interview

- **Morning:** Sheets 1, 2 and 3; two SQL cases solved out loud; the story list.
- **One hour before:** Sheet 6 and Sheet 7. No new material.
- **Fifteen minutes before:** Sheet 9.

Virtual setup checklist

- Laptop charged and plugged in; stable internet, with a phone hotspot as backup.
- Camera at eye level, light in front of you, plain and tidy background.
- Headphones with a microphone, tested on a call beforehand.
- The meeting link opened 10 minutes early; the right app or browser already signed in.
- A plain text editor open (no autocomplete) plus paper and pen, for SQL.
- Screen sharing tested. Close everything else, especially notes you shouldn't be reading.
- Phone on silent and out of reach; notifications off.
- Water within reach.

Dress and presentation

A plain collared shirt, solid colour, neat. Consulting partners expect smart business casual on video. Avoid busy patterns, which flicker on camera.

Body language

- Sit upright, slightly forward, with shoulders relaxed.

- Look at the camera when making a key point.
- Nod while listening; don't interrupt.
- Keep your hands still or use small gestures; no fidgeting with pens.

Speaking pace

- Aim for about 130 words a minute: slower than normal conversation. Nerves speed you up.
- Pause between sections of an answer.
- Short sentences. If you notice a long sentence, stop and restart it shorter.

How to pause and structure an answer

- "That's a good question. Let me take a second." Then **actually take two seconds**.
- Then signpost: "There are three parts to this…" and give three.
- End with a one-line summary: "So in short, correctness first, then performance."

How to handle a question you don't know

> "I don't know that one precisely. Here's how I'd reason about it: [principle]. And I'd verify by [how]."

Never guess a fact, a number or a feature as if you're sure.

How to recover after a weak answer

- Don't dwell or apologise repeatedly. Everyone has one weak answer.
- If it matters, come back to it briefly later: "Coming back to the earlier question about X, a clearer way to put it is…"
- Make the next answer structured and calm. Partners remember the overall pattern, not one moment.

How to close strongly

When they ask "anything else?", use the closing statement.

Exact closing statement:

> "Just that I'm genuinely excited about this role. I've spent close to four years making a regulated platform reliable, faster and cheaper, and I'd love to bring that to Vivanti's clients. I'm honest about what I know and what I don't, and I learn quickly. Thank you for your time today. I've really enjoyed the conversation."

Exact thank-you at the end of the call:

> "Thank you very much for your time, and for the questions. I've learned a lot about Vivanti today. I look forward to hearing about next steps."

After the call

- Within a few hours, email Anisha a short thank-you, and ask her to pass your thanks to the partner.
- Write down every question you were asked, and what you'd improve, while it's fresh.
- If there's no update after five to seven working days, send one polite follow-up.